



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Universidad Autónoma de Nuevo León

Facultad de Ingeniería Mecánica y Eléctrica

Lenguajes de Programación y Lab

Actividad Fundamental 4

Unidad 1

Semestre: 3ro

Equipo: 04

Grupo: 025

Salón: 3109

Periodo: Enero – Junio

Docente: Ing. Selene Guadalupe Pinal Torres

Hora: M4-M6

Nombre	Matricula	Carrera
Ruedas Vazquez Jordan Emanuel	2103677	ITS
Enríquez Nungaray Héctor Miguel	2042781	IAS
Hernández Silva Carlos Yahir	1975208	ITS
Heredia López Betsy Aracely	2054413	IAS

Martes 11 de Febrero del 2025

Monterrey, Nuevo León

Contenido

Introducción	2
Diagramas de Flujo.....	3
Tipos de Datos (Lenguajes de Programación).....	8
☐ Tipos de datos en Python	9
☐ Tipos de datos en JavaScript	10
☐ Tipos de datos en Angular (TypeScript)	11
☐ Tipos de datos en HTML	12
Tipos de Estructuras de Datos	15
Estructuras Secuenciales	16
<i>Estructura Simple.....</i>	<i>23</i>
<i>Estructura Doble</i>	<i>26</i>
<i>Estructura Multiple</i>	<i>29</i>
Contadores.....	32
Ciclos	35
<i>While</i>	<i>43</i>
<i>Do – While</i>	<i>49</i>
<i>Estructura desde para (For)</i>	<i>52</i>
Arreglos	57
<i>Unidimensionales:.....</i>	<i>57</i>
<i>Multidimensionales:</i>	<i>62</i>
<i>Matrices y Vectores.....</i>	<i>65</i>
Pilas	70
<i>Aplicaciones de Pilas (Ventajas Y Desventajas)</i>	<i>74</i>
Colas	75
<i>Colas Circulares</i>	<i>77</i>
<i>Doble Cola</i>	<i>79</i>
<i>Aplicaciones de Colas (Ventajas y Desventajas)</i>	<i>82</i>
Listas.....	84
<i>Listas simplemente ligadas.....</i>	<i>84</i>
<i>Listas Circulares.....</i>	<i>75</i>
<i>Listas Doblemente Ligadas</i>	<i>81</i>
<i>Listas Doblemente Ligadas Circulares.....</i>	<i>88</i>
<i>Aplicaciones de Listas (Ventajas y Desventajas)</i>	<i>94</i>

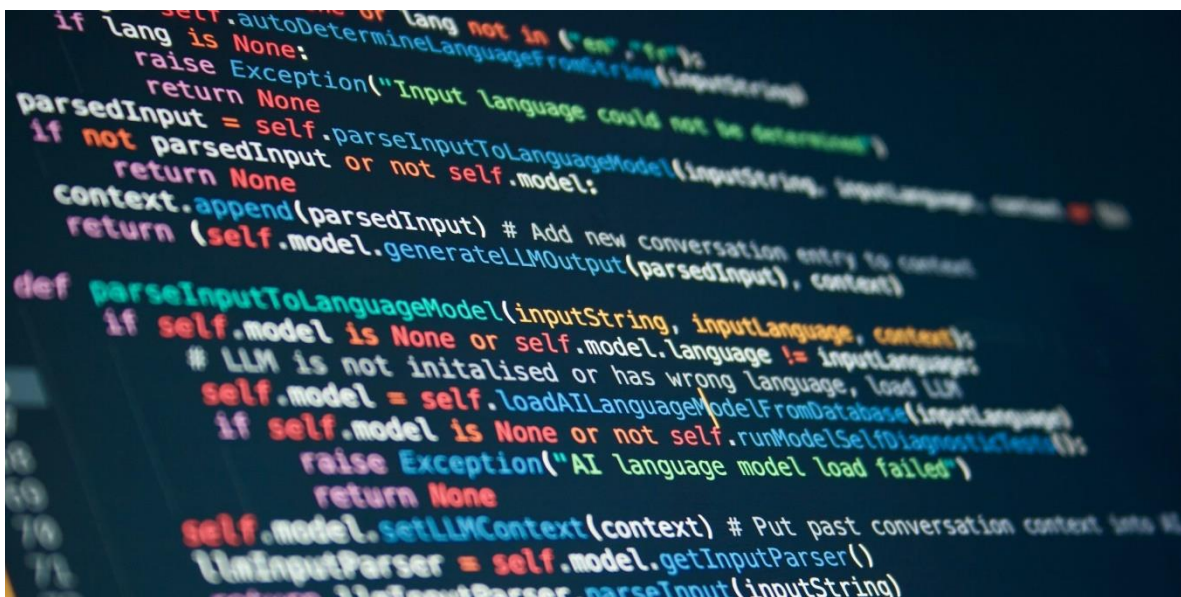
<i>Árboles</i>	95
<i>Árboles Multi-caminos</i>	100
<i>Graficas</i>	102
<i>Graficas Dirigidas</i>	102
<i>Graficas No Dirigidas</i>	104
<i>Métodos de Ordenación</i>	105
<i>Ordenación Interna</i>	105
<i>Ordenación Externa</i>	110
<i>Métodos de Búsqueda</i>	115
<i>Búsqueda Interna</i>	115
<i>Búsqueda Externa</i>	120
<i>Programación Orientada a Objetos</i>	132
<i>Conclusión General</i>	140
<i>Conclusiones Individuales</i>	145
<i>Referencias Bibliográficas</i>	146

Introducción

La programación es una disciplina fundamental en el desarrollo de software y sistemas informáticos, permitiendo la creación de soluciones para diversas problemáticas a través de la implementación de algoritmos. En este contexto, existen conceptos esenciales que todo programador debe comprender para estructurar y desarrollar código de manera eficiente. Entre estos conceptos se encuentran las variables, que permiten almacenar y manipular datos; las estructuras de control, que dirigen el flujo de ejecución del programa; y las estructuras de datos, que optimizan la organización y el acceso a la información. Cada uno de estos elementos es crucial para la construcción de programas funcionales y bien estructurados.

Además, la programación orientada a objetos (POO) representa un paradigma clave que facilita la organización del código mediante la definición de clases y objetos, promoviendo la reutilización y modularidad del software. Asimismo, la resolución de problemas es un aspecto fundamental dentro del desarrollo de software, ya que requiere que los programadores diseñen soluciones lógicas y eficientes para situaciones complejas. Para lograrlo, es esencial el uso de herramientas de programación que permitan escribir, probar y depurar el código, asegurando su correcto funcionamiento y optimización.

A pesar de que existen múltiples lenguajes de programación, como Python, C++, C y Java, los conceptos básicos mencionados son comunes en todos ellos, lo que permite a los programadores adaptarse con mayor facilidad a diferentes entornos de desarrollo. Cada lenguaje presenta sus propias sintaxis y características particulares, pero la lógica de programación y los principios fundamentales siguen siendo aplicables en cualquier plataforma. Por esta razón, comprender estos conceptos básicos no solo facilita el aprendizaje de nuevos lenguajes, sino que también mejora la capacidad de diseñar algoritmos eficientes y estructurados para resolver problemas de manera efectiva.



```
self.autoDetermineLang not in ("en", "es")
if lang is None:
    raise Exception("Input language could not be determined")
    return None
parsedInput = self.parseInputToLanguageModel(inputString, inputLanguage, None)
if not parsedInput or not self.model:
    return None
context.append(parsedInput) # Add new conversation entry to context
return (self.model.generateLLMOutput(parsedInput), context)

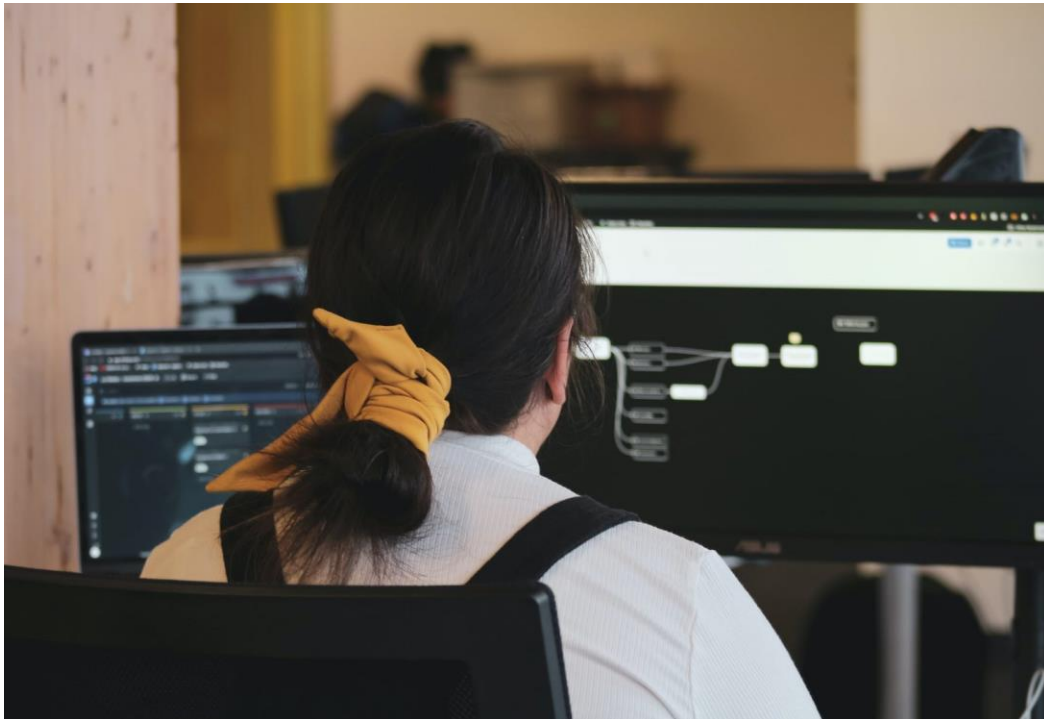
def parseInputToLanguageModel(inputString, inputLanguage, context):
    if self.model is None or self.model.language != inputLanguage:
        # LLM is not initialised or has wrong language, load LLM
        self.model = self.loadAILanguageModelFromDatabase(inputLanguage)
        if self.model is None or not self.runModelSelfDiagnosticTest():
            raise Exception("AI language model load failed")
            return None
        self.model.setLLMContext(context) # Put past conversation context into it
    llmInputParser = self.model.getInputParser()
    return llmInputParser.parseInput(inputString)
```


Diagramas de Flujo

Los diagramas de flujo son una herramienta fundamental en la representación de algoritmos, ya que permiten visualizar de manera gráfica la secuencia de pasos que debe seguir un programa para resolver un problema. Su uso es esencial en el desarrollo de software y en la enseñanza de la lógica de programación, ya que facilita la comprensión de los procesos involucrados en la ejecución de un algoritmo. Gracias a la simbología estandarizada utilizada en estos diagramas, es posible representar operaciones, decisiones, entradas, salidas y ciclos de manera clara y ordenada.

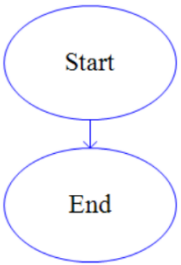
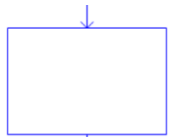
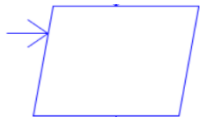
Dentro de la simbología de los diagramas de flujo, cada símbolo tiene un propósito específico que contribuye a la estructura y lógica del algoritmo. Algunos de los símbolos más importantes incluyen el óvalo para representar el inicio y el fin del programa, el paralelogramo para las operaciones de entrada y salida, el rectángulo para la asignación de valores o cálculos, el rombo para la toma de decisiones y el hexágono para representar ciclos. La correcta aplicación de estos símbolos permite a los programadores y analistas diseñar soluciones eficientes antes de implementarlas en un lenguaje de programación.

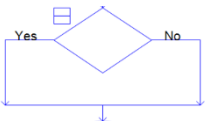
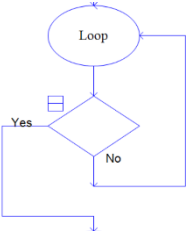
La relación entre los diagramas de flujo y la codificación en lenguajes como C++ es directa, ya que estos diagramas pueden traducirse en estructuras de control como condicionales, bucles y funciones dentro de un código fuente. Comprender la simbología utilizada en los diagramas de flujo es, por tanto, un paso clave en la formación de cualquier programador. En este trabajo, se explorará en detalle la simbología de los diagramas de flujo y su aplicación en la codificación, proporcionando ejemplos en C++ para ilustrar cómo se traducen estos elementos gráficos en código funcional.



En la representación de algoritmos, el uso de símbolos en diagramas de flujo es clave para estructurar de manera visual los pasos que seguirá un programa. Cada símbolo tiene un propósito bien definido que permite representar de manera clara las operaciones fundamentales de un algoritmo, facilitando su comprensión y depuración antes de su implementación en un lenguaje de programación. Los diagramas de flujo son especialmente útiles en la fase de diseño, ya que permiten identificar posibles errores en la lógica del programa y optimizar su estructura antes de codificarlo.

A continuación, se presentará una descripción detallada de cada uno de los símbolos principales utilizados en los diagramas de flujo (*simbología de Raptor Avalonia*). Se explicará el propósito de cada símbolo, su función dentro de la estructura del algoritmo y se proporcionarán ejemplos en C++ para ilustrar cómo se traduce cada uno de estos elementos gráficos en código funcional. La relación entre diagramas de flujo y programación es crucial para entender cómo los conceptos abstractos de un algoritmo se implementan en un lenguaje de programación real.

Simbología	Descripción	Ejemplo (Codigo)	Lenguaje de Programación
Inicio / Fin 	El símbolo de inicio y fin se representa con un óvalo y marca el punto donde comienza y termina el algoritmo. Todo diagrama de flujo debe contener un único inicio y al menos un fin. El inicio no tiene entrada de datos, mientras que el fin señala el término del proceso, asegurando que no hay más instrucciones por ejecutar.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 cout << "Inicio del programa" << endl; 5 6 // Código del programa 7 cout << "Fin del programa" << endl; 8 return 0; 9 } </pre>	
Asignación/Assignment 	Este símbolo se representa con un rectángulo y se utiliza para definir o modificar el valor de una variable. En un algoritmo, las asignaciones permiten almacenar resultados intermedios y realizar cálculos. Es un paso fundamental para cualquier proceso computacional, ya que permite manipular datos y guardar información relevante para la ejecución del programa.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 int a, b, suma; 5 a = 5; // Asignación de valores 6 b = 10; 7 suma = a + b; // Operación y asignación 8 cout << "La suma es: " << suma << endl; 9 return 0; 10 } </pre>	
Entrada/Input 	El símbolo de entrada se representa con un paralelogramo inclinado hacia la derecha y se usa para recibir datos del usuario. En un diagrama de flujo, es fundamental para interactuar con el usuario y permitir que el	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 int numero; 5 cout << "Ingresa un número: "; 6 cin >> numero; // Entrada de datos 7 cout << "El número ingresado es: " << numero << endl; 8 return 0; 9 } 10 } </pre>	

	algoritmo funcione con diferentes valores según la necesidad.		
Salida/Output 	El símbolo de salida también es un paralelogramo y se usa para mostrar resultados en pantalla o imprimir información relevante. En un programa, la salida es esencial para comunicar los resultados de las operaciones realizadas por el algoritmo.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 int resultado = 25; 5 cout << "El resultado es: " << resultado << endl; // Salida de datos 6 return 0; 7 } 8 </pre>	
Decisión/Selection 	El símbolo de decisión se representa con un rombo y permite evaluar condiciones lógicas dentro de un algoritmo. En función de si la condición es verdadera o falsa, el flujo del programa tomará un camino diferente. Es la base de las estructuras condicionales como if, else y switch.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 int numero; 5 cout << "Ingrese un número: "; 6 cin >> numero; 7 if (numero % 2 == 0) { // Evaluación de condición 8 cout << "El número es par" << endl; 9 } else { 10 cout << "El número es impar" << endl; 11 } 12 return 0; 13 } </pre>	
Llamada a Subproceso/Call 	Este símbolo se representa con un rectángulo con una flecha de señalamiento y se usa para indicar la ejecución de una función o subproceso dentro del algoritmo. Los subprocesos ayudan a dividir un programa en secciones reutilizables, mejorando su organización y mantenimiento.	<pre> 1 #include <iostream> 2 using namespace std; 3 // Función para calcular el cuadrado de un número 4 int cuadrado(int num) { 5 return num * num; 6 } 7 int main() { 8 int valor; 9 cout << "Ingrese un número: "; 10 cin >> valor; 11 // Llamada a subproceso 12 cout << "El cuadrado de " << valor << " es: " << cuadrado(valor) << endl; 13 return 0; 14 } </pre>	
Ciclo/Bucle/Loop 	El símbolo de bucle (loop) en Raptor se representa con un hexágono y se usa para repetir un conjunto de instrucciones mientras una condición se cumpla. Se pueden utilizar bucles while, for o do-while, dependiendo del caso de uso. Son esenciales en procesos iterativos y automatización de tareas repetitivas.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 int contador = 1; 5 // Bucle que se ejecuta mientras la condición sea verdadera 6 while (contador <= 5) { 7 cout << "Iteración: " << contador << endl; 8 contador++; 9 } 10 return 0; 11 } </pre>	

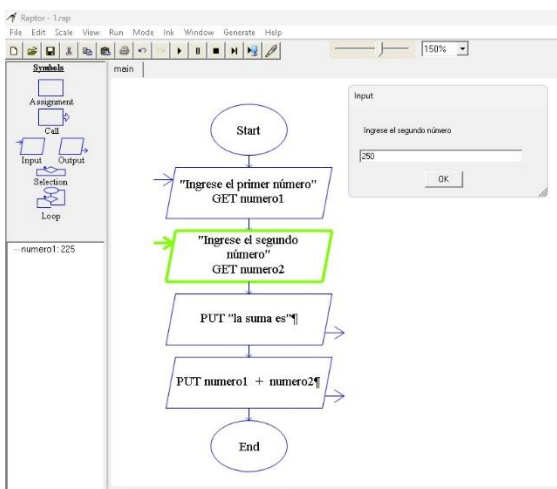
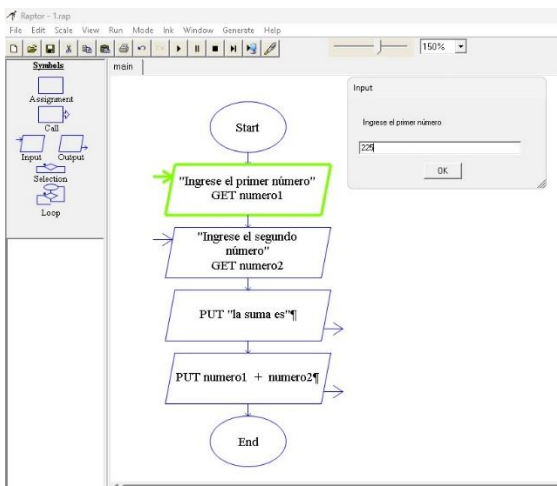
Comentario/Comment 	El símbolo de comentario se usa para añadir notas explicativas dentro del diagrama de flujo. No afecta la ejecución del programa, pero es útil para mejorar la comprensión del código.	<pre> 1 #include <iostream> 2 using namespace std; 3 int main() { 4 // Este programa calcula la suma de dos números 5 int a = 10, b = 20, suma; 6 suma = a + b; // Suma de los números 7 cout << "La suma es: " << suma << endl; // Muestra el resultado 8 return 0; 9 } </pre>	
--	---	---	---

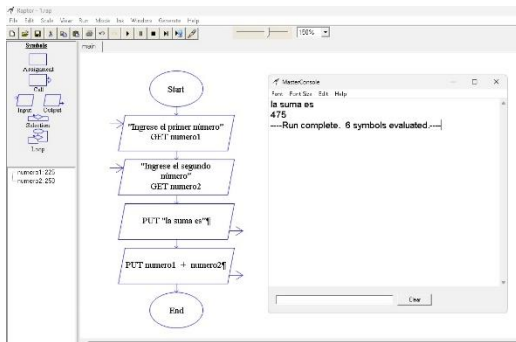
Ejemplo: Este algoritmo en RAPTOR solicita al usuario ingresar dos números, luego los suma y muestra el resultado. Los pasos son los siguientes:

Entrada de datos: El programa pide al usuario que ingrese dos números. Los valores se almacenan en las variables numero1 y numero2.

Proceso de suma: El algoritmo suma los dos números ($\text{numero1} + \text{numero2}$) y guarda el resultado en la variable suma.

Salida de datos: El resultado de la suma se muestra en la pantalla mediante un mensaje que indica "La suma es: " seguido del valor calculado.

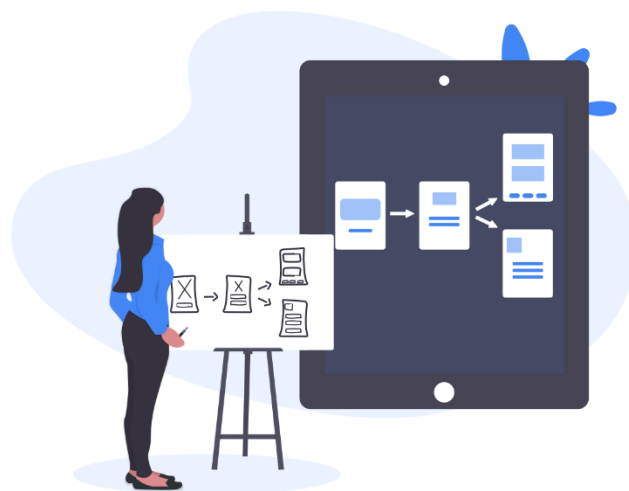




El uso de la simbología en los diagramas de flujo es esencial para representar de manera clara y estructurada los diferentes procesos de un algoritmo. Gracias a la estandarización de estos símbolos, se facilita la interpretación y el análisis de los procedimientos que seguirá un programa, permitiendo identificar errores y mejorar la eficiencia antes de su implementación en código. Esto resulta especialmente útil en entornos de desarrollo colaborativo, donde es fundamental que varios programadores comprendan la lógica del algoritmo de manera visual antes de traducirlo a un lenguaje de programación.

La correcta comprensión de la simbología utilizada en los diagramas de flujo también permite una mejor transición hacia la codificación en lenguajes como C++. Al traducir los elementos gráficos de un diagrama en estructuras de control, ciclos y funciones, se logra una correspondencia clara entre la lógica del algoritmo y su implementación en código. Esto no solo facilita el proceso de programación, sino que también contribuye a una mejor organización del código, haciéndolo más legible y fácil de mantener.

En conclusión, la simbología en los diagramas de flujo desempeña un papel crucial en el desarrollo de software, proporcionando una base sólida para la escritura de código estructurado y eficiente. Su uso permite optimizar el diseño de algoritmos, mejorar la comunicación entre programadores y reducir el tiempo de depuración y corrección de errores. Por ello, es fundamental que cualquier estudiante o profesional de la informática domine el uso de los diagramas de flujo y su relación con la programación en lenguajes como C++.



Tipos de Datos (Lenguajes de Programación)

Los lenguajes de programación son la base del desarrollo de software moderno, permitiendo la creación de sistemas y aplicaciones en diversos entornos tecnológicos. Cada lenguaje posee características propias, entre las cuales los tipos de datos juegan un papel fundamental, ya que definen la manera en que la información es almacenada, procesada y manipulada dentro de un programa. En esta investigación, se abordarán los tipos de datos utilizados en cuatro lenguajes ampliamente empleados en la industria: Python, JavaScript, Angular y HTML.

Python es reconocido por su simplicidad y versatilidad, lo que lo convierte en una opción popular tanto en desarrollo web como en ciencia de datos e inteligencia artificial. JavaScript, por su parte, es el lenguaje esencial para la interactividad en aplicaciones web, mientras que Angular, basado en TypeScript, permite el desarrollo de aplicaciones web estructuradas y escalables. Finalmente, HTML, aunque no es un lenguaje de programación propiamente dicho, es el pilar de la estructura de las páginas web, organizando el contenido visual en conjunto con CSS y JavaScript.

El objetivo de este análisis es explorar los diferentes tipos de datos en cada uno de estos lenguajes, estableciendo comparaciones sobre su implementación y utilidad. Para ello, se examinarán sus estructuras básicas y cómo influyen en el desarrollo de aplicaciones, proporcionando una visión clara de su aplicabilidad en el mundo real.

```
31 def __init__(self):
32     self.file = None
33     self.fingerprints = set()
34     self.logdups = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, "requests.log"),
39                          "a")
40         self.file.seek(0)
41         self.fingerprints.update(self._get_fingerprints())
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool("SUPERFINGER_DEBUG")
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

Para comprender la importancia de los tipos de datos en la programación, es necesario analizarlos en distintos lenguajes. Los tipos de datos determinan la naturaleza de la información almacenada en memoria y cómo puede manipularse dentro de un programa. A continuación, se detallan los principales tipos de datos en cuatro lenguajes de programación ampliamente utilizados: Python, JavaScript, Angular (TypeScript) y HTML.

➤ Tipos de datos en Python

Python es un lenguaje de programación de alto nivel con tipado dinámico, lo que significa que los datos pueden cambiar de tipo en tiempo de ejecución. Esta flexibilidad lo convierte en una herramienta poderosa para diversas aplicaciones, desde análisis de datos hasta inteligencia artificial.



Principales tipos de datos en Python

- **Enteros (int):** Representan números enteros sin decimales. Pueden ser positivos o negativos.
 - *Ejemplo:* 1, -5, 100, 2024
- **Flotantes (float):** Son números con punto decimal o en notación científica.
 - *Ejemplo:* 3.14, -0.001, 2.5e3
- **Cadenas de texto (str):** Secuencias de caracteres delimitadas por comillas simples o dobles.
 - *Ejemplo:* "Hola, mundo", 'Python es poderoso'
- **Booleanos (bool):** Representan valores lógicos True o False.
 - *Ejemplo:* True, False
- **Listas (list):** Conjuntos ordenados de elementos, pueden contener distintos tipos de datos.
 - *Ejemplo:* [1, 2, 3], ["manzana", "pera", "uva"]
- **Tuplas (tuple):** Similares a las listas, pero inmutables (no pueden modificarse después de su creación).
 - *Ejemplo:* (4, 5, 6), ("lunes", "martes", "miércoles")
- **Diccionarios (dict):** Conjuntos de pares clave-valor, utilizados para almacenar información estructurada.
 - *Ejemplo:* {"nombre": "Ana", "edad": 25}

Ejemplo en Python

```
calificaciones = [80, 95, 70, 85, 100]

promedio = sum(calificaciones) / len(calificaciones)

print(f"El promedio del grupo es: {promedio}")
```

➤ Tipos de datos en JavaScript

JavaScript es un lenguaje de programación interpretado y de tipado dinámico. Se utiliza principalmente en el desarrollo web para agregar interactividad a las páginas.



Principales tipos de datos en JavaScript

- **Number:** Representa tanto números enteros como flotantes.
 - *Ejemplo:* 10, -5, 3.14, 2e5
- **String:** Secuencia de caracteres delimitada por comillas.
 - *Ejemplo:* "Hola, JavaScript", 'Programación web'
- **Boolean:** Representa valores lógicos true o false.
 - *Ejemplo:* true, false
- **Null y Undefined:** null indica la ausencia intencional de un valor, mientras que undefined significa que una variable no ha sido asignada.
 - *Ejemplo:* let x = null; let y;
- **Array:** Colección ordenada de elementos.
 - *Ejemplo:* [10, 20, 30], ["rojo", "verde", "azul"]
- **Object:** Contenedor de propiedades y valores.
 - *Ejemplo:* {nombre: "Carlos", edad: 30}

Ejemplo en JavaScript

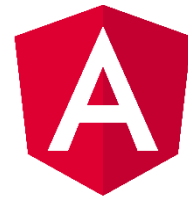
```
let calificaciones = [80, 95, 70, 85, 100];

let promedio = calificaciones.reduce((a, b) => a + b) / calificaciones.length;

console.log(`El promedio del grupo es: ${promedio}`);
```


➤ Tipos de datos en Angular (TypeScript)

Angular es un framework basado en TypeScript, que es un superset de JavaScript con tipado estático. Esto significa que los tipos de datos deben definirse explícitamente.



Principales tipos de datos en TypeScript

- **Number:** Igual que en JavaScript, representa números enteros y flotantes.
 - *Ejemplo:* 100, 5.5, -10
- **String:** Secuencia de caracteres delimitada por comillas.
 - *Ejemplo:* "Angular es un framework poderoso"
- **Boolean:** Representa valores true o false.
 - *Ejemplo:* true, false
- **Array:** Lista ordenada de elementos del mismo tipo.
 - *Ejemplo:* let numeros: number[] = [1, 2, 3, 4];
- **Tuple:** Conjunto de elementos con tipos específicos y longitud fija.
 - *Ejemplo:* let coordenada: [number, number] = [10, 20];
- **Enum:** Conjunto de valores predefinidos.
 - *Ejemplo:*
 - enum Color {Rojo, Verde, Azul};
 - let colorFavorito: Color = Color.Rojo;
- **Any:** Representa cualquier tipo de dato, útil en casos donde se desconoce el tipo exacto.
 - *Ejemplo:* let variable: any = "Texto"; variable = 5;

Ejemplo en Angular (TypeScript)

```
let calificaciones: number[] = [80, 95, 70, 85, 100];
```

```
let promedio: number = calificaciones.reduce((a, b) => a + b) / calificaciones.length;
```

```
console.log(`El promedio del grupo es: ${promedio}`);
```

➤ Tipos de datos en HTML

HTML no es un lenguaje de programación, pero maneja ciertos tipos de datos en formularios y validaciones.



Principales tipos de datos en HTML

- **Text:** Permite ingresar texto en formularios.
 - *Ejemplo:* `<input type="text" name="nombre">`
- **Number:** Acepta solo números.
 - *Ejemplo:* `<input type="number" min="0" max="100">`
- **Date:** Permite seleccionar fechas.
 - *Ejemplo:* `<input type="date">`
- **Boolean (Checkbox):** Casillas de verificación para valores true o false.
 - *Ejemplo:* `<input type="checkbox" checked>`
- **Email y URL:** Validan direcciones de correo y enlaces web.
 - *Ejemplo:* `<input type="email" required>`

Ejemplo en HTML

```
<form>

<label for="calificacion">Ingrese su calificación:</label>

<input type="number" id="calificacion" min="0" max="100">

<button type="submit">Enviar</button>

</form>
```

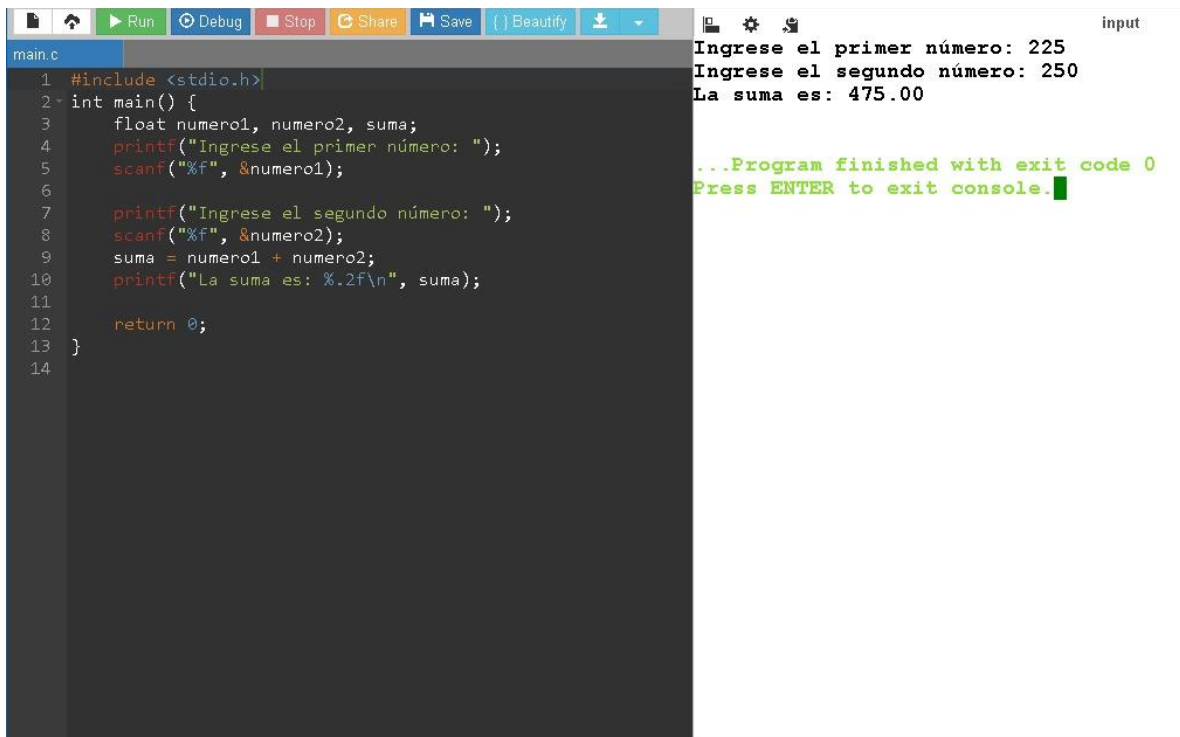
Ejemplo: Este programa en C solicita al usuario ingresar dos números, luego los suma y muestra el resultado. Los pasos son los siguientes:

Declaración de variables: Se definen tres variables (numero1, numero2 y suma) de tipo float para almacenar los números y la suma.

Entrada de datos: El programa pide al usuario que ingrese dos números mediante la función scanf.

Proceso de suma: Los dos números ingresados se suman y el resultado se guarda en la variable suma.

Salida de datos: El resultado de la suma se muestra en pantalla utilizando la función printf.



The image shows a code editor with a C program and its execution output. The code is in a file named 'main.c' and includes the standard input/output header. It defines a main function that declares three float variables: numero1, numero2, and suma. It prompts the user to enter two numbers, reads them using scanf, calculates the sum, and prints the result using printf. The program then returns 0. The console output shows the program running, with the user entering 225 and 250, and the program outputting 'La suma es: 475.00'. The program finished with exit code 0.

```
1 #include <stdio.h>
2 int main() {
3     float numero1, numero2, suma;
4     printf("Ingrese el primer número: ");
5     scanf("%f", &numero1);
6
7     printf("Ingrese el segundo número: ");
8     scanf("%f", &numero2);
9     suma = numero1 + numero2;
10    printf("La suma es: %.2f\n", suma);
11
12    return 0;
13 }
14
```

input
Ingrese el primer número: 225
Ingrese el segundo número: 250
La suma es: 475.00
...Program finished with exit code 0
Press ENTER to exit console.

Este procedimiento ha explorado los diferentes tipos de datos en Python, JavaScript, Angular (TypeScript) y HTML, proporcionando ejemplos prácticos que ilustran su uso en distintos contextos. Conocer estos tipos de datos es esencial para desarrollar software eficiente y adaptado a las necesidades de cada proyecto.

El análisis de los tipos de datos en Python, JavaScript, Angular y HTML demuestra la importancia de comprender cómo se manejan los valores dentro de cada lenguaje de programación. Python, con su tipado dinámico y estructuras flexibles, se destaca por su facilidad de uso en aplicaciones científicas y de análisis de datos. JavaScript, al ser el lenguaje central de la web, maneja datos de forma flexible y dinámica, permitiendo la interactividad en aplicaciones en línea. Angular, basado en TypeScript, introduce un tipado más estricto para mejorar la robustez del código en aplicaciones a gran escala. Por último, HTML, aunque no es un lenguaje de programación, utiliza tipos de datos en formularios y validaciones para mejorar la experiencia del usuario.

La elección del lenguaje adecuado depende del contexto en el que se desee desarrollar una aplicación. En entornos web, la combinación de JavaScript, Angular y HTML es esencial para construir plataformas dinámicas e interactivas. Mientras tanto, Python sigue siendo una opción ideal para el análisis de datos, inteligencia artificial y desarrollo de software con una sintaxis clara y simple.

En el futuro, el conocimiento profundo de estos lenguajes y sus estructuras de datos será fundamental para cualquier ingeniero en software. La correcta elección y manipulación de los tipos de datos permitirá optimizar procesos, mejorar el rendimiento de las aplicaciones y garantizar un desarrollo eficiente y estructurado.



Tipos de Estructuras de Datos

En el ámbito de la programación y las ciencias de la computación, las estructuras de datos y los algoritmos son fundamentales para el manejo eficiente de la información. Estos conceptos son esenciales para desarrollar soluciones optimizadas a problemas complejos, permitiendo un procesamiento de datos más rápido y seguro. Las estructuras secuenciales, condicionales, y los ciclos como *while* o *for*, son pilares de la lógica de programación que permiten ejecutar instrucciones repetitivas y tomar decisiones basadas en condiciones específicas. En este documento, se exploran estas y otras estructuras de datos que forman la base de muchas aplicaciones informáticas modernas.

Además de las estructuras básicas, se introduce el concepto de arreglos, pilas y colas, que son estructuras de datos utilizadas para almacenar y gestionar múltiples elementos de manera organizada. Por ejemplo, las pilas funcionan bajo el principio de LIFO (último en entrar, primero en salir), mientras que las colas siguen el principio FIFO (primero en entrar, primero en salir). Cada una de estas estructuras tiene ventajas y desventajas dependiendo de la aplicación y del tipo de procesamiento que se desee realizar, lo que hace fundamental su estudio en la ingeniería de software.

A medida que se avanza en el análisis de estructuras más complejas, se incluyen las listas y los árboles. Las listas, como las simplemente ligadas o las dobles circulares, ofrecen mayor flexibilidad que los arreglos, permitiendo una manipulación más eficiente de los datos. Los árboles, por otro lado, son estructuras jerárquicas que permiten realizar búsquedas y ordenar elementos de manera óptima. En especial, los árboles binarios y balanceados, así como los multi-caminos, son de gran utilidad en la organización y búsqueda de datos en aplicaciones como bases de datos y sistemas de archivos.

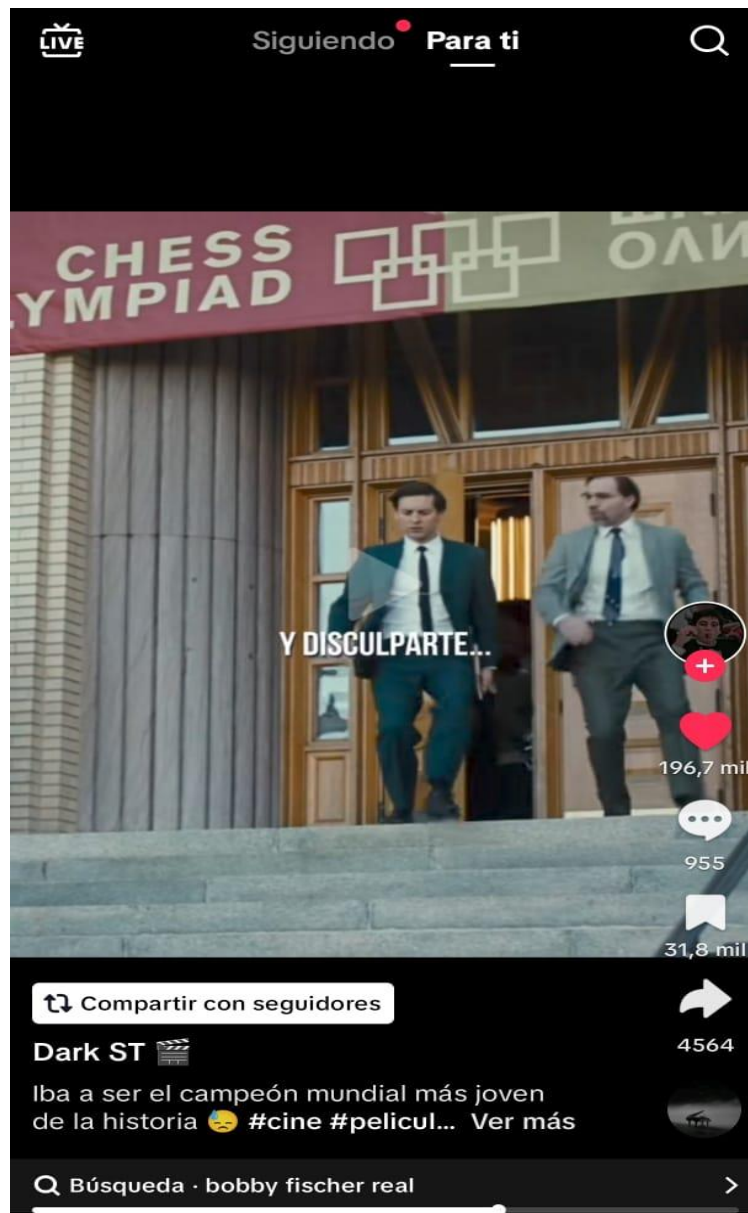
Finalmente, se aborda el uso de grafos y métodos de ordenación y búsqueda, que son esenciales en la resolución de problemas complejos como la optimización de rutas, la planificación y la organización de grandes volúmenes de datos. Estos conceptos se aplican diariamente en plataformas como Spotify, Facebook y Netflix, donde es crucial ordenar y buscar entre millones de registros de manera eficiente para proporcionar una experiencia de usuario fluida y personalizada.

Estructuras Secuenciales

Las estructuras secuenciales en programación son bloques de código en los que las instrucciones se ejecutan en orden, una tras otra, de arriba hacia abajo. Es decir, cada instrucción se realiza de forma continua, sin saltos ni decisiones que alteren el flujo. En una estructura secuencial no hay condiciones ni bucles, cada instrucción se ejecuta una sola vez en el orden en el que aparece aparte son de las estructuras más básicas, suele ser la base de los programas, combinándose luego con estructuras condicionales o de repetición para crear lógica más compleja.

Ejemplo

TikTok: La secuencia para reproducir un video sigue pasos secuenciales: cargar el video, iniciar la reproducción, mostrar comentarios y reacciones, y al finalizar, pasar al siguiente video.



Código: Utilizar la función play() para iniciar la reproducción del video.

Establecer el atributo autoplay para iniciar la reproducción automáticamente.

```
<video id="video" width="400" controls>
  <source src="video.mp4" type="video/mp4">
  Tu navegador no soporta el video.
</video>
<button onclick="reproducirVideo()">Reproducir Video</button>
<p id="mensaje"></p>

<script>
  function reproducirVideo() {
    const video = document.getElementById("video");
    video.play(); // Paso 1: Reproducir el video
    video.onended = () => {
    };
  }
</script>
```

Ejemplo

WhatsApp: Enviar un mensaje sigue una secuencia específica: escribir el mensaje, seleccionar el destinatario, enviar, y confirmar la entrega o lectura.



```

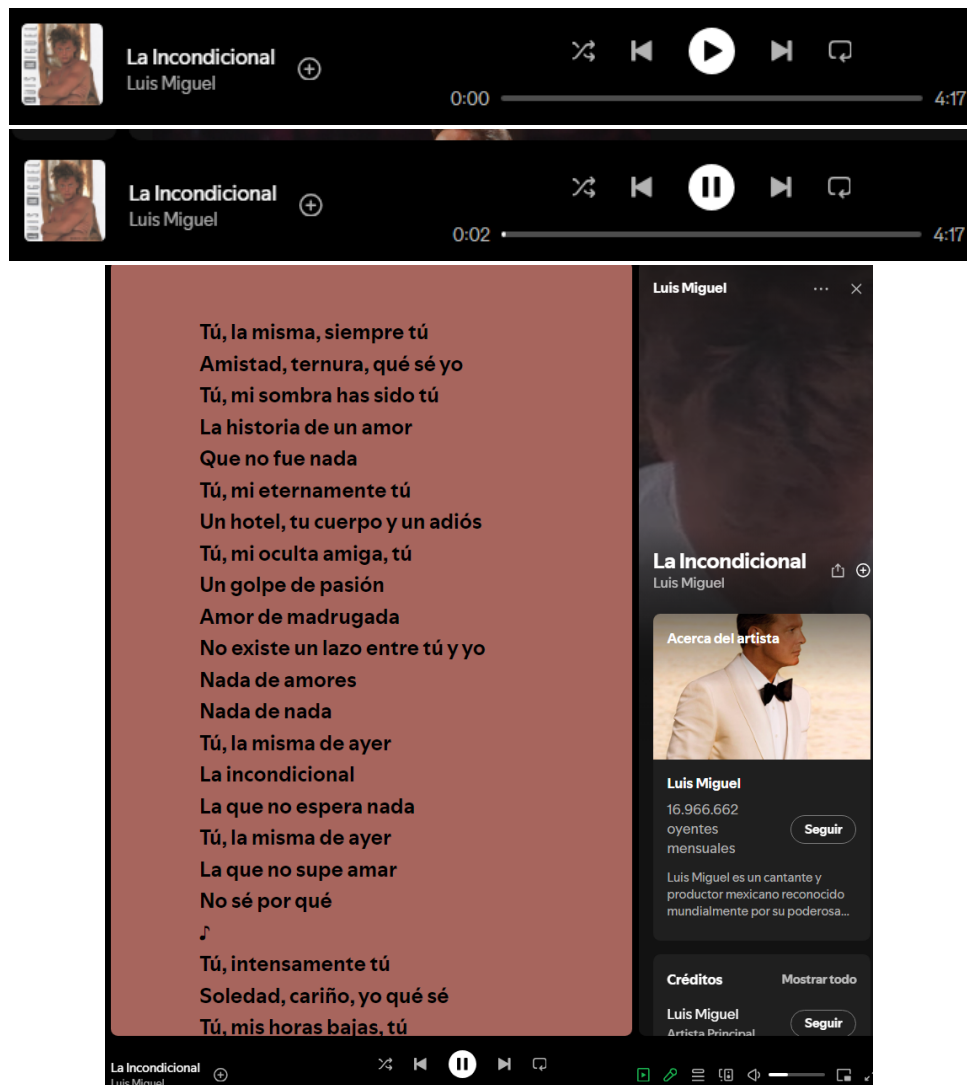
<h2>Enviar Mensaje</h2>
<input type="text" id="mensaje" placeholder="Escribe tu mensaje aquí">
<button onclick="enviarMensaje()">Enviar</button>
<p id="confirmacion"></p>

<script>
  function enviarMensaje() {
    const mensaje = document.getElementById("mensaje").value;
    if (mensaje) {
      document.getElementById("confirmacion").innerText = "Mensaje enviado: " + mensaje;
    } else {
      document.getElementById("confirmacion").innerText = "Por favor, escribe un mensaje.";
    }
  }
</script>

```

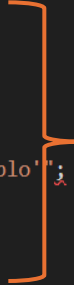
Ejemplo

Spotify: Al reproducir una canción, el proceso sigue pasos secuenciales: cargar la canción, iniciar reproducción, mostrar la carátula y letras (si están disponibles).




```
<h2>Reproducción de Canción</h2>
<button onclick="reproducirCancion()">Reproducir</button>
<p id="estado"></p>

<script>
  function reproducirCancion() {
    document.getElementById("estado").innerText = "Reproduciendo: 'Canción de Ejemplo'";
    setTimeout(() => {
      document.getElementById("estado").innerText = "Canción finalizada.";
    }, 5000); // Simulación de 5 segundos
  }
</script>
```



Estructuras Condicionales

Las estructuras condicionales en programación permiten ejecutar ciertas instrucciones solo si se cumple una condición específica. Estas estructuras permiten que el programa tome decisiones basadas en criterios establecidos, desviando el flujo de ejecución según sea necesario.

Tipos comunes de estructuras condicionales

If: Ejecuta un bloque de código solo si una condición es verdadera.

If-Else: Ejecuta un bloque de código si la condición es verdadera; si no, ejecuta otro bloque.

Else If: Permite probar múltiples condiciones en secuencia y ejecutar el bloque de código correspondiente a la primera condición que sea verdadera.

Switch: Evalúa una variable o expresión y ejecuta un bloque de código según su valor específico. Es útil cuando se tienen varias condiciones que dependen del valor exacto de una variable.

Ejemplo:

Facebook

Condición: Si el usuario ha dado "Me gusta" a una publicación, muestra el botón como "Ya no me gusta"; si no, muestra "Me gusta".

Condición: Si el usuario tiene permisos de administrador en un grupo, permite eliminar o editar publicaciones; si no, muestra solo opciones básicas de interacción.



```

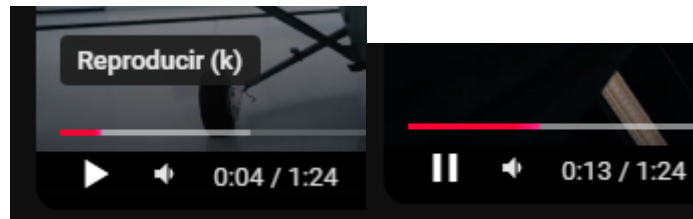
<script>
  let liked = false; // Variable que indica si ya le dio "Me gusta"

  function toggleLike() {
    if (liked) {
      document.getElementById("likeButton").innerHTML = "Me gusta"; // Cambia el texto si ya le dio "Me gusta"
      liked = false;
    } else {
      document.getElementById("likeButton").innerHTML = "Ya no me gusta"; // Cambia el texto si no ha dado "Me gusta"
      liked = true;
    }
  }
}
</script>

```

Ejemplo

Youtube: Muestra como reproducir o pausar un video



```

<video id="video" width="320" height="240" controls>
  <source src="video.mp4" type="video/mp4">
  Tu navegador no soporta video.
</video>
<button onclick="togglePlay()">Reproducir / Pausar</button>

<script>
  function togglePlay() {
    let video = document.getElementById("video");

    if (video.paused) {
      video.play();
    } else {
      video.pause();
    }
  }
</script>

```

Ejemplo

Amazon: En este ejemplo, si el producto está disponible en inventario, permite comprarlo; si no, muestra un mensaje de "Fuera de stock".



A screenshot of an Amazon product card. At the top, the price is displayed as \$351.00. Below the price, it says 'Entrega GRATIS el viernes, 8 de noviembre. Realiza el pedido en 19 hrs 48 mins'. Underneath that, there's a location pin icon followed by 'Enviar a VICTOR - Reynosa 88740'. The word 'Disponible' is shown in green. Below this is a quantity selector showing 'Cantidad: 1' with a dropdown arrow. At the bottom is a yellow button that says 'Agregar al Carrito'.

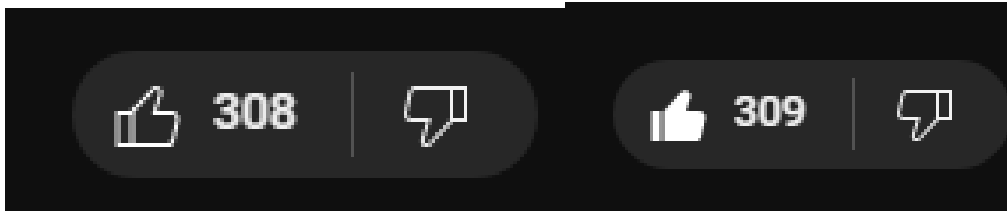
```
int enStock = 1; // 1 significa disponible, 0 significa agotado  
  
if (enStock) {  
    printf("Producto disponible. Puedes agregar al carrito.\n");  
} else {  
    printf("Fuera de stock. No puedes comprar este producto.\n");  
}
```

Estructura Simple

La estructura simple es una estructura de control condicional en programación que permite ejecutar un bloque de código solo si se cumple una condición específica. A diferencia de otras estructuras condicionales, la estructura simple no tiene una alternativa o acción que se ejecute si la condición es falsa.

Ejemplo

Youtube: Condición: Si el usuario ha dado "Me gusta" al video, muestra el ícono en color; si no, no hace nada y se queda en su estado original.

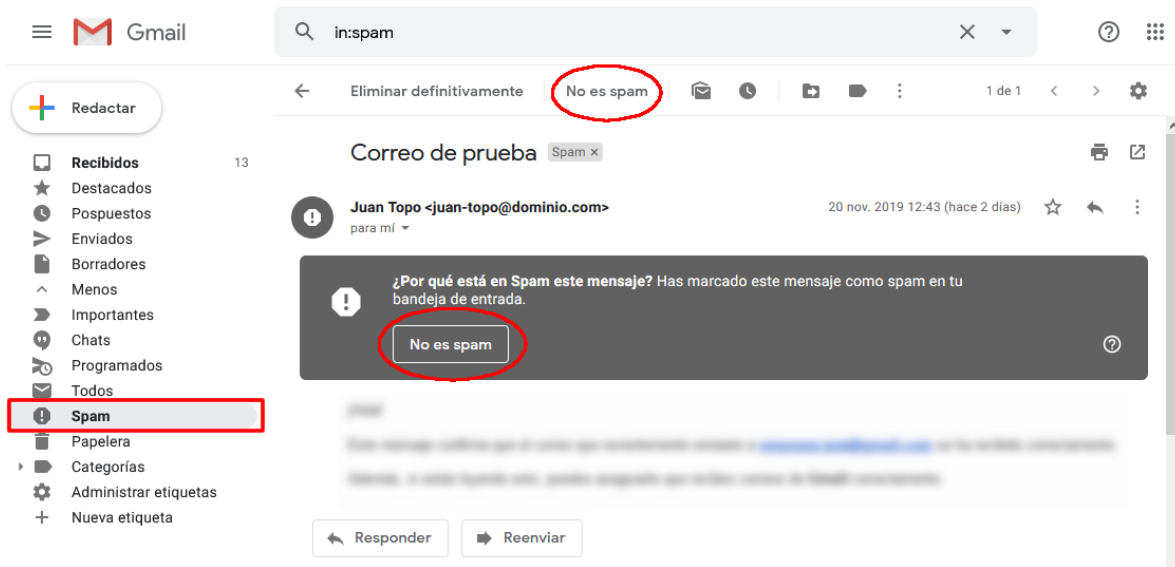


```
<script>
  let liked = false;

  function toggleLike() {
    if (!liked) { // Si el usuario no ha dado "Me gusta"
      document.getElementById("likeButton").style.color = "red";
      liked = true;
    }
  }
</script>
```

Ejemplo:

Gmail: Si el correo es sospechoso de spam, muestra una advertencia en la parte superior del mensaje; si no, no hace nada.



```
if (correoSospechoso) { // Estructura simple
    printf("⚠ Advertencia: Este correo podría ser sospechoso.\n");
}

printf("Mostrando contenido del correo...\n");
```

Ejemplo Amazon:

Verificar Suscripción para Envío Gratis



```
if (esMiembroPrime) { // Estructura simple
    printf("Envío gratis disponible.\n");
}

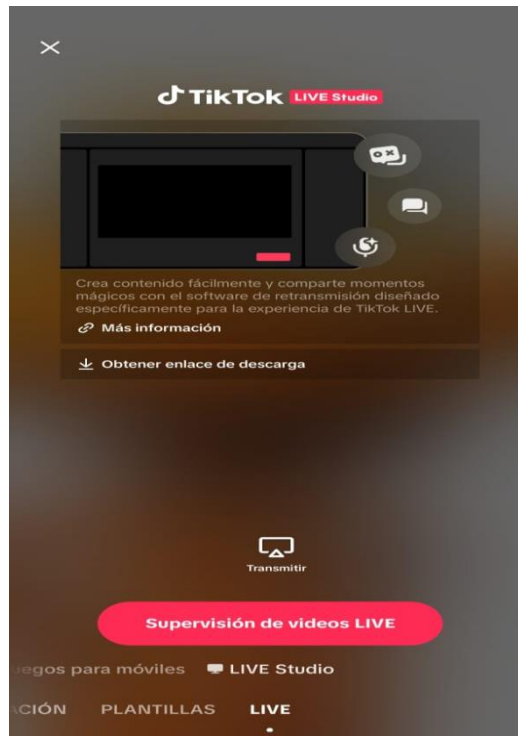
printf("Proceso de compra en curso...\n");
```

Estructura Doble

Una estructura doble es una estructura de control condicional que permite ejecutar uno de dos bloques de código según el resultado de una condición. Si la condición es verdadera, se ejecuta un bloque de código; si es falsa, se ejecuta otro bloque.

Ejemplo TikTok:

Si el usuario tiene más de 1,000 seguidores, se le permite publicar videos en vivo; de lo contrario, se le muestra un mensaje que dice "Necesitas más seguidores para transmitir en vivo".



```
<button onclick="publicarVideo()">Publicar Video</button>
<p id="mensaje"></p>

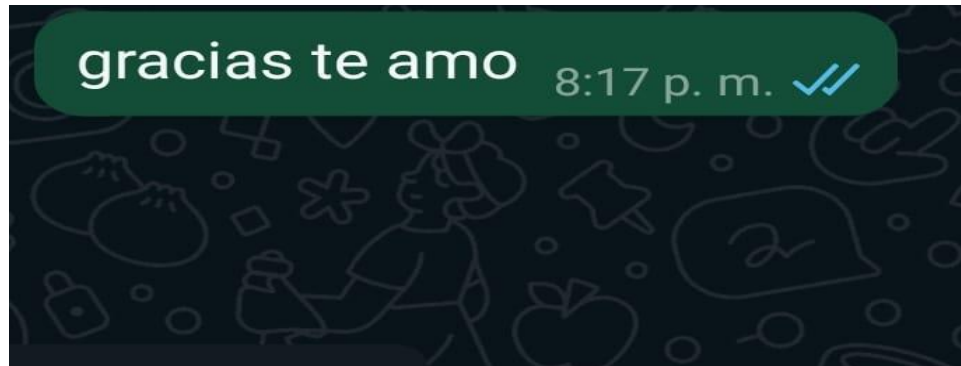
<script>
  function publicarVideo() {
    const seguidores = document.getElementById("seguidores").value;
    const mensaje = document.getElementById("mensaje");

    if (seguidores > 1000) {
      mensaje.innerText = "Video publicado exitosamente.";
    } else {
      mensaje.innerText = "Necesitas más seguidores para publicar.";
    }
  }
</script>
```


Ejemplo

Whatsapp:

Si el mensaje se ha entregado, se muestra un doble check azul; si no se ha entregado, se muestra un check simple.



```
<script>
  function mostrarEstado() {
    const entregado = document.getElementById("mensajeEntregado").checked;
    const estado = document.getElementById("estado");

    if (entregado) {
      estado.innerText = "✅ Mensaje entregado (check azul).";
    } else {
      estado.innerText = "❌ Mensaje no entregado (check simple).";
    }
  }
</script>
```

Ejemplo

Gmail: Si el correo electrónico está marcado como importante, se muestra en la sección de correos destacados; si no, se muestra en la bandeja de entrada normal.



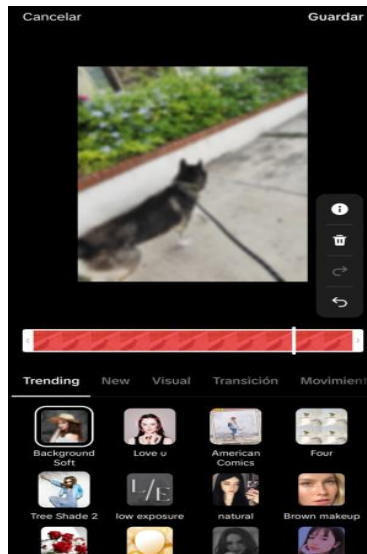
```
if (correoImportante) { // Estructura doble  
    printf("Este correo es importante.\n");  
} else {  
    printf("Este correo no es importante.\n");  
}
```

Estructura Multiple

Una estructura múltiple es una forma de control condicional que permite evaluar múltiples condiciones y ejecutar diferentes bloques de código según cuál de ellas se cumpla. A diferencia de las estructuras simples y dobles, que manejan uno o dos caminos posibles, las estructuras múltiples pueden manejar tres o más opciones.

Ejemplo:

Tiktok: Al seleccionar un filtro de video, si se elige "Retro", se aplica un filtro retro; si se elige "Brillante", se aplica un filtro brillante; si se elige "Oscuro", se aplica un filtro oscuro. Si no se selecciona ninguno, se muestra un mensaje de error.



```
<select id="filtro">
  <option value="ninguno">--Selecciona un filtro--</option>
  <option value="retro">Retro</option>
  <option value="brillante">Brillante</option>
  <option value="oscuro">Oscuro</option>
</select>
<button onclick="aplicarFiltro()">Aplicar Filtro</button>
<p id="resultado"></p>

<script>
  function aplicarFiltro() {
    const filtroSeleccionado = document.getElementById("filtro").value;
    const resultado = document.getElementById("resultado");

    switch (filtroSeleccionado) {
      case "retro":
        resultado.innerHTML = "Filtro Retro aplicado.";
        break;
      case "brillante":
        resultado.innerHTML = "Filtro Brillante aplicado.";
        break;
      case "oscuro":
        resultado.innerHTML = "Filtro Oscuro aplicado.";
        break;
      default:
        resultado.innerHTML = "Por favor, selecciona un filtro.";
        break;
    }
  }
</script>
```

Ejemplo

Whatsapp: Al enviar un mensaje, si el mensaje es de texto, se muestra un check; si es un mensaje de voz, se muestra un ícono de micrófono; si es un video, se muestra un ícono de video. Si hay un error en el envío, se muestra un mensaje de error.

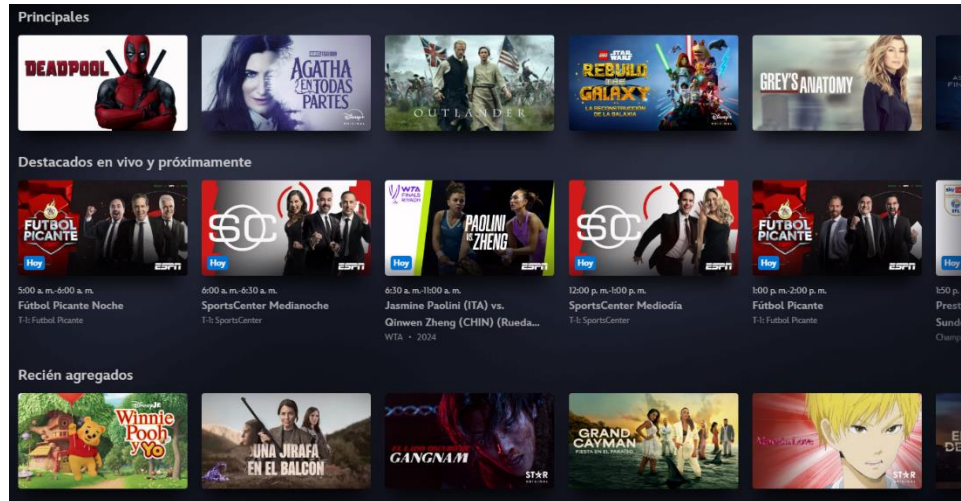


```
printf("Selecciona el tipo de mensaje:\n");
printf("1. Mensaje de texto\n");
printf("2. Mensaje de voz\n");
printf("3. Mensaje de video\n");
printf("4. Error en el envío\n");
scanf("%d", &tipoMensaje);

switch (tipoMensaje) {
    case 1:
        printf("✅ Mensaje de texto enviado.\n");
        break;
    case 2:
        printf("🎤 Mensaje de voz enviado.\n");
        break;
    case 3:
        printf("📺 Mensaje de video enviado.\n");
        break;
    case 4:
        printf("❌ Error en el envío del mensaje.\n");
        break;
    default:
        printf("Opción no válida. Por favor, selecciona un número del 1 al 4.\n");
        break;
}
```

Ejemplo

DisneyPlus: Al elegir un contenido, si el contenido está disponible en la región, se permite la reproducción; si está en la lista de "Nuevos lanzamientos", se muestra un banner; si es popular, se destaca como "Éxito". Si el contenido no está disponible, se muestra un mensaje de "No disponible en tu región".



```
printf("Selecciona el tipo de contenido en Disney:\n");
printf("1. Película\n");
printf("2. Serie\n");
printf("3. Documental\n");
printf("4. Especial\n");
printf("5. Error\n");
scanf("%d", &tipoContenido);

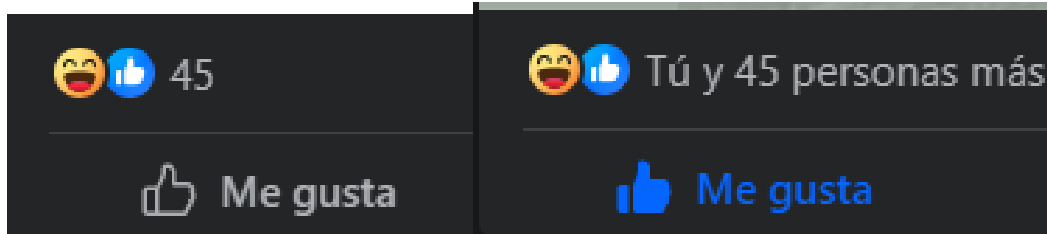
switch (tipoContenido) {
    case 1:
        printf("Has seleccionado una Película.\n");
        break;
    case 2:
        printf("Has seleccionado una Serie.\n");
        break;
    case 3:
        printf("Has seleccionado un Documental.\n");
        break;
    case 4:
        printf("Has seleccionado un Especial.\n");
        break;
    case 5:
        printf("Error: Contenido no disponible.\n");
        break;
    default:
        printf("Opción no válida. Por favor, selecciona un número del 1 al 5.\n");
        break;
}
```

Contadores

Los contadores son variables que se utilizan en programación para llevar un registro de la cantidad de veces que ocurre un evento específico o para contar elementos en un conjunto. Son comunes en bucles y estructuras de control, donde se incrementan o decrementan de acuerdo con ciertas condiciones.

Ejemplo

Facebook: Cada publicación tiene un contador que indica cuántas reacciones ha recibido (me gusta, me encanta, etc.). Se incrementa cada vez que un usuario reacciona a la publicación.



```
<h2>Publicación</h2>
<p>¡Esta es una publicación genial!</p>
<button onclick="darLike()">Me gusta</button>
<p id="contador">Me gusta: 0</p>

<script>
  let contadorLikes = 0; // Inicialización del contador

  function darLike() {
    contadorLikes++; // Incrementar el contador
    document.getElementById("contador").innerText = "Me gusta: " + contadorLikes;
  }
</script>
```

Ejemplo

Whatsapp: Contador de Mensajes No Leídos, muestra cuántos mensajes no han sido leídos en un chat. Cada vez que un mensaje llega y no se abre, se incrementa este contador.



```
printf("Mensajes no leídos: %d\n", mensajesNoLeídos);
printf("Selecciona una opción:\n");
printf("1. Marcar mensaje como leído\n");
printf("2. Salir\n");
scanf("%d", &opcion);

switch (opcion) {
    case 1:
        if (mensajesNoLeídos > 0) {
            mensajesNoLeídos--; // Decrementar el contador
            printf("Mensaje marcado como leído. Mensajes no leídos restantes: %d\n", mensajesNoLeídos);
        } else {
            printf("No hay mensajes no leídos.\n");
        }
        break;
    case 2:
        printf("Saliendo...\n");
        break;
    default:
        printf("Opción no válida.\n");
        break;
}
```

Ejemplo

Youtube: Contador de Suscriptores, cada canal tiene un contador que muestra cuántos suscriptores tiene. Este número se incrementa cada vez que un usuario se suscribe al canal.

16 suscriptores · 103 videos

· 17 suscriptores · 103 videos

```
printf("Suscriptores actuales: %d\n", suscriptores);  
printf("Cuántos nuevos suscriptores deseas agregar?: ");  
scanf("%d", &nuevosSuscriptores);  
  
suscriptores += nuevosSuscriptores; // Incrementar el contador  
printf("Suscriptores totales ahora: %d\n", suscriptores);
```


Ciclos

Los ciclos son estructuras de control que permiten ejecutar un bloque de código repetidamente mientras se cumpla una condición específica. Las estructuras que repiten una secuencia de instrucciones, un número determinado de veces, se nombra bucles, y se denomina interacion al hecho de repetir la ejecución de una secuencia de acciones. Hay varios métodos y técnicas que se utilizan en los ciclos para controlar su comportamiento y optimizar su funcionamiento como son las condiciones, contadores, listas y arrays y anidación. Cada uno tiene un uso especial para cada cosa que los hace especiales para el uso de los ciclos.

Ventajas:

- **Automatización de tareas repetitivas:** los ciclos permiten automatizar tareas repetitivas y ahorrar tiempo y esfuerzo. En aplicaciones que manejan muchos datos, como listas de usuarios, productos, o transacciones, los ciclos son indispensables para manejar y procesar cada elemento de forma individual.
- **Procesamiento de grandes cantidades de datos:** los ciclos permiten procesar grandes cantidades de datos de manera eficiente y simplificar tareas complejas.
- **Reducción de Código Repetitivo:** Los ciclos permiten ejecutar el mismo bloque de código varias veces, lo cual evita escribir líneas repetidas. Esto hace que el código sea más corto, claro y fácil de leer.

Desventajas:

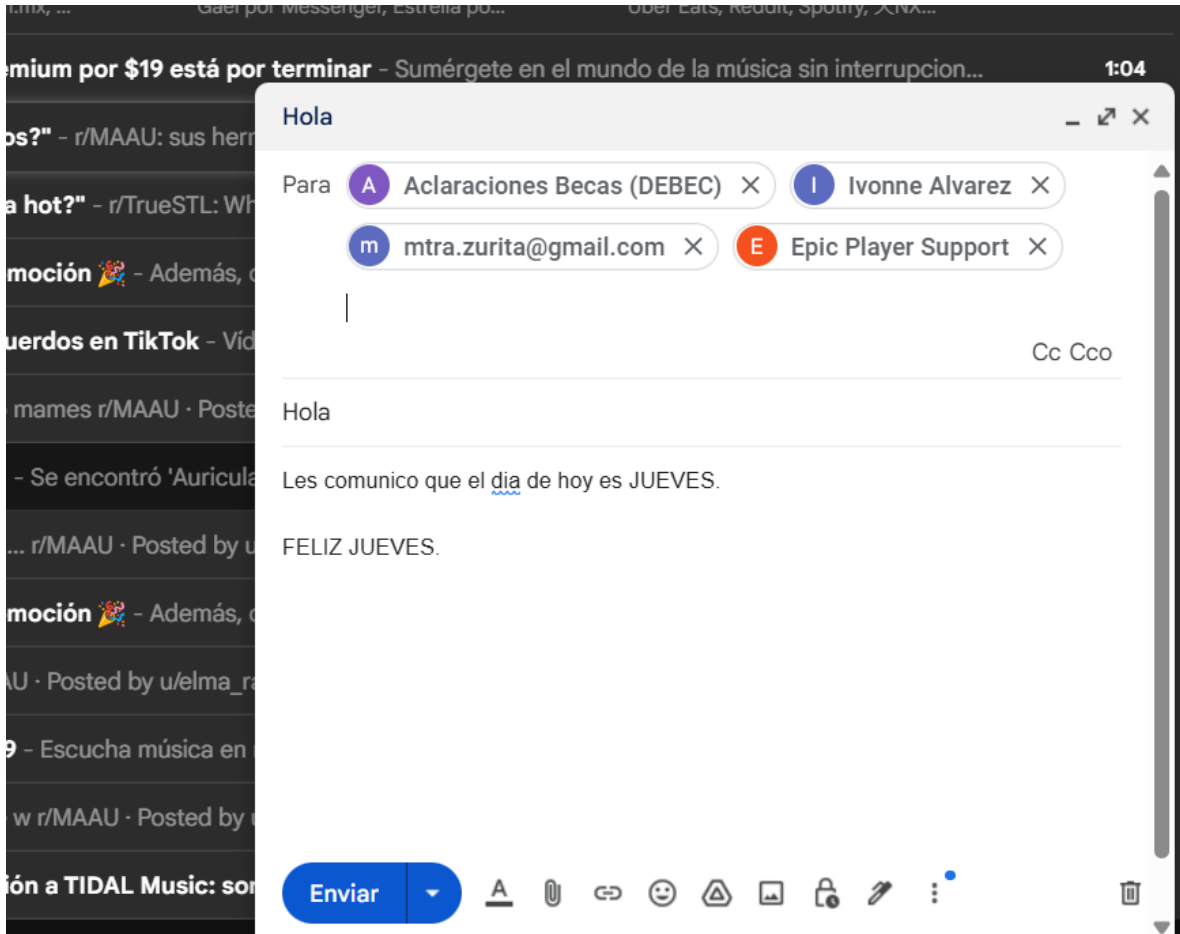
- **Ineficiencia en el rendimiento:** Uno de los principales problemas de utilizar ciclos en programación es que pueden afectar el rendimiento de tu aplicación. Al ejecutar un ciclo muchas veces, especialmente en grandes conjuntos de datos, el tiempo de procesamiento puede incrementarse significativamente, ralentizando la ejecución del programa.
- **Dificultad para depurar:** Otra desventaja de los ciclos es que pueden complicar el proceso de depuración de errores. Cuando tienes múltiples iteraciones en un ciclo, identificar dónde se encuentra el problema puede volverse más complicado, lo que dificulta la corrección de errores y el mantenimiento del código.
- **Posibilidad de caer en ciclos infinitos:** Los ciclos mal diseñados pueden llevar a lo que se conoce como un “ciclo infinito”, donde el programa queda atrapado en una iteración continua sin llegar a un punto de salida. Esto puede provocar que la aplicación se bloquee o se vuelva inestable, afectando negativamente la experiencia del usuario.

Aplicación en la vida cotidiana:

A continuacion se mostra el como los ciclos son usados en la vida cotidiana, como lo son apps, paginas web, etc.

Ejemplos:

Gmail: Al enviar correos masivos, un ciclo puede utilizarse para iterar sobre una lista de direcciones de correo electrónico y enviar el mismo mensaje a cada uno.



Como funcionaria en codigo:

```
const nodemailer = require('nodemailer');

// Configuración del transporte SMTP
const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: 'tuemail@gmail.com',
    pass: 'tu_contraseña'
  }
});

// Función para enviar el correo a múltiples destinatarios
async function enviarCorreoMultiple(destinatarios, asunto, cuerpo) {
  // Convertimos la lista de destinatarios en una cadena separada por comas
  const destinatariosLista = destinatarios.join(',');

  // Configuramos el mensaje
  const mensaje = {
    from: 'tuemail@gmail.com',
    to: destinatariosLista, // Agregamos la lista de destinatarios
    subject: asunto,
    text: cuerpo
  };
}
```

```
try {
  // Enviamos el correo
  let info = await transporter.sendMail(mensaje);
  console.log('Correo enviado:', info.response);
} catch (error) {
  console.error('Error al enviar el correo:', error);
}

// Ejemplo de uso
const destinatarios = ['correo1@example.com', 'correo2@example.com', 'correo3@example.com'];
const asunto = 'Prueba de correo a múltiples destinatarios';
const cuerpo = 'Este es un mensaje de prueba enviado a varios destinatarios usando Nodemailer';

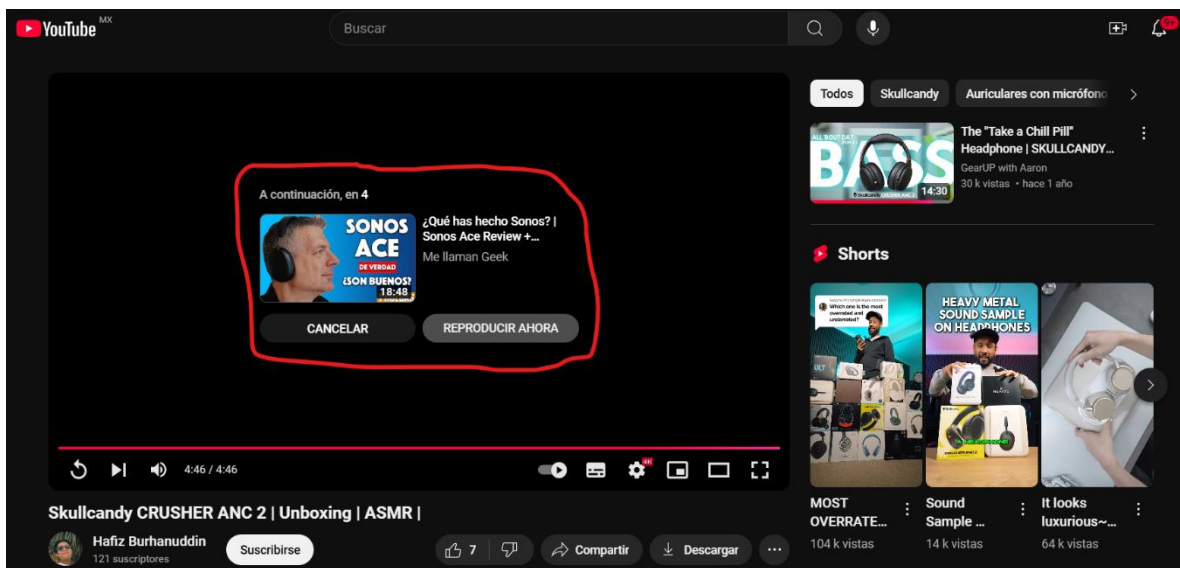
// Llamamos a la función para enviar el correo
enviarCorreoMultiple(destinatarios, asunto, cuerpo);
```

Funcion del codigo:

1. **Configuración del transporte SMTP:** Configuramos el transporte SMTP usando `nodemailer.createTransport`, especificando el servicio (gmail en este caso) y las credenciales de acceso del remitente. Para Gmail, es posible que necesites habilitar el acceso de aplicaciones menos seguras o configurar una contraseña de aplicaciones si tienes la autenticación en dos pasos.
2. **Función para enviar el correo:** La función `enviarCorreoMultiple` toma como parámetros una lista de destinatarios (`destinatarios`), el asunto (`asunto`) y el cuerpo del mensaje (`cuerpo`).
3. **Conversión de la lista de destinatarios:** Convertimos la lista de destinatarios en una cadena separada por comas (`destinatarios.join(',')`), ya que es el formato que espera el campo `to` de Nodemailer.
4. **Enviar el correo:** Llamamos a `transporter.sendMail(mensaje)` dentro de un bloque `try-catch` para manejar errores y mostrar un mensaje de confirmación cuando el correo se envía correctamente.

Gmail fue desarrollado en JavaScript en el lado del cliente, pero para los servidores usan Java y C++ para un buen procesamiento de datos, así como también html y css para su navegador web.

Reproducción continua: En YouTube, los ciclos permiten reproducir automáticamente el siguiente video o canción, basándose en tu historial o preferencias



Como funcionaria el codigo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Reproducción Automática de Video</title>
</head>
<body>
  <!-- Contenedor del video de YouTube -->
  <div id="player"></div>

  <script>
    // Lista de videos para reproducir automáticamente
    const videos = ['VIDEO_ID_1', 'VIDEO_ID_2', 'VIDEO_ID_3'];
    let currentIndex = 0; // Índice del video actual

    // Cargar la API de YouTube de forma asíncrona
    var tag = document.createElement('script');
    tag.src = "https://www.youtube.com/iframe_api";
    var firstScriptTag = document.getElementsByTagName('script')[0];
    firstScriptTag.parentNode.insertBefore(tag, firstScriptTag);
```

```
    // Crear el reproductor cuando la API esté lista
    var player;
    function onYouTubeIframeAPIReady() {
      player = new YT.Player('player', {
        height: '390',
        width: '640',
        videoId: videos[currentIndex], // Reproduce el primer video de la lista
        events: {
          'onStateChange': onPlayerStateChange
        }
      });
    }

    // Detectar el final del video y reproducir el siguiente
    function onPlayerStateChange(event) {
      if (event.data === YT.PlayerState.ENDED) {
        currentIndex++; // Avanzar al siguiente video
        if (currentIndex < videos.length) {
          player.loadVideoById(videos[currentIndex]); // Cargar el siguiente vi
        } else {
          console.log("Se terminó la lista de videos.");
        }
      }
    }
  }
</script>
</body>
</html>
```

Funcion del codigo:

1. **Lista de Videos:** La variable videos contiene una lista de IDs de videos que se reproducirán en secuencia. Cada ID de video corresponde a un video específico de YouTube.
2. **Carga de la API de YouTube:** Se crea una etiqueta de <script> que carga la API de IFrame de YouTube de forma asíncrona.
3. **Creación del Reproductor:** La función onYouTubeIframeAPIReady es llamada automáticamente por la API cuando está lista. En esta función, se crea un nuevo reproductor (player) y se carga el primer video de la lista.
4. **Detección del Fin de un Video:** La función onPlayerStateChange escucha el estado del reproductor. Cuando event.data es YT.PlayerState.ENDED, significa que el video terminó. Se incrementa el índice (currentIndex) y se carga el siguiente video con player.loadVideoById().
5. **Condición para el Fin de la Lista de Videos:** Si currentIndex alcanza el final de la lista, se muestra un mensaje en la consola indicando que la lista terminó.

Youtube fue desarrollado combinando muchos lenguajes de programacion ya que la tecnologia a fue evolucionando para poder adaptarse al enorme crecimiento del sitio. Algunos lenguajes utilizados son python, C++, JavaScript, Java, Go y muchas más que hicieron que youtube crezca como página de videos.

Cálculo de gastos y presupuesto: En la app de BBVA usa ciclos para iterar sobre las transacciones y calcular totales de gastos, ahorros y presupuesto.



Como funcionaria el codigo:

```
import java.util.ArrayList;
import java.util.List;

// Clase que representa una Cuenta Bancaria
class Cuenta {
    private String numeroCuenta;
    private double saldo;

    public Cuenta(String numeroCuenta, double saldo) {
        this.numeroCuenta = numeroCuenta;
        this.saldo = saldo;
    }

    public String getNumeroCuenta() {
        return numeroCuenta;
    }

    public double getSaldo() {
        return saldo;
    }

    public void aplicarInteres(double tasaInteres) {
        saldo += saldo * tasaInteres;
    }

    @Override
    public String toString() {
        return "Cuenta: " + numeroCuenta + ", Saldo: $" + saldo;
    }
}
```

```
// Clase principal
public class Banco {
    public static void main(String[] args) {
        // Crear una lista de cuentas de clientes
        List<Cuenta> cuentas = new ArrayList<>();
        cuentas.add(new Cuenta("1001", 1500.00));
        cuentas.add(new Cuenta("1002", 2500.00));
        cuentas.add(new Cuenta("1003", 3000.00));

        // Tasa de interés mensual (por ejemplo, 1%)
        double tasaInteres = 0.01;

        // Aplicar la tasa de interés a cada cuenta usando un ciclo
        for (Cuenta cuenta : cuentas) {
            cuenta.aplicarInteres(tasaInteres);
        }

        // Mostrar el saldo actualizado de cada cuenta
        System.out.println("Saldo después de aplicar la tasa de interés:");
        for (Cuenta cuenta : cuentas) {
            System.out.println(cuenta);
        }
    }
}
```

Funcion del codigo:

1. **Clase Cuenta:** La clase Cuenta representa una cuenta bancaria con un número de cuenta y un saldo. Tiene un método aplicarInteres(double tasaInteres) que calcula y aplica el interés mensual al saldo actual.
2. **Lista de Cuentas:** Se crea una lista cuentas de objetos Cuenta, que simula varias cuentas de clientes. Cada cuenta tiene un saldo inicial.
3. **Aplicación de la Tasa de Interés:** Se define una tasa de interés mensual (tasaInteres) del 1% (0.01). Un ciclo for-each recorre la lista de cuentas y llama a aplicarInteres(tasaInteres) en cada cuenta, actualizando el saldo de cada una.
4. **Mostrar el Saldo Actualizado:** Otro ciclo for-each recorre la lista nuevamente para imprimir el saldo actualizado de cada cuenta después de aplicar el interés.

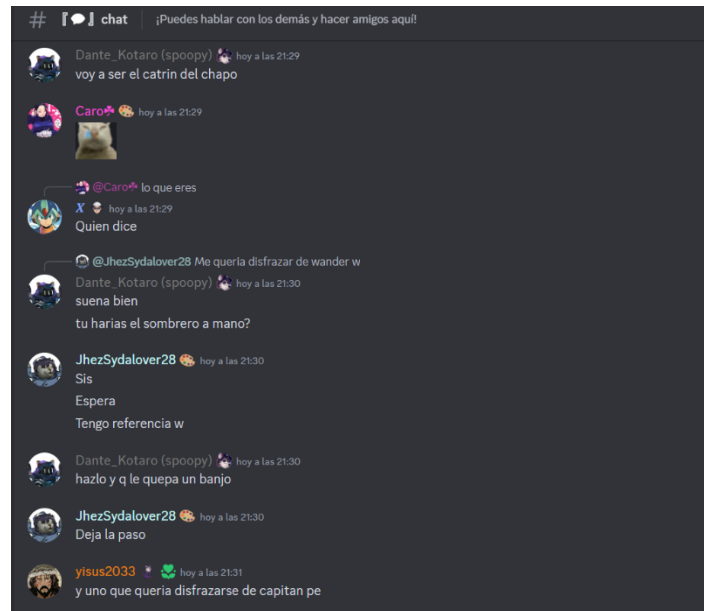
Las apps de banco usan bancked ya que se encarga de procesar transacciones, manejar la logica empresarial y acceder a la base de datos de manera segura. Alunos lenguajes comunes son Java, C++, C#, Python y Go.

While

El while es una estructura de control que se utiliza en programación para repetir una serie de instrucciones mientras una condición específica se cumple. En otras palabras, el while permite que un programa repita ciertas acciones hasta que se deje de cumplir una condición particular. El **while** es una herramienta poderosa que se utiliza para automatizar tareas que deben repetirse varias veces. Imagina que tienes una lista de tareas que quieres que tu programa realice de forma repetitiva hasta que se cumpla una cierta condición.

Ejemplos:

Discord: Discord utiliza un ciclo while para mantener la conexión con los servidores y recibir mensajes en tiempo real. Esto permite que la aplicación esté siempre lista para recibir nuevas entradas, notificaciones o mensajes mientras el usuario está en la plataforma.



Codigo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Simulación de WebSocket de Discord</title>
</head>
<body>
  <h1>Simulación de Conexión en Tiempo Real</h1>
  <p id="estado">Conectando...</p>
  <div id="mensajes"></div>

  <script>
    // URL del servidor WebSocket (aquí se usa una URL de ejemplo)
    const serverUrl = "wss://ejemplo-servidor-websocket.com"; // Cambia a una URL válida
    const socket = new WebSocket(serverUrl);

    // Función para agregar mensajes a la página
    function agregarMensaje(mensaje) {
      const divMensajes = document.getElementById("mensajes");
      const nuevoMensaje = document.createElement("p");
      nuevoMensaje.textContent = mensaje;
      divMensajes.appendChild(nuevoMensaje);
    }

    // Conectar al servidor WebSocket
    socket.addEventListener("open", function (event) {
      document.getElementById("estado").textContent = "Conectado al servidor";
      console.log("Conexión establecida con el servidor WebSocket.");
    });
```

```
    // Escuchar los mensajes entrantes
    socket.addEventListener("message", function (event) {
      console.log("Mensaje recibido del servidor:", event.data);
      agregarMensaje("Nuevo mensaje: " + event.data);
    });

    // Manejar la desconexión
    socket.addEventListener("close", function (event) {
      document.getElementById("estado").textContent = "Desconectado del servidor";
      console.log("Conexión cerrada con el servidor WebSocket.");
    });

    // Manejar errores en la conexión
    socket.addEventListener("error", function (error) {
      console.log("Error en la conexión WebSocket:", error);
      document.getElementById("estado").textContent = "Error en la conexión";
    });

    // Enviar un mensaje al servidor (ejemplo de uso)
    function enviarMensaje(mensaje) {
      if (socket.readyState === WebSocket.OPEN) {
        socket.send(mensaje);
        console.log("Mensaje enviado:", mensaje);
      } else {
        console.log("No se pudo enviar el mensaje: el WebSocket no está conectado.");
      }
    }
  }
}
```

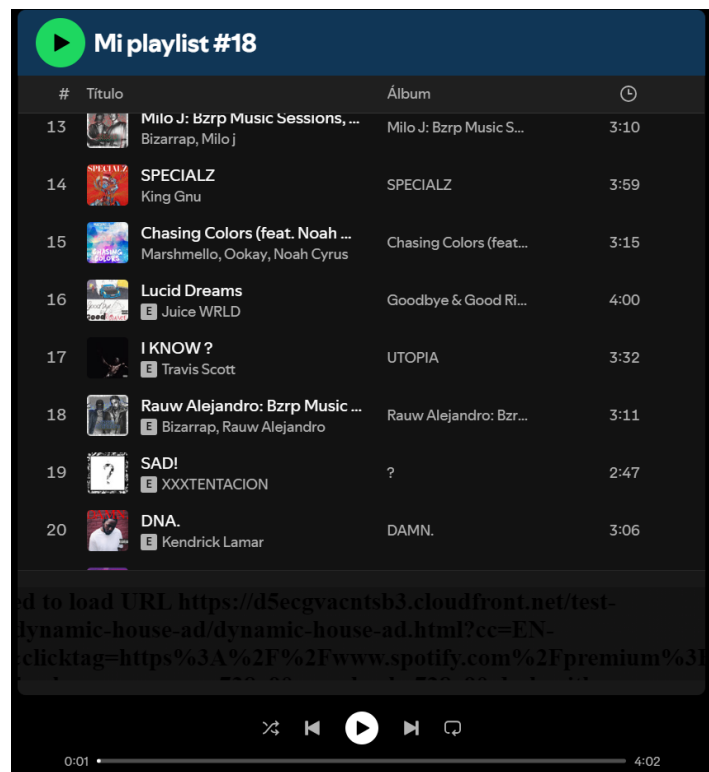
```

// Ejemplo de enviar un mensaje al servidor después de 3 segundos
setTimeout(() => {
    enviarMensaje("¡Hola, servidor!");
}, 3000);

</script>
</body>
</html>

```

Reproducción continua: Mientras el usuario tenga una lista de reproducción activa, un while puede iterar para reproducir el siguiente elemento automáticamente (siguiente canción, episodio, o video).



Codigo:

```
import time

# Clase que representa una Canción
class Cancion:
    def __init__(self, titulo, artista, duracion):
        self.titulo = titulo
        self.artista = artista
        self.duracion = duracion # duración en segundos

    def reproducir(self):
        print(f"Reproduciendo '{self.titulo}' de {self.artista}...")
        time.sleep(self.duracion) # Simula la duración de la canción
        print(f"'{self.titulo}' terminó.\n")

# Clase que representa la Lista de Reproducción
class ListaReproduccion:
    def __init__(self, nombre):
        self.nombre = nombre
        self.canciones = []

    def agregar_cancion(self, cancion):
        self.canciones.append(cancion)

    def reproducir_lista(self):
        print(f"Iniciando la lista de reproducción: {self.nombre}\n")
        indice = 0 # Empezamos con la primera canción
```

```
def reproducir_lista(self):
    print(f"Iniciando la lista de reproducción: {self.nombre}\n")
    indice = 0 # Empezamos con la primera canción

    while indice < len(self.canciones):
        cancion_actual = self.canciones[indice]
        cancion_actual.reproducir()
        indice += 1 # Avanzamos a la siguiente canción

    print("Lista de reproducción completada.")

# Crear algunas canciones de ejemplo
cancion1 = Cancion("Song 1", "Artista 1", 3)
cancion2 = Cancion("Song 2", "Artista 2", 4)
cancion3 = Cancion("Song 3", "Artista 3", 5)

# Crear una lista de reproducción y agregar canciones
mi_lista = ListaReproduccion("Mis Favoritos")
mi_lista.agregar_cancion(cancion1)
mi_lista.agregar_cancion(cancion2)
mi_lista.agregar_cancion(cancion3)

# Reproducir la lista de reproducción
mi_lista.reproducir_lista()
```

Registro de sueño: En las apps de salud y ejercicio, mientras el usuario duerme, un while podría activarse para monitorear la actividad del sueño y registrar las fases de sueño profundo o ligero.



Codigo:

```
from datetime import datetime, timedelta

# Clase que representa un Registro de Sueño
class RegistroSueno:
    def __init__(self, fecha, hora_sueno, hora_despertar):
        self.fecha = fecha
        self.hora_sueno = hora_sueno
        self.hora_despertar = hora_despertar
        self.duracion = self.calcular_duracion()

    def calcular_duracion(self):
        # Calcula la duración entre hora de sueño y hora de despertar
        if self.hora_despertar < self.hora_sueno:
            # Caso donde el sueño se extiende a la madrugada
            self.hora_despertar += timedelta(days=1)
        return self.hora_despertar - self.hora_sueno

    def __str__(self):
        return (f"Fecha: {self.fecha}\n"
                f"Hora de sueño: {self.hora_sueno.time()}\n"
                f"Hora de despertar: {self.hora_despertar.time()}\n"
                f"Duración del sueño: {self.duracion}\n")
```

```

# Clase que representa el Historial de Sueño
class HistorialSueno:
    def __init__(self):
        self.registros = []

    def registrar_sueno(self, hora_sueno, hora_despertar):
        fecha = datetime.now().date()
        registro = RegistroSueno(fecha, hora_sueno, hora_despertar)
        self.registros.append(registro)
        print("Registro de sueño agregado exitosamente.\n")

    def mostrar_historial(self):
        if not self.registros:
            print("No hay registros de sueño.")
            return
        for registro in self.registros:
            print(registro)
            print("-" * 30)

```

```

# Función principal para el registro de sueño
def registrar_sueno_usuario():
    historial = HistorialSueno()
    while True:
        try:
            # Solicitar la hora de sueño
            hora_sueno = input("Ingresa la hora que te dormiste (HH:MM en formato 24 horas): ")
            hora_sueno = datetime.strptime(hora_sueno, "%H:%M")

            # Solicitar la hora de despertar
            hora_despertar = input("Ingresa la hora que despertaste (HH:MM en formato 24 h): ")
            hora_despertar = datetime.strptime(hora_despertar, "%H:%M")

            # Registrar en el historial
            historial.registrar_sueno(hora_sueno, hora_despertar)

            # Mostrar historial
            ver_historial = input("¿Quieres ver el historial de sueño? (s/n): ").strip().lower()
            if ver_historial == 's':
                historial.mostrar_historial()

            # Salir o registrar otro día
            continuar = input("¿Quieres registrar otro día? (s/n): ").strip().lower()
            if continuar != 's':
                break

        except ValueError:
            print("Formato de hora incorrecto. Por favor, intenta nuevamente.")

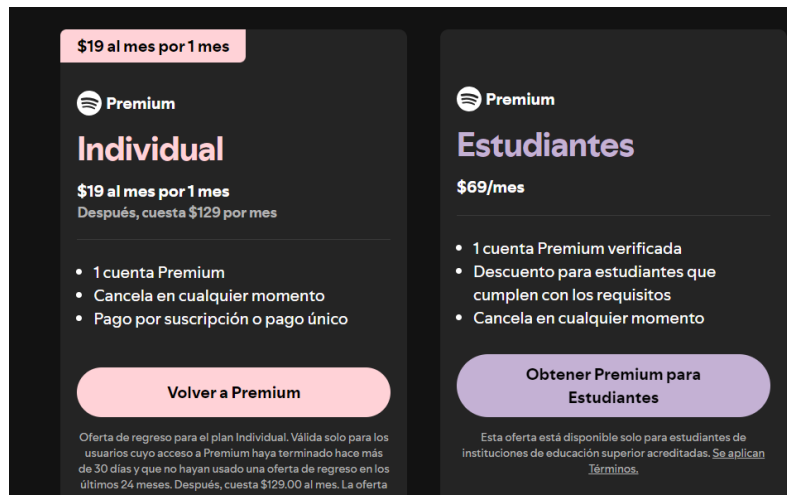
```

Do – While

El ciclo do-while es una estructura de control de flujo en programación que permite ejecutar un bloque de código al menos una vez y luego continuar ejecutándolo mientras se cumpla una condición específica. La principal diferencia con el ciclo while es que en el do-while, la condición se evalúa **después** de ejecutar el bloque de código, lo que garantiza que el código se ejecute al menos una vez.

Ejemplos:

Spotify: En la configuración de preferencias, un ciclo do-while podría usarse para solicitar al usuario que elija un plan de suscripción hasta que se seleccione uno válido.



Código:

```
def mostrar_planes():
    print("Planes de Suscripción de Spotify:")
    print("1. Gratis")
    print("2. Individual")
    print("3. Familiar")
    print("4. Estudiante")

def elegir_plan():
    while True:
        mostrar_planes()
        try:
            opcion = int(input("Selecciona un plan de suscripción (1-4): "))

            # Verificamos que la opción esté en el rango válido
            if opcion in [1, 2, 3, 4]:
                if opcion == 1:
                    print("Has seleccionado el plan Gratis.")
                elif opcion == 2:
                    print("Has seleccionado el plan Individual.")
                elif opcion == 3:
                    print("Has seleccionado el plan Familiar.")
                elif opcion == 4:
                    print("Has seleccionado el plan Estudiante.")
                break # Salir del bucle cuando elige una opción válida
            else:
                print("Opción no válida. Por favor, selecciona un número entre 1 y 4.")
        except ValueError:
            print("Opción no válida. Por favor, selecciona un número entre 1 y 4.")
```

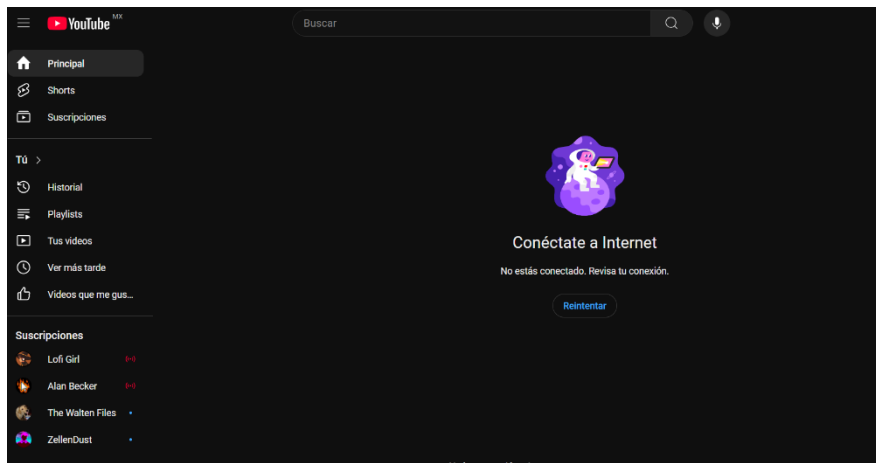
```

except ValueError:
    print("Entrada inválida. Por favor, ingresa un número entre 1 y 4.")

# Llamar a la función para que el usuario seleccione un plan
elegir_plan()

```

Reconexión en caso de pérdida de red: Si la conexión se pierde, un do-while puede intentar la reconexión automáticamente al menos una vez y continuar haciéndolo mientras la conexión siga fallando.



Codigo:

```

import time
import random

def conectar_al_servidor():
    """
    Simula la conexión a un servidor.
    Devuelve True si la conexión es exitosa, y False si falla.
    """
    # Simulación: 30% de probabilidad de éxito en la conexión
    return random.random() < 0.3

def intentar_conexion():
    intentos = 0

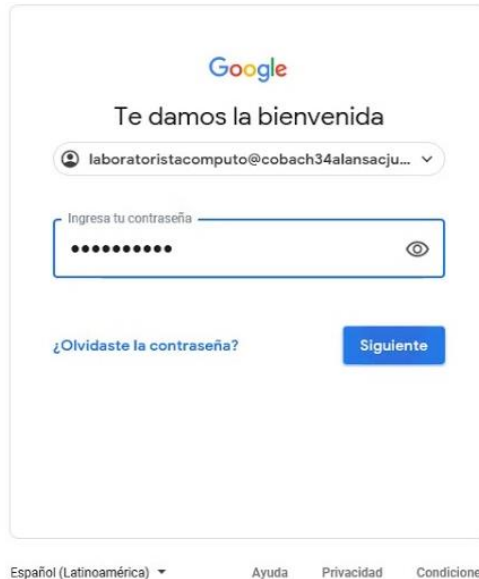
    while True:
        intentos += 1
        print(f"Intento de conexión #{intentos}...")

        if conectar_al_servidor():
            print("Conexión establecida exitosamente.")
            break # Salir del bucle si la conexión es exitosa
        else:
            print("Conexión fallida. Reintentando en 5 segundos...\n")
            time.sleep(5) # Espera antes de reintentar

# Ejecutar la función para intentar la conexión
intentar_conexion()

```


Verificación de PIN o contraseña: Al iniciar sesión, se solicita al menos una vez que el usuario ingrese su contraseña o un código de autenticación. Si es incorrecto, el ciclo do-while repite el proceso hasta que se ingrese el código correcto o se superen los intentos permitidos.



Código:

```
# PIN correcto almacenado
PIN_CORRECTO = "1234"

def verificar_pin():
    intentos = 0
    max_intentos = 3

    while intentos < max_intentos:
        # Solicitar al usuario que ingrese el PIN
        pin_ingresado = input("Por favor, ingresa tu PIN: ")

        # Verificar si el PIN es correcto
        if pin_ingresado == PIN_CORRECTO:
            print("PIN correcto. Acceso concedido.")
            return True # Salir de la función si el PIN es correcto
        else:
            intentos += 1
            print(f"PIN incorrecto. Intento {intentos} de {max_intentos}.")

    # Si se superan los intentos permitidos
    print("Demasiados intentos fallidos. Acceso bloqueado.")
    return False

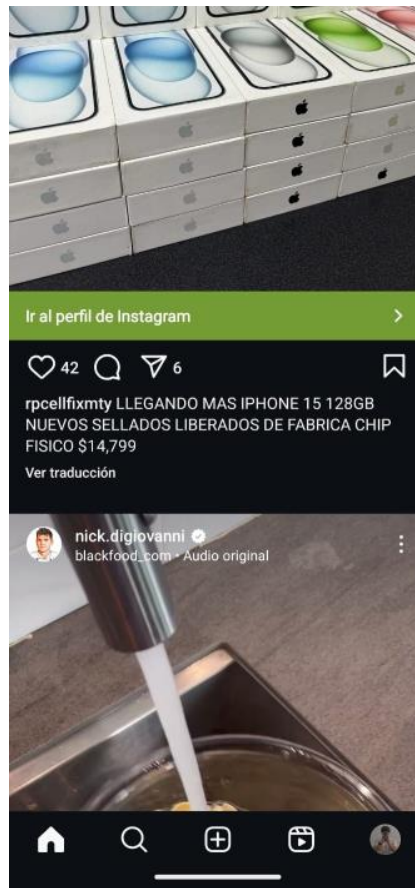
# Ejecutar la función para verificar el PIN
verificar_pin()
```

Estructura desde para (For)

El **for** en programación es una estructura de control utilizada para crear ciclos o bucles. Un ciclo es una serie de instrucciones que se ejecutan repetidamente mientras se cumple una condición específica. En esencia, el **for** te permite automatizar tareas repetitivas, lo que ahorra tiempo y reduce la repetición de código. El **for** es especialmente útil cuando necesitas realizar una acción un número conocido de veces o cuando deseas iterar sobre una colección de datos, como una lista o un arreglo. Su uso más común es recorrer una secuencia de elementos y aplicar una serie de instrucciones a cada uno de ellos.

Ejemplos:

Instagram: Utiliza ciclos **for** para cargar y mostrar publicaciones en el feed del usuario, iterando a través de las imágenes y descripciones.



Código:

```
# Lista de publicaciones en el feed, cada una con una imagen y descripción
feed = [
    {"imagen": "🌅 Imagen de un amanecer en la playa", "descripcion": "Comienza el día con"},
    {"imagen": "🐶 Imagen de un cachorro jugando", "descripcion": "No hay nada como la ale"},
    {"imagen": "🌃 Imagen de la ciudad al atardecer", "descripcion": "La ciudad bajo un ci"},
    {"imagen": "🍜 Imagen de un plato de ramen", "descripcion": "Delicioso ramen casero 🍜"},
    {"imagen": "🏔️ Imagen de una montaña nevada", "descripcion": "Aventuras en la cima del"}
]

def mostrar_feed(feed):
    # Iterar a través de cada publicación en el feed
    for index, publicacion in enumerate(feed, start=1):
        print(f"Publicación #{index}")
        print(publicacion["imagen"])
        print(publicacion["descripcion"])
        print("-" * 40) # Separador entre publicaciones

# Llamar a la función para mostrar el feed
mostrar_feed(feed)
```

Twitter (X): Cuando haces “scroll” en twitter te va generando nuevas publicaciones de usuarios, haciendo que cada vez que vas bajando se vayan cargando mas publicaciones.



Código:

```
import time

# Lista inicial de publicaciones
publicaciones_iniciales = [
    "Publicación 1: Buen día, Twitter! ☀️",
    "Publicación 2: Comenzando el día con una taza de café ☕",
    "Publicación 3: ¿Alguien ha visto el último episodio de la serie X? 📺",
    "Publicación 4: ¡Feliz viernes! 🎉",
    "Publicación 5: El clima está increíble hoy 🌤️"
]

# Lista de nuevas publicaciones que se cargarían al hacer scroll
nuevas_publicaciones = [
    "Publicación 6: Alguien quiere ir a correr esta tarde? 🏃",
    "Publicación 7: ¿Qué opinan sobre la película Y? 🎬",
    "Publicación 8: No puedo esperar para el concierto de mañana 🎵",
    "Publicación 9: ¡Es hora de la pizza! 🍕",
    "Publicación 10: Buenas noches, Twitter! 🌙"
]
```

```
def cargar_feed():
    # Comienza con las publicaciones iniciales
    publicaciones = publicaciones_iniciales.copy()

    while True:
        # Mostrar las publicaciones actuales
        for publicacion in publicaciones:
            print(publicacion)

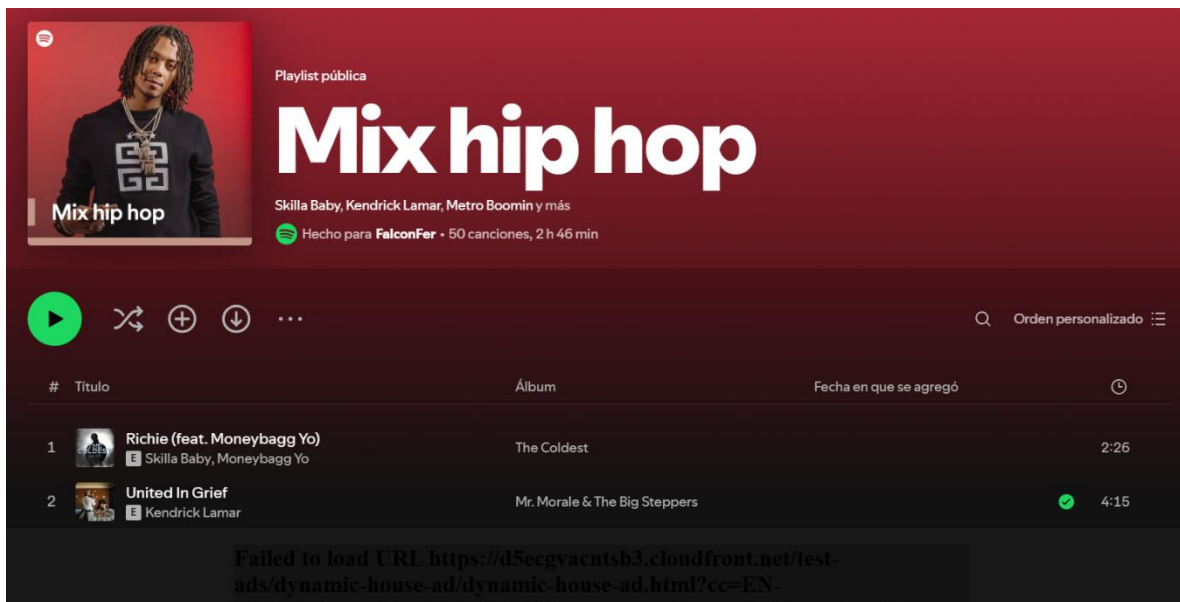
        print("\n----- Más publicaciones cargando... -----")
        time.sleep(1) # Simula el tiempo de carga de nuevas publicaciones

        # Simula el scroll al llegar al final del feed
        usuario_quiere_scroll = input("¿Quieres hacer scroll para ver más publicaciones? (s/n): ")

        if usuario_quiere_scroll == 's':
            # Añadir más publicaciones al feed
            publicaciones.extend(nuevas_publicaciones)
            nuevas_publicaciones.clear() # Ya no hay más publicaciones para cargar después
        else:
            print("Has salido del feed.")
            break

    # Ejecutar la función para cargar el feed con desplazamiento
    cargar_feed()
```

Recomendacion de canciones: Spotify podría usar un ciclo for para mostrar recomendaciones de canciones basadas en una lista de canciones que ya te han gustado. Por ejemplo, al entrar en la sección de recomendaciones, la app puede analizar las canciones similares y generar una lista.



Codigo:

```
# Lista de canciones que al usuario le han gustado
canciones_gustadas = [
    {"titulo": "Blinding Lights", "artista": "The Weeknd", "genero": "Pop"},
    {"titulo": "Levitating", "artista": "Dua Lipa", "genero": "Pop"},
    {"titulo": "Shape of You", "artista": "Ed Sheeran", "genero": "Pop"},
    {"titulo": "Good 4 U", "artista": "Olivia Rodrigo", "genero": "Pop Punk"},
    {"titulo": "Save Your Tears", "artista": "The Weeknd", "genero": "Pop"}
]

# Lista de canciones disponibles para recomendar
canciones_disponibles = [
    {"titulo": "Kiss Me More", "artista": "Doja Cat", "genero": "Pop"},
    {"titulo": "Watermelon Sugar", "artista": "Harry Styles", "genero": "Pop"},
    {"titulo": "Driver's License", "artista": "Olivia Rodrigo", "genero": "Pop"},
    {"titulo": "Midnight City", "artista": "M83", "genero": "Electronic"},
    {"titulo": "Happier Than Ever", "artista": "Billie Eilish", "genero": "Pop"},
    {"titulo": "Good 4 U (Acoustic)", "artista": "Olivia Rodrigo", "genero": "Pop Punk"},
    {"titulo": "Don't Start Now", "artista": "Dua Lipa", "genero": "Pop"}
]
```

```
def recomendar_canciones(canciones_gustadas, canciones_disponibles):  
    # Crear una lista de géneros que el usuario ha disfrutado  
    generos_gustados = [cancion["genero"] for cancion in canciones_gustadas]  
  
    # Recomendaciones basadas en los géneros de las canciones que le gustan al usuario  
    recomendaciones = []  
  
    for cancion in canciones_disponibles:  
        if cancion["genero"] in generos_gustados: # Si el género de la canción está en lo  
            recomendaciones.append(cancion)  
  
    # Mostrar las canciones recomendadas  
    print("Canciones recomendadas basadas en tus gustos:")  
    for recomendacion in recomendaciones:  
        print(f"- {recomendacion['titulo']} de {recomendacion['artista']} ({recomendacion['  
  
# Llamar a la función para recomendar canciones  
recomendar_canciones(canciones_gustadas, canciones_disponibles)
```

Arreglos

Un arreglo es una estructura de datos finita, homogénea y ordenada que permite almacenar un conjunto de elementos del mismo tipo, facilitando el acceso directo a cada elemento mediante un índice. Esta estructura es esencial en programación para organizar y manejar grandes cantidades de datos de manera secuencial, permitiendo operaciones como la búsqueda y la modificación de elementos con eficiencia.

Al ser homogéneos, todos los elementos en un arreglo deben ser del mismo tipo (por ejemplo, enteros, caracteres o cadenas de texto). Los arreglos son fundamentales en algoritmos y aplicaciones de software, donde una gestión ordenada de datos es crucial para el rendimiento y la claridad del programa.

Unidimensionales:

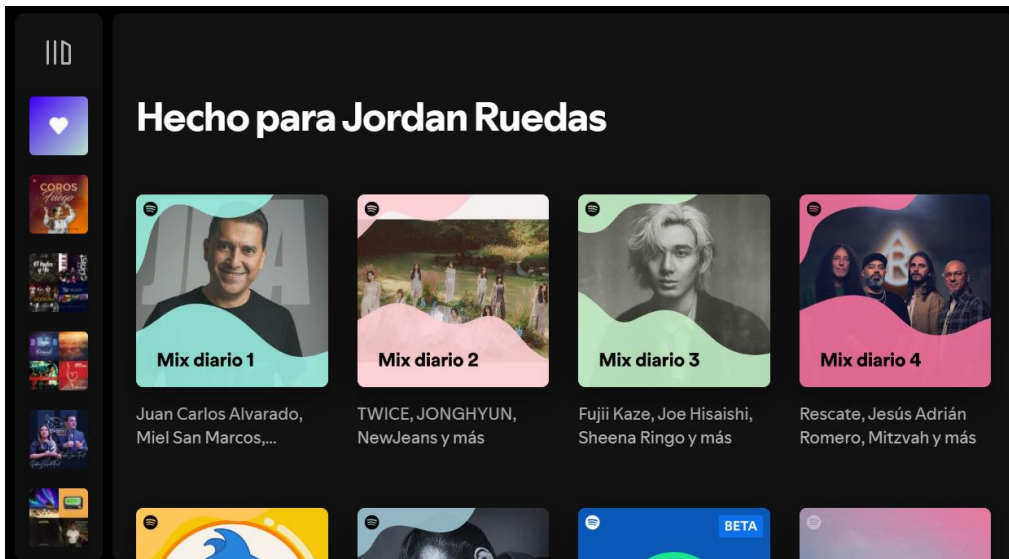
Los arreglos unidimensionales, también conocidos como **vectores**, son una colección lineal de datos donde cada elemento se accede mediante un único índice. Esta estructura resulta adecuada para almacenar secuencias simples de información, como listas de números o cadenas de texto. Los elementos se organizan de forma consecutiva en la memoria, permitiendo un acceso eficiente y rápido a cada uno de ellos.

En programación, los arreglos unidimensionales simplifican la manipulación de datos, ya que los elementos se encuentran en posiciones predecibles. Su estructura básica y accesibilidad los convierten en una herramienta esencial para implementar algoritmos y realizar operaciones en datos organizados linealmente.

Ejemplos:

Spotify

Spotify utiliza arreglos unidimensionales para almacenar listas de reproducción de canciones. Cada lista se organiza de forma secuencial, permitiendo al usuario acceder a cada canción en un orden específico o aleatorio, facilitando así la reproducción continua y ordenada de pistas.



Código: El código utiliza un arreglo unidimensional llamado **playlist** para almacenar de manera secuencial los títulos de las canciones, lo que permite acceder a cada canción por su índice, facilitando la reproducción en orden. Al hacer clic en el botón "Play Next Song", la función **playNextSong** incrementa el índice actual (**currentIndex**) y muestra la canción correspondiente en la interfaz, asegurando que se reproduzcan las canciones en el orden en que están almacenadas en el arreglo, y al llegar al final del mismo, informa al usuario que ha alcanzado el "Fin de la Lista".

```

<div class="playlist">
  <h2>Playlist</h2>
  <ul id="songList"></ul>
  <button onclick="playNextSong()">Play Next Song</button>
  <p id="currentSong"></p>
</div>

<script>
  const playlist = [
    "Song 1 - Artist A",
    "Song 2 - Artist B",
    "Song 3 - Artist C",
    "Song 4 - Artist D",
    "Song 5 - Artist E"
  ];

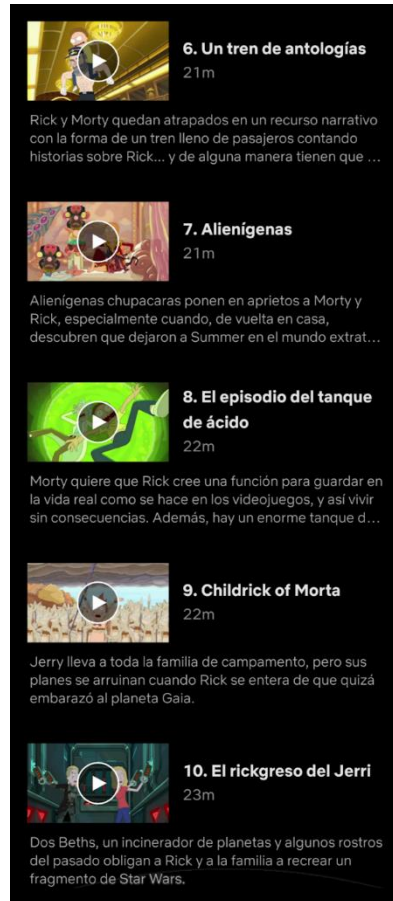
  let currentIndex = 0;
  function displayPlaylist() {
    const songList = document.getElementById('songList');
    playlist.forEach((song) => {
      const listItem = document.createElement('li');
      listItem.textContent = song;
      songList.appendChild(listItem);
    });
  }

  function playNextSong() {
    if (currentIndex < playlist.length) {
      document.getElementById('currentSong').textContent = "Now Playing: " + playlist[currentIndex];
      currentIndex++;
    } else {
      document.getElementById('currentSong').textContent = "End of Playlist";
    }
  }
}

```


Netflix

En Netflix, un arreglo unidimensional se emplea para almacenar y mostrar los episodios de una temporada de serie. Cada episodio tiene un índice único que permite al usuario acceder a los episodios de forma consecutiva o seleccionar uno específico desde el arreglo.



Código: El código utiliza un arreglo unidimensional llamado **episodes** para almacenar los títulos de los episodios de una temporada de una serie, lo que permite acceder a cada episodio mediante su índice único. Al cargar la página, la función **displayEpisodes** itera sobre este arreglo y genera una lista de elementos en la interfaz, donde cada episodio se presenta como un elemento clicable. Cuando el usuario selecciona un episodio, la función **selectEpisode** actualiza el texto en la pantalla para mostrar cuál episodio se está viendo actualmente, facilitando así la navegación secuencial o la selección específica de episodios desde el arreglo.

```

</head>
<body>

  <h1>Netflix Series Episodes</h1>
  <div>
    <ul id="episodeList"></ul>
  </div>
  <p id="currentEpisode">Select an episode to watch.</p>

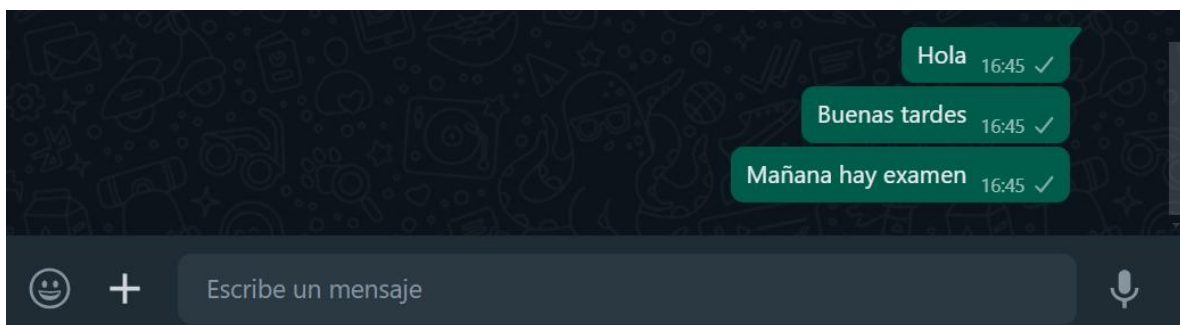
  <script>
    const episodes = [
      "Episode 1: The Beginning",
      "Episode 2: The Journey",
      "Episode 3: The Conflict",
      "Episode 4: The Resolution",
      "Episode 5: The Finale"
    ];

    function displayEpisodes() {
      const episodeList = document.getElementById('episodeList');
      episodes.forEach((episode, index) => {
        const li = document.createElement('li');
        li.textContent = episode;
        li.onclick = () => selectEpisode(index);
        episodeList.appendChild(li);
      });
    }
  </script>

```

WhatsApp

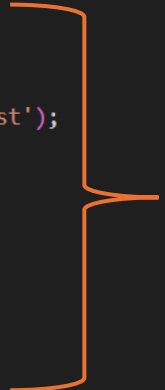
WhatsApp usa arreglos unidimensionales para gestionar los mensajes dentro de una conversación. Cada mensaje ocupa una posición única en el arreglo de mensajes del chat, permitiendo un acceso secuencial para visualizar los mensajes en el orden en el que fueron enviados o recibidos.



Código: El código utiliza un arreglo unidimensional llamado **messages** para gestionar los mensajes dentro de una conversación de chat, donde cada mensaje se almacena como un objeto que contiene el texto del mensaje y su tipo (enviado o recibido). Al enviar un nuevo mensaje, este se añade al final del arreglo, permitiendo un acceso secuencial para visualizar los mensajes en el orden en que fueron enviados o recibidos. La función **displayMessages** se encarga de actualizar la interfaz para mostrar todos los mensajes almacenados en el arreglo, asegurando que se mantenga el orden cronológico y que los mensajes más recientes aparezcan en la parte inferior del área de chat. Además, la simulación de un mensaje recibido se realiza al azar, añadiendo así más interactividad a la experiencia de chat.

```
<div id="messageList"></div>
<input type="text" id="messageInput" placeholder="Type a message..." />
<button onclick="sendMessage()">Send</button>
```

```
<script>
  const messages = [];
  function displayMessages() {
    const messageList = document.getElementById('messageList');
    messageList.innerHTML = '';
    messages.forEach((msg, index) => {
      const div = document.createElement('div');
      div.className = `message ${msg.type}`;
      div.textContent = msg.text;
      messageList.appendChild(div);
    });
    messageList.scrollTop = messageList.scrollHeight;
  }
  function sendMessage() {
    const messageInput = document.getElementById('messageInput');
    const text = messageInput.value.trim();
    if (text) {
      messages.push({ text: text, type: 'sent' });
      messageInput.value = '';
      displayMessages();
      receiveMessage();
    }
  }
</script>
```



Multidimensionales:

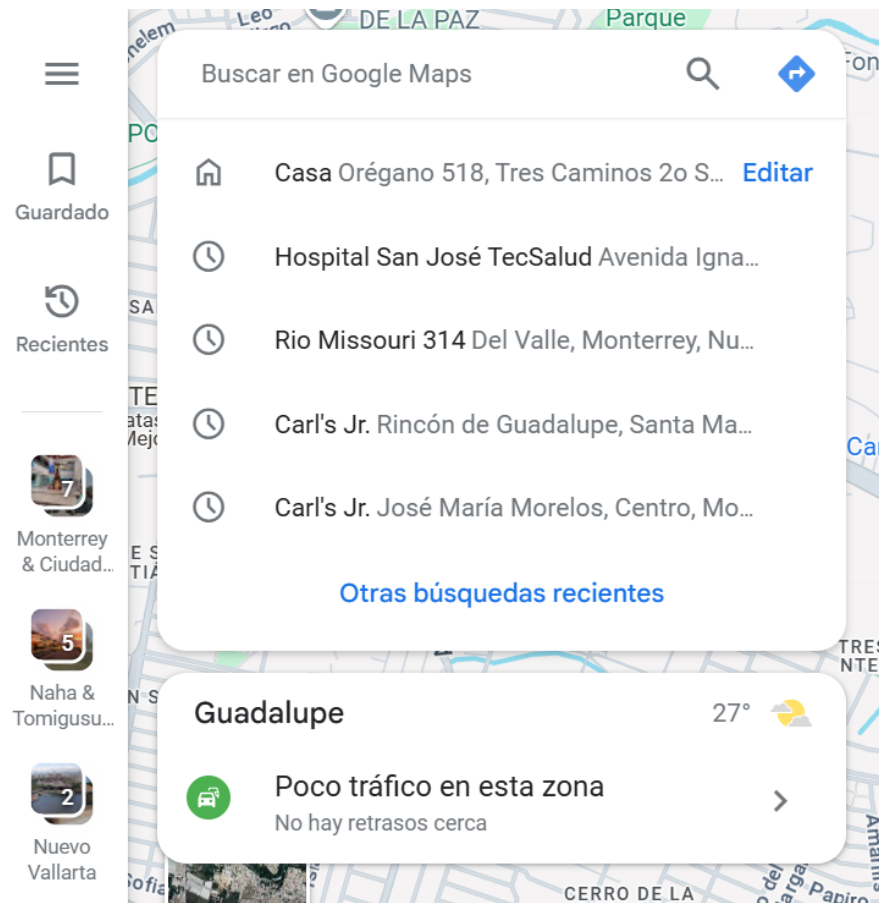
Los arreglos multidimensionales extienden la estructura de un arreglo a múltiples niveles, permitiendo organizar datos en dos o más dimensiones, como tablas (bidimensionales) o incluso estructuras más complejas (tridimensionales o superiores). Cada dimensión adicional en el arreglo agrega un índice, lo que facilita el modelado de estructuras jerárquicas de datos, como matrices o matrices tridimensionales, útiles en programación gráfica y cálculos complejos.

Estas estructuras permiten almacenar y gestionar información en varias dimensiones, aunque requieren mayor memoria y un manejo más complejo que los arreglos unidimensionales. Sin embargo, su flexibilidad en la organización de datos en múltiples niveles los hace muy útiles en aplicaciones que requieren un manejo sofisticado y detallado de la información.

Ejemplos:

Google Maps

Google Maps utiliza arreglos bidimensionales para representar mapas, donde cada coordenada de latitud y longitud se organiza en una "tabla" de ubicaciones. Esto permite manejar y mostrar rutas, distancias y posiciones de forma organizada y accesible para representar el espacio geográfico.



Código: El código utiliza un arreglo bidimensional llamado **locations** para representar un conjunto de ubicaciones en un formato de tabla, donde cada elemento del arreglo contiene un objeto con coordenadas de latitud y longitud. Este arreglo se organiza en filas y columnas, lo que permite visualizar de manera estructurada las ubicaciones en una cuadrícula en la interfaz gráfica. La función **displayMap** itera a través de cada fila y columna del arreglo, creando un elemento visual para cada ubicación que muestra sus coordenadas y permite al usuario interactuar con ellas al hacer clic, lo que genera una alerta con las coordenadas específicas. Esta estructura bidimensional facilita la representación y gestión de múltiples ubicaciones geográficas de forma organizada y accesible.

```
<div id="map"></div>

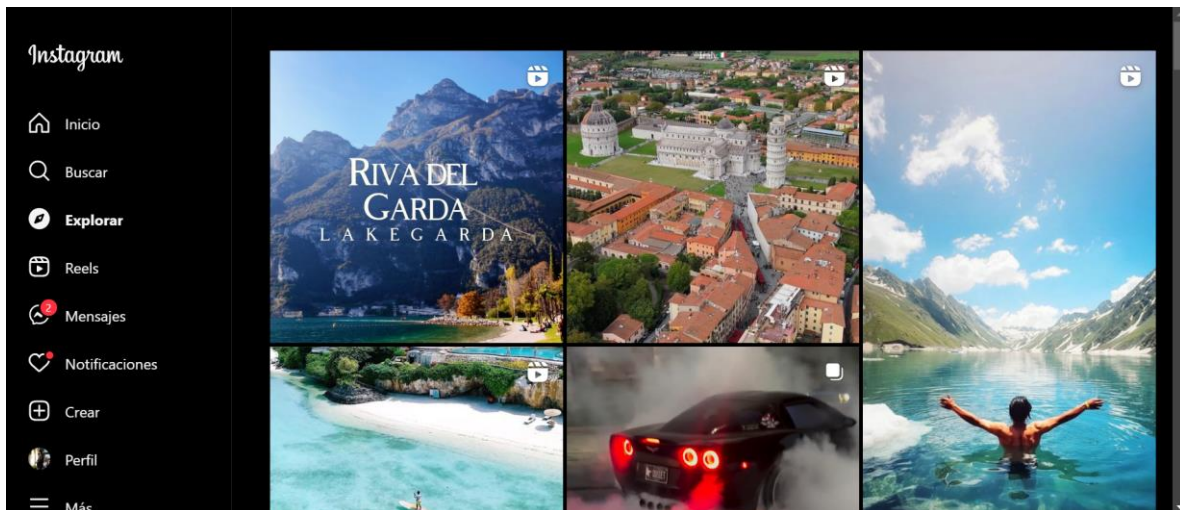
<script>
  const locations = [
    [{ lat: 34.0522, lng: -118.2437 }, { lat: 34.0522, lng: -118.2430 }, { lat: 34.0522, lng: -118.2420 }, { lat: 34.0522, lng: -118.2410 }, {
    [{ lat: 34.0520, lng: -118.2437 }, { lat: 34.0520, lng: -118.2430 }, { lat: 34.0520, lng: -118.2420 }, { lat: 34.0520, lng: -118.2410 }, {
    [{ lat: 34.0515, lng: -118.2437 }, { lat: 34.0515, lng: -118.2430 }, { lat: 34.0515, lng: -118.2420 }, { lat: 34.0515, lng: -118.2410 }, {
    [{ lat: 34.0510, lng: -118.2437 }, { lat: 34.0510, lng: -118.2430 }, { lat: 34.0510, lng: -118.2420 }, { lat: 34.0510, lng: -118.2410 }, {
    [{ lat: 34.0505, lng: -118.2437 }, { lat: 34.0505, lng: -118.2430 }, { lat: 34.0505, lng: -118.2420 }, { lat: 34.0505, lng: -118.2410 }, {
  ];

  function displayMap() {
    const mapElement = document.getElementById('map');
    locations.forEach((row, rowIndex) => {
      row.forEach((location, colIndex) => {
        const locationDiv = document.createElement('div');
        locationDiv.className = 'location';
        locationDiv.textContent = `Lat: ${location.lat}, Lng: ${location.lng}`;
        locationDiv.onclick = () => alert(`Coordinates: ${location.lat}, ${location.lng}`);
        mapElement.appendChild(locationDiv);
      });
    });
  }

  displayMap();
</script>
```

Instagram

Instagram emplea arreglos bidimensionales para almacenar y mostrar las imágenes de una galería en un perfil, organizadas en filas y columnas. Esto permite visualizar varias fotos al mismo tiempo en un formato de cuadrícula, facilitando la navegación y el acceso rápido a cualquier publicación en el perfil del usuario.



Código: El código implementa una estructura bidimensional mediante un arreglo llamado **galleryImages**, que almacena las URLs de las imágenes organizadas en filas y columnas, simulando una galería de perfil al estilo de Instagram. Cada subarreglo representa una fila de imágenes, lo que permite que

se muestren múltiples fotos simultáneamente en un formato de cuadrícula en la interfaz visual. La función **displayGallery** recorre este arreglo y crea elementos de imagen en un contenedor, añadiéndolos al elemento de galería en el HTML, lo que facilita la navegación y el acceso rápido a las publicaciones del usuario al presentar las imágenes de manera estructurada y atractiva.

```
<div id="gallery"></div>

<script>
  const galleryImages = [
    ['image1.jpg', 'image2.jpg', 'image3.jpg'],
    ['image4.jpg', 'image5.jpg', 'image6.jpg'],
    ['image7.jpg', 'image8.jpg', 'image9.jpg']
  ];

  function displayGallery() {
    const galleryElement = document.getElementById('gallery');
    galleryImages.forEach(row => {
      row.forEach(image => {
        const imageContainer = document.createElement('div');
        imageContainer.className = 'image-container';
        const img = document.createElement('img');
        img.src = image;
        img.alt = 'Gallery Image';
        imageContainer.appendChild(img);
        galleryElement.appendChild(imageContainer);
      });
    });
  }

  displayGallery();
</script>
```

Matrices y Vectores

Los **vectores** y **matrices** son estructuras de datos utilizadas para organizar y almacenar datos en una disposición ordenada. Un vector es un tipo especial de matriz donde solo hay una dimensión, es decir, es una fila o una columna de elementos. Estos elementos, todos del mismo tipo, se colocan en una secuencia lineal, como una lista. Los vectores son ideales cuando se requiere manejar una colección simple de datos, como una serie de números o cadenas de texto.

Las **matrices**, por otro lado, son arreglos bidimensionales que organizan los datos en filas y columnas, formando una estructura rectangular. Esta disposición permite acceder a cada elemento utilizando dos índices: uno para la fila y otro para la columna. Las matrices son esenciales en áreas como matemáticas, gráficos por computadora y aprendizaje automático, ya que permiten realizar operaciones complejas sobre los datos y organizar información en tablas, donde cada fila puede representar una observación y cada columna, una característica.

Ejemplos:

WhatsApp: En los chats grupales, los mensajes de cada usuario se pueden almacenar en un vector que representa la conversación en una línea temporal. Cada elemento es un mensaje en orden cronológico, permitiendo acceder a cada mensaje mediante su posición.



Código: El código utiliza una estructura de datos unidimensional, específicamente un vector (arreglo) llamado **chatMessages**, que almacena objetos representando los mensajes de un chat grupal en WhatsApp, donde cada objeto contiene información sobre el usuario que envió el mensaje, el contenido del mensaje y una marca de tiempo. Esta organización permite que los mensajes se mantengan en un orden cronológico, facilitando su visualización en una interfaz de chat que simula la experiencia real de la aplicación. La función **displayChat** recorre este vector y genera dinámicamente los elementos HTML correspondientes para cada mensaje, mostrando así la conversación de manera estructurada y accesible, permitiendo que cada mensaje se pueda identificar fácilmente por su posición en el vector.

```

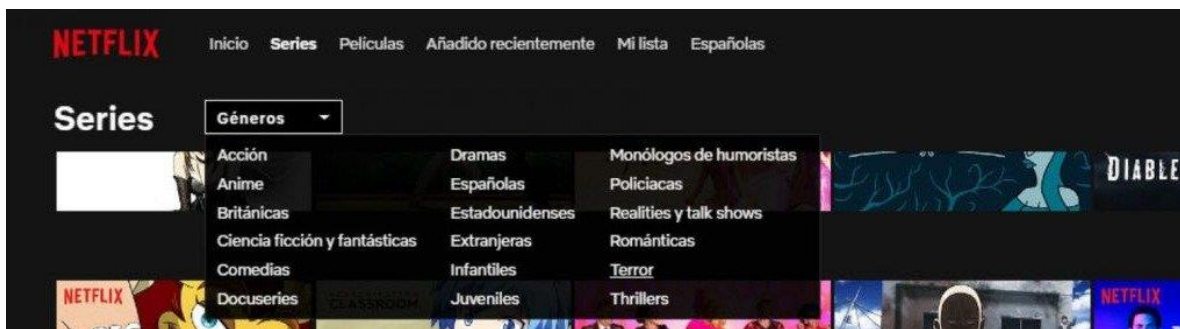
<body>
  <h1>WhatsApp Group Chat Simulation</h1>
  <div id="chat"></div>
</body>

<script>
  const chatMessages = [
    { user: 'User 1', message: 'Hello everyone!', timestamp: '10:00 AM' },
    { user: 'User 2', message: 'Hi! How are you?', timestamp: '10:01 AM' },
    { user: 'User 1', message: 'I am good, thanks! And you?', timestamp: '10:02 AM' },
    { user: 'User 2', message: 'Doing well, just busy with work.', timestamp: '10:03 AM' },
    { user: 'User 1', message: 'Same here. Any plans for the weekend?', timestamp: '10:04 AM' },
    { user: 'User 2', message: 'Thinking of going hiking.', timestamp: '10:05 AM' },
    { user: 'User 1', message: 'Sounds fun! Count me in.', timestamp: '10:06 AM' }
  ];

  function displayChat() {
    const chatElement = document.getElementById('chat');
    chatMessages.forEach((msg, index) => {
      const messageDiv = document.createElement('div');

```

Netflix: Los catálogos de contenido se organizan en matrices, donde cada fila representa un título y cada columna detalla atributos como género, duración, popularidad, y recomendación. Esto permite una organización rápida y eficiente del contenido en categorías.



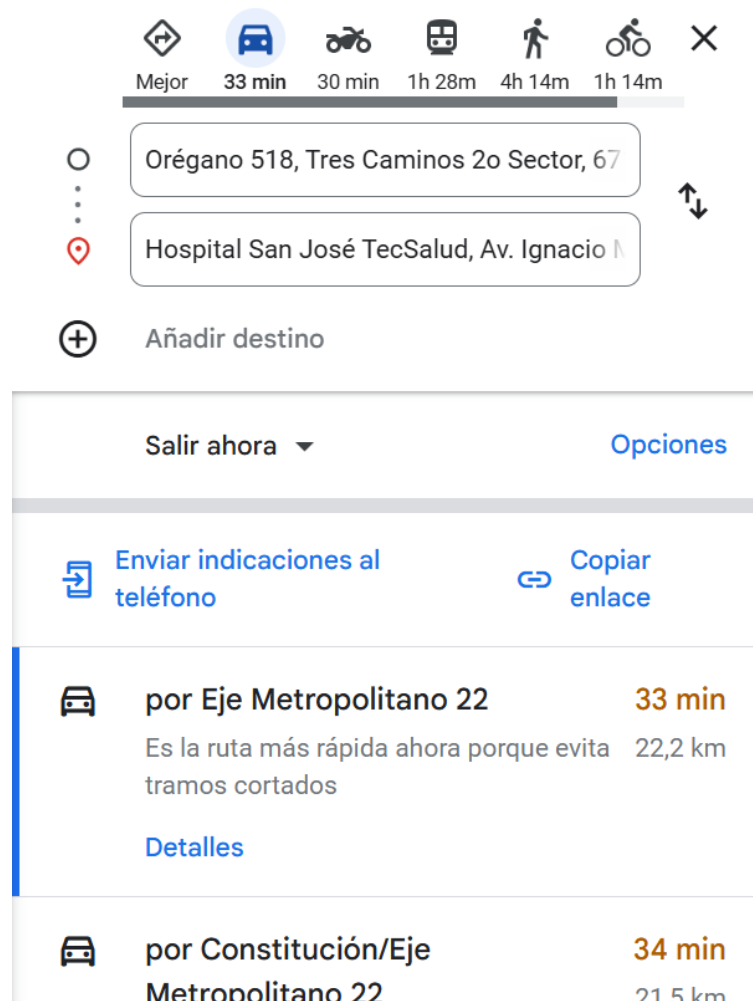
Código: El código utiliza una estructura de datos bidimensional, específicamente una matriz (array de arrays) llamada **contentCatalog**, que organiza el catálogo de contenido de Netflix, donde cada fila representa un título y cada columna detalla atributos como género, duración, popularidad y recomendación. Esta organización permite una visualización clara y estructurada de la información en una tabla HTML, facilitando la rápida identificación y categorización de los títulos. La función **displayCatalog** recorre la matriz y genera dinámicamente las filas y celdas de la tabla, mostrando así el contenido de manera eficiente y accesible para el usuario.


```
<h1>Netflix Content Catalog</h1>
<table>
  <thead>
    <tr>
      <th>Title</th>
      <th>Genre</th>
      <th>Duration</th>
      <th>Popularity</th>
      <th>Recommendation</th>
    </tr>
  </thead>
  <tbody id="catalog"></tbody>
</table>

<script>
  const contentCatalog = [
    ["Stranger Things", "Sci-Fi", "3 Seasons", "High", "Recommended"],
    ["The Crown", "Drama", "4 Seasons", "High", "Highly Recommended"],
    ["Breaking Bad", "Crime", "5 Seasons", "Very High", "Must Watch"],
    ["Money Heist", "Thriller", "5 Seasons", "Very High", "Must Watch"],
    ["The Witcher", "Fantasy", "2 Seasons", "High", "Recommended"],
    ["Bridgerton", "Romance", "2 Seasons", "High", "Recommended"],
    ["Squid Game", "Drama", "1 Season", "Very High", "Must Watch"]
  ];

  function displayCatalog() {
    const catalogElement = document.getElementById('catalog');
    contentCatalog.forEach(row => {
      const tr = document.createElement('tr');
      row.forEach(cell => {
        const td = document.createElement('td');
        td.textContent = cell;
        tr.appendChild(td);
      });
    });
  }
</script>
```

Google Maps: El sistema de rutas y distancias puede representarse en una matriz, donde cada fila y columna representan ubicaciones y los valores indican la distancia o el tiempo de viaje entre ellas. Esto es útil para calcular rutas óptimas entre distintas ubicaciones.



Código: El código utiliza una estructura de datos bidimensional, conocida como matriz, llamada **distanceMatrix**, que representa las distancias entre diferentes ubicaciones, donde cada fila y cada columna corresponden a una ubicación específica y los valores dentro de la matriz indican la distancia (en kilómetros) entre ellas. Esta organización permite visualizar de manera clara y estructurada las distancias entre las ubicaciones, facilitando el cálculo de rutas óptimas. La función **displayDistanceMatrix** recorre la matriz y genera dinámicamente las filas y celdas de una tabla HTML, mostrando así la información de forma accesible y comprensible para el usuario.

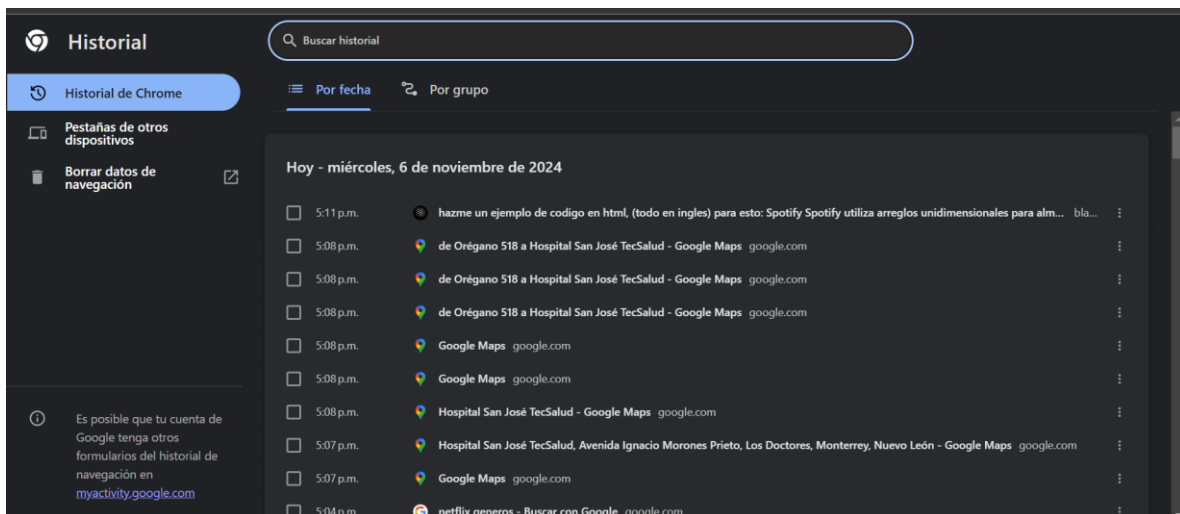
```
<h1>Google Maps Distance Matrix</h1>
<table>
  <thead>
    <tr>
      <th>Location</th>
      <th>Location A</th>
      <th>Location B</th>
      <th>Location C</th>
      <th>Location D</th>
    </tr>
  </thead>
  <tbody id="distanceMatrix"></tbody>
</table>
<script>
  const distanceMatrix = [
    [0, 5, 10, 15],
    [5, 0, 7, 12],
    [10, 7, 0, 8],
    [15, 12, 8, 0]
  ];
  const locations = ['Location A', 'Location B', 'Location C', 'Location D'];
  function displayDistanceMatrix() {
    const matrixElement = document.getElementById('distanceMatrix');
    distanceMatrix.forEach((row, index) => {
      const tr = document.createElement('tr');
      const th = document.createElement('th');
      th.textContent = locations[index];
      tr.appendChild(th);
      row.forEach(distance => {
        const td = document.createElement('td');
```

Pilas

Las pilas son estructuras de datos lineales que funcionan siguiendo el principio LIFO (Last In, First Out), es decir, el último elemento que se añade es el primero que se retira. En una pila, los datos se almacenan uno encima de otro, de manera que sólo se puede acceder al elemento en la parte superior de la estructura. Las principales operaciones de una pila son push, que agrega un nuevo elemento al tope, y pop, que lo elimina. Esta estructura se representa gráficamente como una serie de elementos apilados verticalmente, donde solo el último elemento añadido es accesible de inmediato.

Ejemplos:

□ **Navegadores web (Google Chrome, Safari, etc.):** Los navegadores utilizan pilas para gestionar el historial de navegación. Cuando un usuario visita una nueva página, esta se apila en la estructura, permitiendo que al presionar el botón de "atrás", se mueva al último sitio visitado. Esto funciona según el principio LIFO, ya que la última página agregada al historial es la primera que se visualiza al retroceder.



Código: El código simula la gestión del historial de navegación de un navegador web utilizando una estructura de datos tipo pila, donde las páginas visitadas se apilan en un arreglo denominado **historyStack**. Cada vez que el usuario "visita" una nueva página, esta se agrega a la parte superior de la pila, y al presionar el botón "Atrás", se elimina la última página añadida, siguiendo el principio LIFO (Last In, First Out), donde la última página visitada es la primera en ser desechada. La interfaz actualiza dinámicamente la visualización del historial, permitiendo al usuario ver las páginas visitadas y gestionar su navegación de manera intuitiva.

```

<h1>Browser History Simulation</h1>
<div id="history"></div>
<button id="backButton" disabled>Back</button>
<button id="visitButton">Visit Page</button>
<script>
  const historyStack = [];
  const historyElement = document.getElementById('history');
  const backButton = document.getElementById('backButton');
  function visitPage() {
    const newPage = `Page ${historyStack.length + 1}`;
    historyStack.push(newPage);
    updateHistoryDisplay();
    backButton.disabled = false;
  }
  function goBack() {
    if (historyStack.length > 0) {
      historyStack.pop();
      updateHistoryDisplay();
    }
    if (historyStack.length === 0) {
      backButton.disabled = true;
    }
  }
  function updateHistoryDisplay() {
    historyElement.innerHTML = '';
    historyStack.forEach(page => {
      const div = document.createElement('div');
      div.textContent = page;
      historyElement.appendChild(div);
    });
  }

```

□ **Deshacer en aplicaciones de edición (Google Docs):** Las opciones de "deshacer" en editores de texto o gráficos emplean pilas para almacenar las acciones recientes. Cada cambio se añade a la pila y, al deshacer, se retiran las últimas acciones en orden inverso. Así, el usuario puede revertir los pasos en el orden en que fueron realizados, siguiendo el principio de LIFO.



Código: El código simula la funcionalidad de "deshacer" en aplicaciones de edición utilizando una estructura de datos tipo pila, donde cada acción realizada por el usuario se apila en un arreglo llamado **actionStack**. Al realizar una acción, esta se añade a la parte superior de la pila, y al presionar el botón "Deshacer", se retira la última acción realizada, siguiendo el principio LIFO (Last In, First Out), lo que permite al usuario revertir sus acciones en el orden inverso en que fueron ejecutadas. La interfaz se actualiza dinámicamente para reflejar el estado actual de las acciones disponibles.

```
<h1>Undo Functionality Simulation</h1>
<div id="actionList"></div>
<button id="performActionButton">Perform Action</button>
<button id="undoButton" disabled>Undo</button>
<script>
  const actionStack = [];
  const actionListElement = document.getElementById('actionList');
  const undoButton = document.getElementById('undoButton');
  function performAction() {
    const actionNumber = actionStack.length + 1;
    const action = `Action ${actionNumber}`;
    actionStack.push(action);
    updateActionDisplay();
    undoButton.disabled = false;
  }
  function undoAction() {
    if (actionStack.length > 0) {
      actionStack.pop();
      updateActionDisplay();
    }
    if (actionStack.length === 0) {
      undoButton.disabled = true;
    }
  }
</script>
```

□ **Redes sociales (Instagram):** Al visualizar historias o publicaciones en modo “explorar”, estas aplicaciones muestran las imágenes en una secuencia apilada, de modo que el usuario puede regresar a la publicación previa, siguiendo el orden de apilado inverso. Aquí, las publicaciones se organizan en una estructura tipo pila, en la que el usuario avanza o retrocede en función de la última interacción.



Código: El código simula un visualizador de publicaciones en redes sociales utilizando una estructura de datos tipo pila, donde las publicaciones se almacenan en un arreglo (**postStack**). Al hacer clic en el botón "Next", se avanza a la siguiente publicación, que se muestra en pantalla, y se habilita el botón "Previous" para regresar a la anterior. Este comportamiento sigue el principio LIFO (Last In, First Out), ya que el usuario puede navegar hacia adelante y hacia atrás en el orden en que interactuó con las publicaciones, permitiendo que la última publicación vista sea la primera en ser desecheda al retroceder.

```

<script>
  const postStack = [
    'https://via.placeholder.com/300/FF5733/FFFFFF?text=Post+1',
    'https://via.placeholder.com/300/33FF57/FFFFFF?text=Post+2',
    'https://via.placeholder.com/300/3357FF/FFFFFF?text=Post+3',
    'https://via.placeholder.com/300/FF33A1/FFFFFF?text=Post+4'
  ];
  const postContainer = document.getElementById('postContainer');
  const postImage = document.getElementById('postImage');
  const prevButton = document.getElementById('prevButton');
  const nextButton = document.getElementById('nextButton');

  let currentIndex = -1; // Start before the first post
  function showNextPost() {
    if (currentIndex < postStack.length - 1) {
      currentIndex++;
      postImage.src = postStack[currentIndex];
      prevButton.disabled = false;
    }
    if (currentIndex === postStack.length - 1) {
      nextButton.disabled = true;
    }
  }
  function showPreviousPost() {
    if (currentIndex > 0) {
      currentIndex--;
      postImage.src = postStack[currentIndex];
      nextButton.disabled = false;
    }
    if (currentIndex === 0) {
      prevButton.disabled = true;
    }
  }

```

Aplicaciones de Pilas (Ventajas Y Desventajas)

Las pilas son útiles en problemas donde es necesario llevar un registro temporal de información, como el control de llamadas de función en programación recursiva, o en navegadores web, donde permiten regresar a páginas previamente visitadas. Además, se utilizan en el manejo de expresiones matemáticas y en la conversión entre notaciones algebraicas.

Sin embargo, presentan desventajas, como el acceso restringido a elementos, pues no se pueden obtener datos intermedios sin eliminar los superiores. En pilas implementadas con arreglos, el espacio es limitado, ya que la pila tiene un tamaño fijo, lo cual puede resultar en errores si se intenta añadir más elementos de los permitidos.

Colas

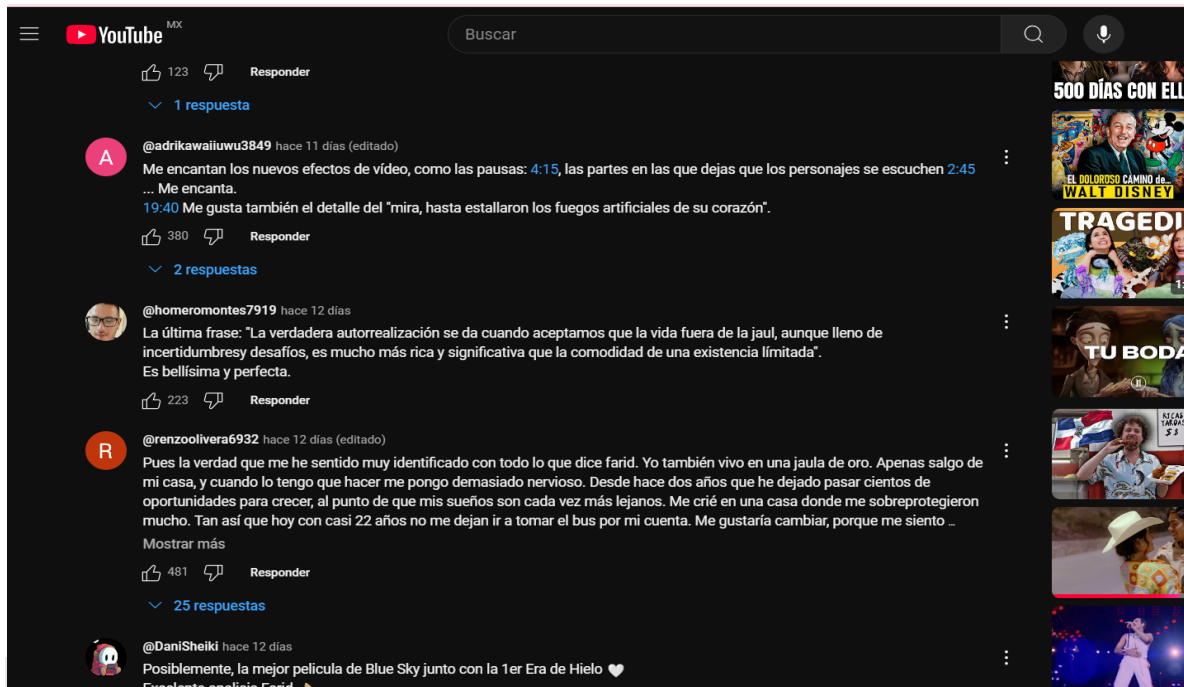
Una cola constituye una estructura lineal de datos en la que nuevos elementos se introducen por un extremo y los ya existentes se eliminan por el otro. Los componentes de la cola se eliminan en el mismo orden en el cual se insertaron. Es decir, el primer elemento que se introduce en la estructura será el que se eliminará en primer orden. Reciben también el nombre de estructuras FIFO (First-In, First-Out; el primero en entrar es el primero en salir).

Las colas se pueden representar mediante el uso de:

- Arreglos
- Listas

Se sabe que las operaciones básicas que pueden efectuarse son:

- Insertar un elemento en la cola
- Eliminar un elemento de la cola



Ejemplo 1: Comentarios en videos de YouTube.

Esta es la parte del código que se utiliza para los comentarios.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Comentario en YouTube</title>
  <style>
    .comment {
      background-color: #f4f4f4;
      padding: 10px;
      border-radius: 4px;
      margin-bottom: 10px;
    }
    .comment-header {
      font-weight: bold;
    }
    .comment-content {
      margin-top: 5px;
    }
  </style>
</head>
<body>
  <div class="comment">
    <div class="comment-header">Usuario123</div>
    <div class="comment-content">¡Me encantó este video! Gracias por coi
  </div>
</body>
</html>
```

Comentario en estructura HTML se define en esta estructura también JavaScript y CSS para construir y gestionar los comentarios.

Cola de carga de comentarios

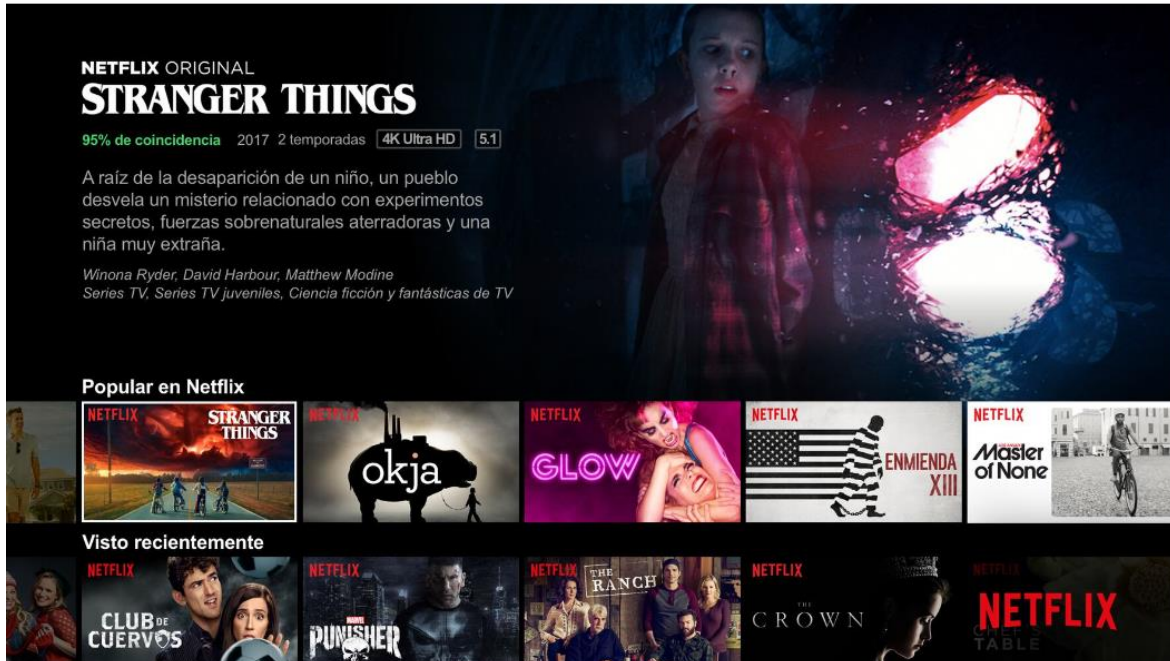
Al ver un video en YouTube, los comentarios se cargan y se colocan en una cola antes de mostrarse. Dando un flujo continuo de información.

Colas Circulares

Una cola circular constituye una estructura de datos lineal en la cual el siguiente elemento del último en realidad es el primero.

En pocas palabras están conectados entre sí.

Una de sus ventajas es la velocidad no requiere el deslizamiento de elementos, lo que mejora la eficiencia en términos de tiempo.



Ejemplo 2: Recomendaciones en Netflix

Ejemplo del código utilizando HTML Y JavaScript

Cada vez que damos a siguiente recomendación, se muestra una nueva recomendación y se reencola la anterior, creando un ciclo continuo.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Recomendaciones de Netflix</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #141414;
      color: #ffffff;
      text-align: center;
    }
    .recommendation {
      background-color: #333;
      padding: 20px;
      border-radius: 10px;
      margin: 20px auto;
      width: 50%;
    }
    .button {
      background-color: #e50914;
      color: white;
      border: none;
      padding: 10px 20px;
      cursor: pointer;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <h1>Recomendaciones para ti</h1>
  <div id="recommendations"></div>

```

```

class ColaCircular {
  constructor(capacidad) {
    this.cola = new Array(capacidad);
    this.frente = 0;
    this.final = 0;
    this.lleno = false;
  }

  encolar(elemento) {
    this.cola[this.final] = elemento;
    this.final = (this.final + 1) % this.cola.length;
    if (this.final === this.frente) {
      this.lleno = true;
    }
  }

  desencolar() {
    if (this.frente === this.final && !this.lleno) {
      throw new Error("La cola está vacía");
    }
    const elemento = this.cola[this.frente];
    this.frente = (this.frente + 1) % this.cola.length;
    this.lleno = false;
    return elemento;
  }
}

const recomendaciones = new ColaCircular(5);
["Serie A", "Película B", "Documental C", "Serie D", "Película E"].forEach((item) => {
  recomendaciones.encolar(item);
});

function mostrarSiguiente() {
  const item = recomendaciones.desencolar();
  recomendaciones.encolar(item);
  document.getElementById('recommendations').innerHTML = `<div class="recommendation">
    <h2>Recomendación Siguiente</h2>
    <div class="button">Ver recomendación</div>
  </div>`;
}

```

Mostrar la primera recomendación al cargar la página, el código nos da mostrar siguiente.

Ejemplo

Netflix usa colas circulares en el procesamiento de datos para sus algoritmos de recomendación. Analizando el comportamiento, para ofrecernos recomendaciones personalizadas.

Doble Cola

Una doble cola o bicola.

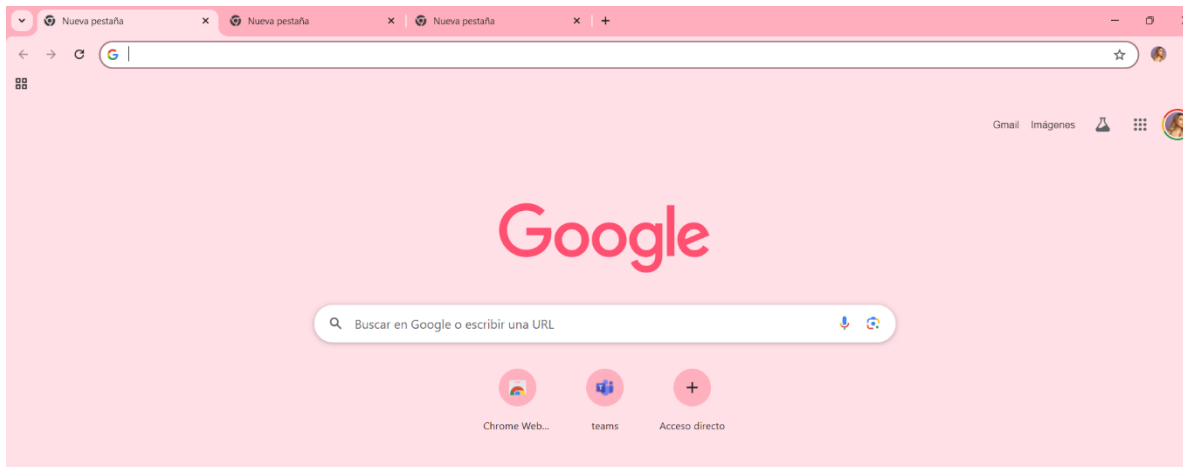
ola es una generalización de una estructura de datos tipo cola. En una doble cola, los elementos se pueden insertar o eliminar por cualquiera de los dos extremos. Se pueden insertar y eliminar valores tanto de frente como por el final de la cola.

Existen 2 variantes de las dobles colas:

- Doble cola con entrada restringida
- Doble cola con salida restringida

La primera de ellas permite que las eliminaciones se realicen por cualesquiera de los 2 extremos, mientras que las inserciones sólo por el final de la cola.

La segunda variante permite que las inserciones se realicen por cualesquiera de los 2 extremos, mientras que las eliminaciones sólo por el frente de la cola.



Ejemplo

En la gestión de pestañas: Las pestañas abiertas se pueden manejar utilizando una doble cola

Podemos observar que se pueden agregar nuevas pestañas al final de la cola y cerrar pestañas en cualquier punto.

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Administrar Pestañas</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
      color: #333;
    }
    .tabs {
      display: flex;
      justify-content: space-between;
      padding: 10px;
      border: 1px solid #ccc;
      background-color: #fff;
      border-radius: 5px;
      margin-bottom: 10px;
    }
    .button {
      background-color: #007bff;
      color: white;
      border: none;
      padding: 10px 20px;
      cursor: pointer;
      border-radius: 5px;
      margin: 5px;
    }
  </style>
</head>
<body>
  <h1>Administrar Pestañas Abiertas</h1>
  <div id="tabsContainer"></div>
  <button class="button" onclick="agregarAlFrente()">Agregar al Frente</button>
  <button class="button" onclick="agregarAlFinal()">Agregar al Final</button>
  <button class="button" onclick="cerrarDelFrente()">Cerrar del Frente</button>
  <button class="button" onclick="cerrarDelFinal()">Cerrar del Final</button>

  <script>
    class Deque {
      constructor() {
        this.items = [];
      }

      agregarAlFrente(elemento) {
        this.items.unshift(elemento);
      }

      agregarAlFinal(elemento) {
        this.items.push(elemento);
      }

      cerrarDelFrente() {
        return this.items.shift();
      }

      cerrarDelFinal() {
        return this.items.pop();
      }

      mostrarPestañas() {
        return this.items;
      }
    }

    const pestañas = new Deque();
  </script>
</body>
</html>

```

```

function agregarAlFrente() {
  const nombrePestaña = `Nueva Pestaña ${pestañas.mostrarPestañas.length}`;
  pestañas.agregarAlFrente(nombrePestaña);
  actualizarVista();
}

function agregarAlFinal() {
  const nombrePestaña = `Nueva Pestaña ${pestañas.mostrarPestañas.length}`;
  pestañas.agregarAlFinal(nombrePestaña);
  actualizarVista();
}

function cerrarDelFrente() {
  pestañas.cerrarDelFrente();
  actualizarVista();
}

function cerrarDelFinal() {
  pestañas.cerrarDelFinal();
  actualizarVista();
}

// Inicializar con algunas pestañas abiertas
["Google", "YouTube", "Gmail", "Noticias"].forEach(pestaña => pestañas.agregarAlFrente(pestaña));
actualizarVista();
</script>
</body>
</html>

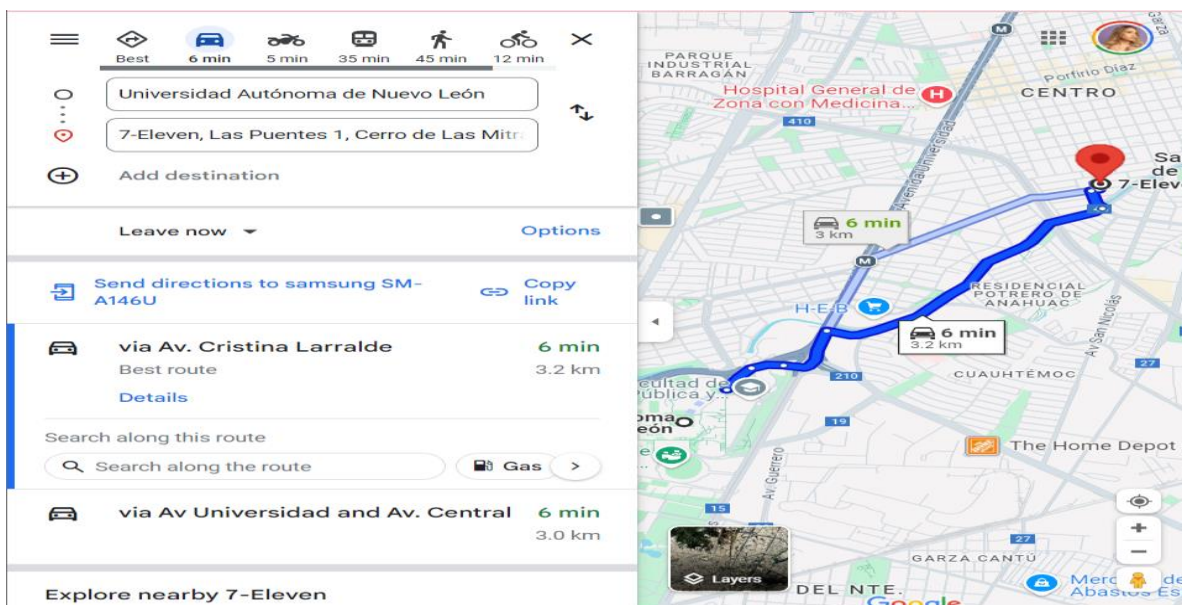
```

Ejemplo de código utilizado en Google Chrome para las pestañas abiertas

En esta última imagen muestra cómo se usa el código, la función, agrega al frente, agrega al final. Cerrar al frente, cerrar el final. Y al último actualizando vista.

Aplicaciones de Colas (Ventajas y Desventajas)

- Una aplicación común de las colas se presenta cuando se envía imprimir algún documento o programa en las colas de impresión.
- Cuando hay una sola impresión para atender a varios usuarios, suele suceder que algunos de ellos soliciten los servicios de impresión al mismo tiempo o mientras el dispositivo está ocupado, en estos casos se formó una cola con los trabajos que esperan para ser impresos, estos se procesan en el orden en el cual fueron introducidos en la cola.
- Otro caso de aplicaciones de colas en computación es en el que se presentan los sistemas de tiempo compartido. Varios usuarios compartir ciertos recursos como CPU y memoria de la computadora.
- Los recursos se asignan a los procesos que están en la cola de espera, suponiendo que todos tienen una misma prioridad en el orden en el cual fueron introducidos en la cola.
- Una aplicación muy común es en atención al cliente
- Gestionar la cola de clientes esperando ser atendidos (Call Center, bancos y hospitales)
- Al igual en la transmisión de audio y video
- Almacenar datos en buffer para asegurar una reproducción continua (servicios como Spotify y Netflix)
- Algoritmos de búsqueda en amplitud (encontrar la ruta más corta en grafos y redes)



1. Ejemplos de búsqueda

Ventajas

1. **Orden FIFO:** Las colas siguen el principio de "First In, First Out" (Primero en entrar, primero en salir), lo que es ideal para la gestión de tareas en secuencia, como la impresión de documentos o la programación de tareas en un sistema operativo.
2. **Simplicidad:** Son fáciles de implementar y entender, lo que las hace una buena opción para muchas aplicaciones básicas de programación.
3. **Eficiencia:** En operaciones de encolar y desencolar, las colas tienen una complejidad de tiempo , lo que significa que estas operaciones son muy rápidas.

4. **Versatilidad:** Pueden ser utilizadas en una amplia variedad de aplicaciones, desde la gestión de procesos en sistemas operativos hasta la simulación de colas en sistemas de atención al cliente.

Desventajas

- **Acceso Limitado:** Solo se puede acceder al primer y último elemento de la cola. Esto puede ser una limitación en casos donde se necesita acceder a elementos intermedios.
- **Espacio Adicional:** Si se utiliza una implementación basada en arreglos, puede requerir la relocalización de memoria si la cola crece más allá del tamaño del arreglo, lo que puede ser ineficiente.
- **Falta de Flexibilidad en Implementaciones Simples:** En implementaciones simples, las colas pueden no ser tan flexibles como otras estructuras de datos (por ejemplo, listas enlazadas) para ciertas operaciones.
- **Problemas de Overflow:** En colas estáticas, puede ocurrir un "overflow" si la cola alcanza su capacidad máxima y no se pueden añadir más elementos.

Listas

Las listas son una estructura de datos fundamental y muy utilizada en programación.

¿Qué es una lista? Una lista es una colección ordenada de elementos. Los elementos pueden ser de cualquier tipo: números, cadenas, otros objetos, e incluso otras listas. Las listas permiten almacenar múltiples valores en una sola variable, lo que facilita la organización y manipulación de datos.

Características principales de las listas:

- **Ordenadas:** Los elementos tienen una posición definida.
- **Mutables:** Los elementos se pueden cambiar, agregar o eliminar después de que la lista ha sido creada.
- **Permiten duplicados:** Una lista puede contener elementos repetidos.

Usos comunes de las listas:

- Almacenar secuencias de datos, como resultados de operaciones.
- Implementar pilas y colas.
- Gestionar colecciones de objetos.

Listas simplemente ligadas

Las listas simplemente ligadas son un tipo de estructura de datos en la que cada elemento, llamado nodo, contiene dos cosas: un valor (dato) y una referencia (enlace) al siguiente nodo en la secuencia. Aquí tienes una breve descripción:

Características de una lista simplemente ligada:

- **Nodo:** Cada nodo contiene un dato y una referencia al siguiente nodo.
- **Cabeza (Head):** El primer nodo de la lista.
- **Último nodo:** Su referencia es null o None, indicando el final de la lista.

Historial de navegación Chrome

En navegadores web, el historial de páginas visitadas se gestiona secuencialmente. Cada nueva página se convierte en la "cabeza" de la lista, y se puede recorrer en orden de visitas.

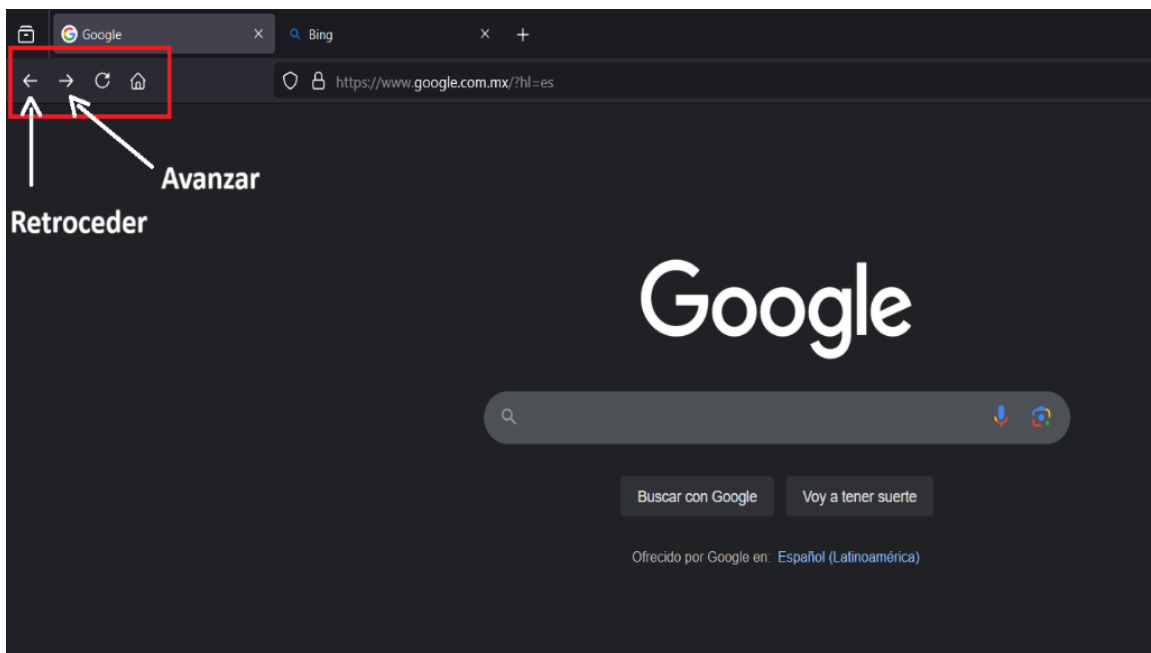
```
class PaginaWeb:
    def __init__(self, url):
        self.url = url
        self.siguiente = None

class HistorialNavegacion:
    def __init__(self):
        self.cabeza = None

    def visitar_pagina(self, url):
        nueva_pagina = PaginaWeb(url)
        nueva_pagina.siguiente = self.cabeza
        self.cabeza = nueva_pagina
        print(f"Página visitada: {url}")

    def mostrar_historial(self):
        actual = self.cabeza
        print("Historial de navegación:")
        while actual:
            print(f"- {actual.url}")
            actual = actual.siguiente
```

Este código crea un historial de navegación donde cada nueva página visitada se agrega al inicio de la lista (haciendo que la lista sea más eficiente para ver el historial reciente). HistorialNavegacion permite añadir nuevas páginas con visitar_pagina y recorrer el historial con mostrar_historial.



Lista de reproducción en YouTube

Cada vez que guardas con la etiqueta “Ver mas tarde” un video en YouTube, se puede añadir a una lista de reproducción de videos.

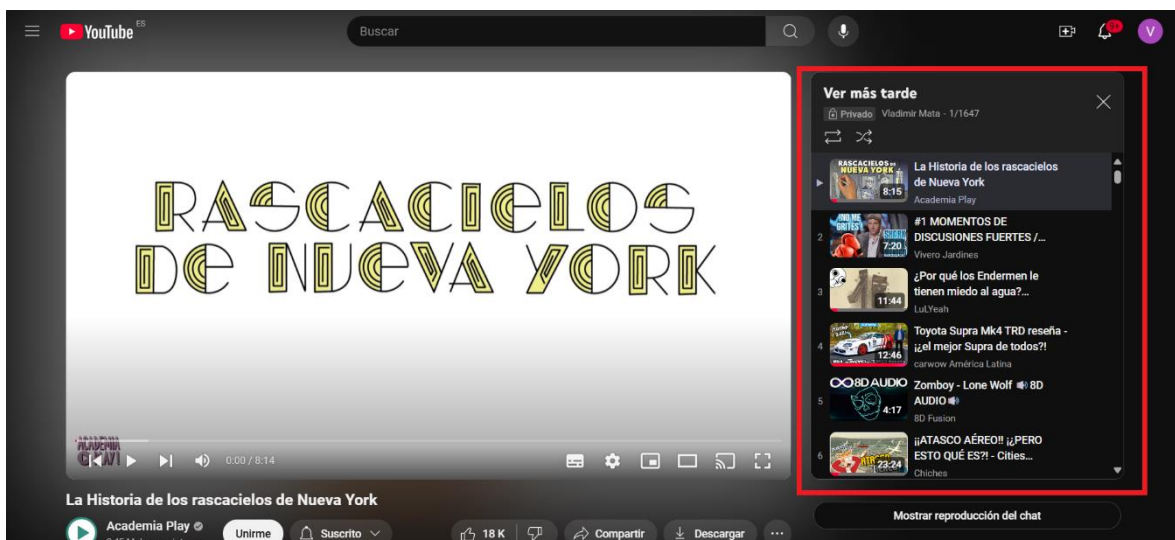
```
class Video:
    def __init__(self, titulo):
        self.titulo = titulo
        self.siguiente = None

class ListaReproduccion:
    def __init__(self):
        self.cabeza = None

    def agregar_video(self, titulo):
        nuevo_video = Video(titulo)
        if not self.cabeza:
            self.cabeza = nuevo_video
        else:
            actual = self.cabeza
            while actual.siguiente:
                actual = actual.siguiente
            actual.siguiente = nuevo_video
        print(f"Video agregado a la lista: {titulo}")

    def mostrar_lista(self):
        actual = self.cabeza
        print("Lista de reproducción:")
        while actual:
            print(f"📺 {actual.titulo}")
            actual = actual.siguiente
```

Esta lista de reproducción en ListaReproduccion se implementa como una lista simplemente ligada, donde cada Video tiene un título y una referencia al siguiente video. Se añade un video al final de la lista y se puede recorrer para mostrar todos los videos en orden de adición.



Lista de tareas en una app de productividad (Microsoft To-Do)

Aplicaciones de productividad usan listas ligadas para añadir, completar o eliminar tareas secuencialmente.

```
class Tarea:
    def __init__(self, descripcion):
        self.descripcion = descripcion
        self.siguiente = None

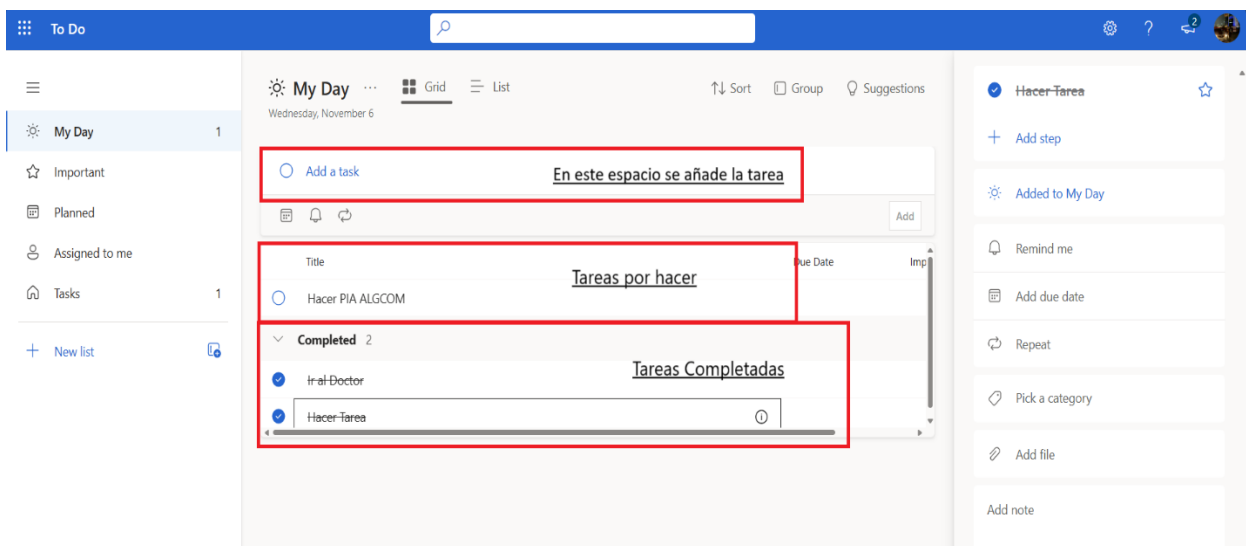
class ListaTareas:
    def __init__(self):
        self.cabeza = None

    def agregar_tarea(self, descripcion):
        nueva_tarea = Tarea(descripcion)
        nueva_tarea.siguiente = self.cabeza
        self.cabeza = nueva_tarea
        print(f"Tarea añadida: {descripcion}")

    def completar_tarea(self):
        if self.cabeza is None:
            print("No hay tareas para completar.")
            return
        print(f"Tarea completada: {self.cabeza.descripcion}")
        self.cabeza = self.cabeza.siguiente

    def mostrar_tareas(self):
        actual = self.cabeza
        print("Lista de tareas:")
        while actual:
            print(f"- {actual.descripcion}")
            actual = actual.siguiente
```

En esta implementación, ListaTareas permite agregar nuevas tareas al inicio, completarlas (eliminarlas de la lista) y mostrar todas las tareas pendientes. Cada tarea se conecta solo con la siguiente tarea, permitiendo recorrer la lista de tareas de manera eficiente.



Listas Circulares

Las listas circulares son una variación de las listas enlazadas, ya sean simple o doblemente enlazadas, donde el último nodo se enlaza de nuevo al primero, formando un ciclo continuo.

Características de las Listas Circulares:

- **Ciclo continuo:** El último nodo apunta de vuelta al primer nodo.
- **Navegación sin fin:** Puedes recorrer la lista indefinidamente, ya que no hay un final definido.

Aplicaciones de las Listas Circulares:

- **Juegos y simulaciones:** Por ejemplo, para gestionar turnos de jugadores.
- **Sistemas operativos:** Para implementar colas de procesos circulares.
- **Reproducción continua:** En listas de reproducción musicales que se repiten indefinidamente.

Ejemplo 1: Historias en Instagram

Instagram Stories permite ver las historias de los usuarios en un ciclo continuo. Una vez que llegas al final, regresas al inicio.

```
class Historia:
    def __init__(self, usuario):
        self.usuario = usuario
        self.siguiente = None

class HistoriasInstagram:
    def __init__(self):
        self.cabeza = None

    def agregar_historia(self, usuario):
        nueva_historia = Historia(usuario)
        if not self.cabeza:
            self.cabeza = nueva_historia
            self.cabeza.siguiente = self.cabeza
        else:
            actual = self.cabeza
            while actual.siguiente != self.cabeza:
                actual = actual.siguiente
            actual.siguiente = nueva_historia
            nueva_historia.siguiente = self.cabeza
        print(f"Historia añadida de: {usuario}")

    def ver_historias(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"Historia de {actual.usuario}")
            actual = actual.siguiente
```

Cada historia de usuario en `HistoriasInstagram` está enlazada de modo que al llegar al final regresa al primer usuario. Esto permite que el sistema muestre un ciclo infinito de historias, como en Instagram.



Ejemplo 2: Cola de impresión en una oficina (impresión continua)

En una oficina, una cola de impresión funciona en un ciclo donde los trabajos de impresión son atendidos continuamente, regresando al inicio si hay un flujo constante de documentos.

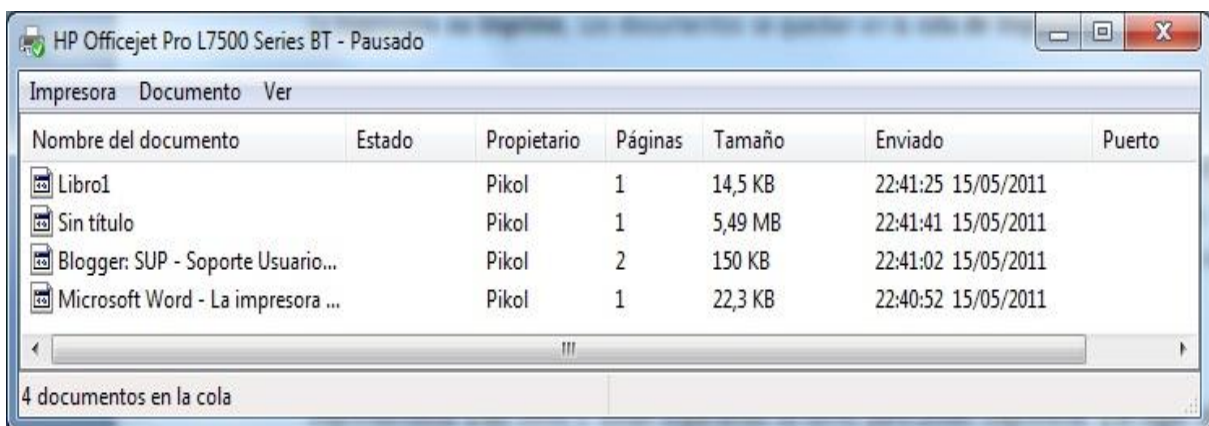
```
class TrabajoImpresion:
    def __init__(self, documento):
        self.documento = documento
        self.siguiente = None

class ColaImpresion:
    def __init__(self):
        self.cabeza = None

    def agregar_trabajo(self, documento):
        nuevo_trabajo = TrabajoImpresion(documento)
        if not self.cabeza:
            self.cabeza = nuevo_trabajo
            self.cabeza.siguiente = self.cabeza
        else:
            actual = self.cabeza
            while actual.siguiente != self.cabeza:
                actual = actual.siguiente
            actual.siguiente = nuevo_trabajo
            nuevo_trabajo.siguiente = self.cabeza
        print(f"Documento añadido a la cola: {documento}")

    def imprimir_documentos(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"Imprimiendo: {actual.documento}")
            actual = actual.siguiente
```

La ColaImpresion permite que los documentos se agreguen y se impriman continuamente. Al terminar de imprimir, la cola vuelve al primer documento, haciendo un ciclo que se repite si los documentos están pendientes.



Nombre del documento	Estado	Propietario	Páginas	Tamaño	Enviado	Puerto
Libro1		Pikol	1	14,5 KB	22:41:25 15/05/2011	
Sin título		Pikol	1	5,49 MB	22:41:41 15/05/2011	
Blogger: SUP - Soporte Usuario...		Pikol	2	150 KB	22:41:02 15/05/2011	
Microsoft Word - La impresora ...		Pikol	1	22,3 KB	22:40:52 15/05/2011	

4 documentos en la cola

Ejemplo 3: Lista de reproducción continua en Spotify

Spotify permite reproducir una lista en modo bucle infinito. Al llegar al último elemento de la lista, vuelve al primero.

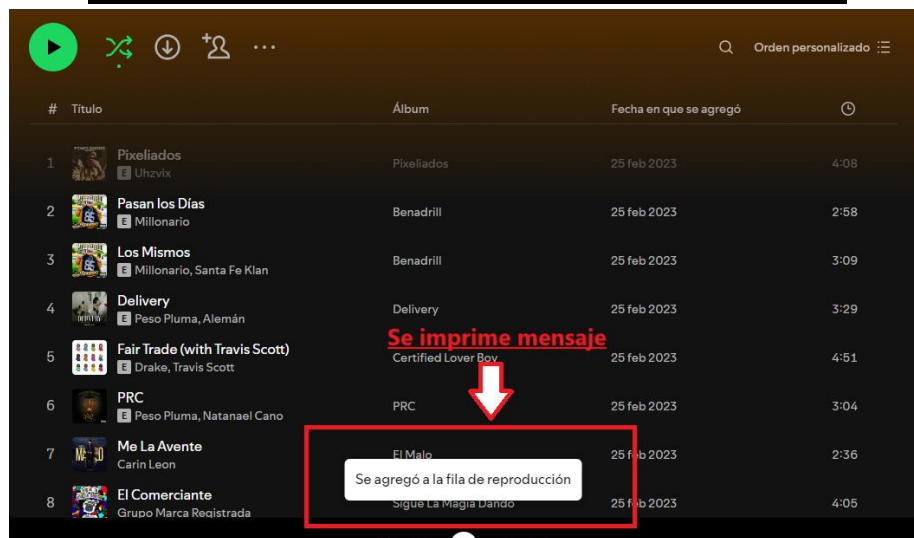
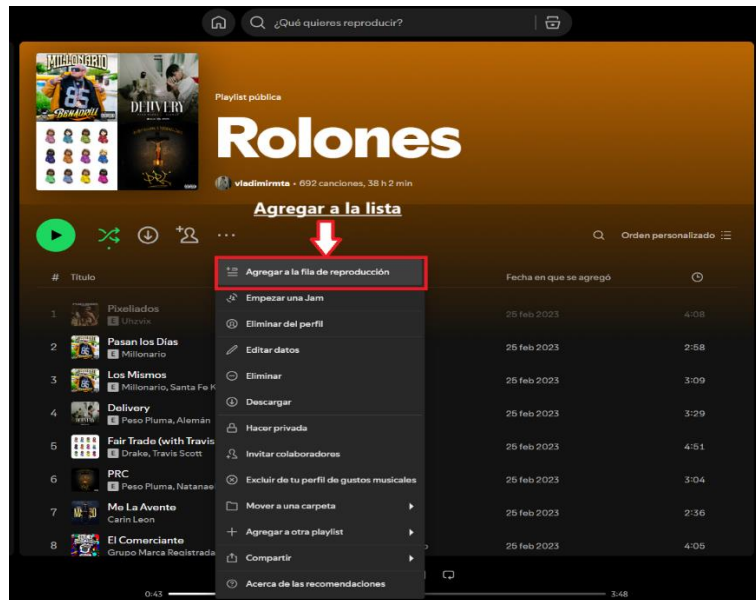
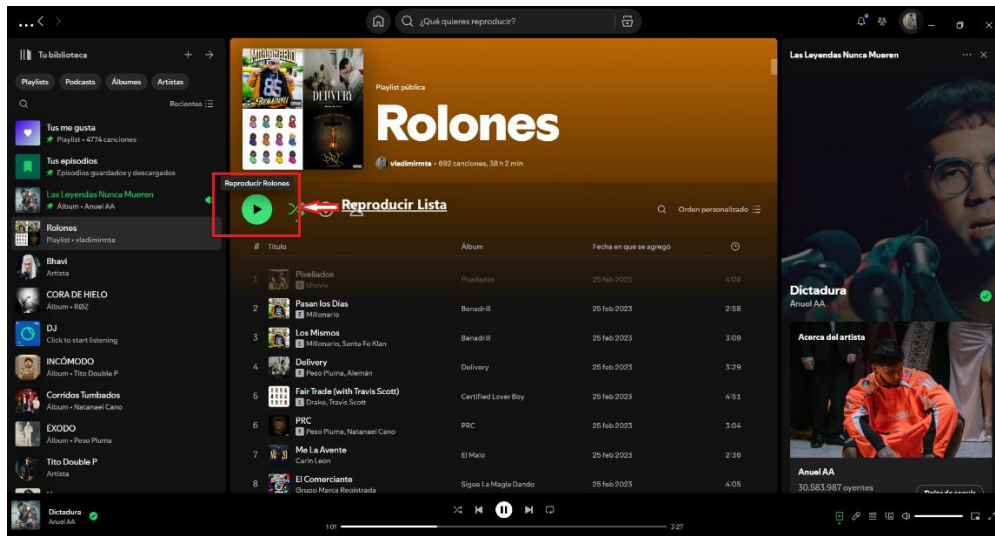
```
class Cancion:
    def __init__(self, titulo):
        self.titulo = titulo
        self.siguiente = None

class ListaSpotify:
    def __init__(self):
        self.cabeza = None

    def agregar_cancion(self, titulo):
        nueva_cancion = Cancion(titulo)
        if not self.cabeza:
            self.cabeza = nueva_cancion
            self.cabeza.siguiente = self.cabeza
        else:
            actual = self.cabeza
            while actual.siguiente != self.cabeza:
                actual = actual.siguiente
            actual.siguiente = nueva_cancion
            nueva_cancion.siguiente = self.cabeza
            print(f"Canción añadida: {titulo}")

    def reproducir_lista(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"Reproduciendo: {actual.titulo}")
            actual = actual.siguiente
```

En ListaSpotify, cada canción se enlaza de forma circular. Esto permite que al terminar la última canción, la lista vuelva al inicio y continúe reproduciendo en bucle, como lo haría Spotify en el modo de repetición.



Listas Doblemente Ligadas

Las listas doblemente ligadas son otra estructura de datos útil, donde cada nodo apunta no solo al siguiente nodo, sino también al nodo anterior. Esto permite una navegación bidireccional a través de la lista.

Características de las Listas Doblemente Ligadas:

- **Navegación bidireccional:** Cada nodo tiene una referencia al siguiente nodo y al nodo anterior.
- **Nodo doble:** Contiene datos y dos punteros, uno para el siguiente nodo y otro para el nodo anterior.

Aplicaciones de las Listas Doblemente Ligadas:

- **Navegación de historial:** En navegadores web, para ir hacia adelante y hacia atrás entre páginas.
- **Editor de texto:** Permite moverse fácilmente entre caracteres y palabras en ambos sentidos.
- **Gestionar listas de reproducción:** En aplicaciones multimedia, para moverse adelante y atrás en una lista de canciones o videos.

3. Listas doblemente ligadas

Las listas doblemente ligadas permiten que cada nodo apunte tanto al siguiente como al anterior nodo. Esto facilita el recorrido en ambas direcciones, haciéndolas ideales para aplicaciones donde el usuario navega hacia adelante y atrás.

Ejemplo 1: Sistema de comentarios en redes sociales (similar a Reddit o Twitter)

Cada comentario en un hilo de discusión puede estar vinculado con el anterior y el siguiente, permitiendo desplazarse entre ellos hacia adelante y atrás.

```
class Comentario:
    def __init__(self, contenido):
        self.contenido = contenido
        self.siguiente = None
        self.anterior = None

class HiloComentarios:
    def __init__(self):
        self.cabeza = None

    def agregar_comentario(self, contenido):
        nuevo_comentario = Comentario(contenido)
        if not self.cabeza:
            self.cabeza = nuevo_comentario
        else:
            actual = self.cabeza
            while actual.siguiente:
                actual = actual.siguiente
            actual.siguiente = nuevo_comentario
            nuevo_comentario.anterior = actual
            print(f"Comentario añadido: {contenido}")

    def mostrar_comentarios(self):
        actual = self.cabeza
        print("Comentarios:")
        while actual:
            print(f"- {actual.contenido}")
            actual = actual.siguiente
```

En HiloComentarios, cada Comentario apunta al siguiente y al anterior, lo que permite recorrer el hilo en ambas direcciones. Esto hace posible visualizar todos los comentarios en el orden en que fueron añadidos o retroceder a comentarios anteriores en el mismo hilo.

Home

Explore

Notifications

Messages

Grok

Bookmarks

Jobs

Communities

Premium

Verified Orgs

Profile

More

Post

Vlady Mata

@Vladimirmtaaa

70332/photo/1

Post

Reply

Prófugos del Ácido Fólico

@EsdeProfugos

¿Ángela Aguilar nombrada Mujer del Año? Hilo 🧵 de mujeres mexicanas galardonadas con el Premio L'Oréal-UNESCO Para las Mujeres en la Ciencia 2024, que realmente merecen ese título.

Translate post





6:44 PM · Nov 5, 2024 · 25.8K Views

27

64

408

32

V

Post your reply

Reply

Prófugos del Ácido Fólico

@EsdeProfugos · 10h

Tatiana Klimova Berestneva (UNAM) 🌱

Premio L'Oréal-UNESCO Para las Mujeres en la Ciencia 2024 en Ciencias Ambientales por su trabajo en el desarrollo de tecnologías de fotocatalisis para la purificación de gases contaminantes y la producción de biocombustibles. Su investigación

Show more



Search

Relevant people

Prófugos del Ácido Fó

@EsdeProfugos

Tragicomédias Sociales: Entre la Risa y la Indignación – No Culpes al Mensajero. Perfil sin afiliación política.

Follow

What's happening

Politics · Trending

#EleccionesEstadosUnidos

117K posts

Politics · Trending

Trump

3.59M posts

Politics · Trending

Michelle Obama

62.2K posts

Politics · Trending

Kamala

1.89M posts

Trending in Mexico

#MinistraBurra

4,495 posts

Show more

Terms of Service

Privacy Policy

Cookie Policy

Accessibility

Ads info

More

© 2024 X Corp.

Home

Explore

Notifications

Messages

Grok

Lists

Bookmarks

Jobs

Communities

Premium

Verified Orgs

Profile

More

Post

Vlady Mata

@Vladimirmtaaa

Post

Reply

biocombustibles. Su Investigación

Show more



1

5

62

1.3K

Prófugos del Ácido Fólico

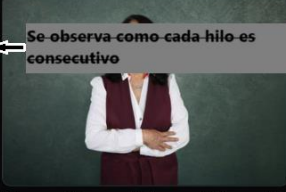
@EsdeProfugos · 10h

Refugio Rodríguez Velázquez (INAO) 🌱

Premio L'Oréal-UNESCO Para las Mujeres en la Ciencia 2024 en Biotecnología por su innovador trabajo en la bioremediación de suelos y tratamiento de agua utilizando microorganismos. Sus proyectos han demostrado cómo los organismos pueden

Show more

Se observa como cada hilo es consecutivo



Prófugos del Ácido Fólico

@EsdeProfugos · 10h

María Cristina Rodríguez Padilla (IANIGLA) 🌱

Ganadora del Premio L'Oréal-UNESCO Para las Mujeres en la Ciencia 2024 en Ciencias de la Salud por su investigación sobre inmunoterapia y componentes antitumorales. Su trabajo busca crear terapias que fortalezcan el sistema

Show more



Search

Relevant people

Prófugos del Ácido Fó

@EsdeProfugos

Tragicomédias Sociales: Entre la Risa y la Indignación – No Culpes al Mensajero. Perfil sin afiliación política.

Follow

What's happening

Politics · Trending

#EleccionesEstadosUnidos

117K posts

Politics · Trending

Trump

3.59M posts

Politics · Trending

Kamala

1.89M posts

Trending in Mexico

#MinistraBurra

4,495 posts

Politics · Trending

Michelle Obama

62.2K posts

Show more

Terms of Service

Privacy Policy

Cookie Policy

Accessibility

Ads info

More

© 2024 X Corp.

Ejemplo 2: Historial de edición en documentos (similar a Google Docs)

Google Docs permite hacer deshacer y rehacer cambios, recorriendo versiones en ambos sentidos. Cada edición se enlaza a la anterior y la siguiente.

```
class Edicion:
    def __init__(self, contenido):
        self.contenido = contenido
        self.siguiente = None
        self.anterior = None

class HistorialEdiciones:
    def __init__(self):
        self.cabeza = None
        self.actual = None

    def agregar_edicion(self, contenido):
        nueva_edicion = Edicion(contenido)
        if not self.cabeza:
            self.cabeza = nueva_edicion
            self.actual = nueva_edicion
        else:
            self.actual.siguiente = nueva_edicion
            nueva_edicion.anterior = self.actual
            self.actual = nueva_edicion
        print(f"Edición guardada: {contenido}")

    def deshacer(self):
        if self.actual and self.actual.anterior:
            self.actual = self.actual.anterior
            print(f"Deshacer: {self.actual.contenido}")

    def rehacer(self):
        if self.actual and self.actual.siguiente:
            self.actual = self.actual.siguiente
            print(f"Rehacer: {self.actual.contenido}")
```

El historial de edición permite navegar hacia adelante y hacia atrás entre ediciones. deshacer retrocede a una edición anterior y rehacer avanza a una edición posterior, facilitando la navegación en ambas direcciones.

Hoy, 5:43

100%

Total: 14 cambios

←

Pestañas del documento

Pestaña 1

EXPERIENCIA

EmpresaMuncher House, Ciudad: puesto San Nicolas de los Garza, Cocinero A

MES DEL 200X-PRESENTEMAYO 2023-PRESENTE

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Cocinero A en Muncher House

Empresa, Ciudad: puestoBurger House, Apodaca, Cocinero

MES DEL 200X-MES DEL 200X

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Empresa, Ciudad: puesto

MES DEL 200X-MES DEL 200X

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

FORMACIÓN

Nombre del centro, Ciudad: título

MES DEL 200X-MES DEL 200X

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

APTITUDES

Escribe aquí tu texto.

Escribe aquí tu texto.

Escribe aquí tu texto Escribe aquí tu texto.

Escribe aquí tu texto Escribe aquí tu texto.

Escribe aquí tu texto Escribe aquí tu texto.

LOGROS

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Historial de versiones

Todas las versiones

Hoy

6 de noviembre, 5:43

Versión actual

Vladimir Mata

6 de noviembre, 5:39

Vladimir Mata

☒Mostrar cambios

Hoy, 5:39

Restaurar esta versión

100%

Total: 1 cambio

←

Pestañas del documento

Pestaña 1

Tu nombre

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

EXPERIENCIA

Empresa, Ciudad: puesto

MES DEL 200X-PRESENTE

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Empresa, Ciudad: puesto

MES DEL 200X-MES DEL 200X

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Empresa, Ciudad: puesto

MES DEL 200X-MES DEL 200X

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

APTITUDES

Tu calle, 123

12345 Tu ciudad (Provincia)

(34) 000 000 000

no_reply@example.com

Escribe aquí tu texto.

Escribe aquí tu texto.

Escribe aquí tu texto Escribe aquí tu texto

Escribe aquí tu texto Escribe aquí tu texto.

LOGROS

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto Escribe aquí tu texto

Historial de versiones

Todas las versiones

Hoy

6 de noviembre, 5:43

Versión actual

Vladimir Mata

6 de noviembre, 5:39

Vladimir

Restaurar esta versión

Poner nombre a esta versión

Hacer una copia

Ejemplo 3: Revisión de imágenes en una galería (similar a Google Photos)

Las galerías de fotos permiten desplazarse entre imágenes hacia adelante y atrás, lo cual es fácil de gestionar con listas doblemente ligadas.

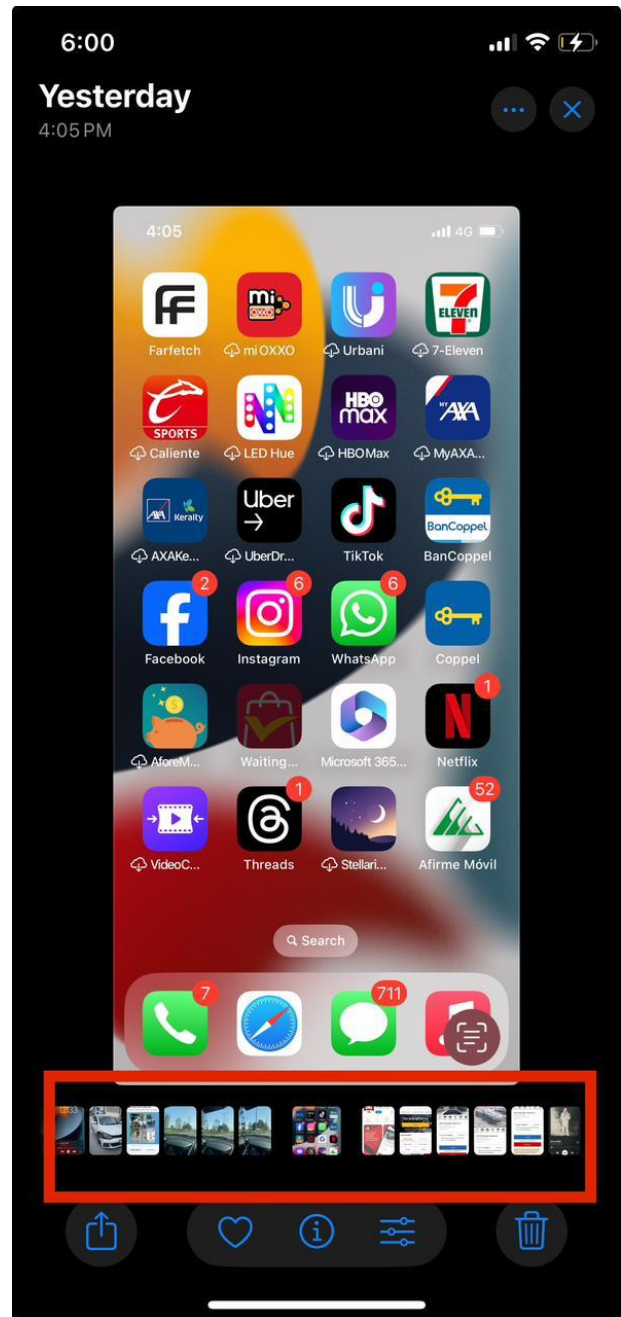
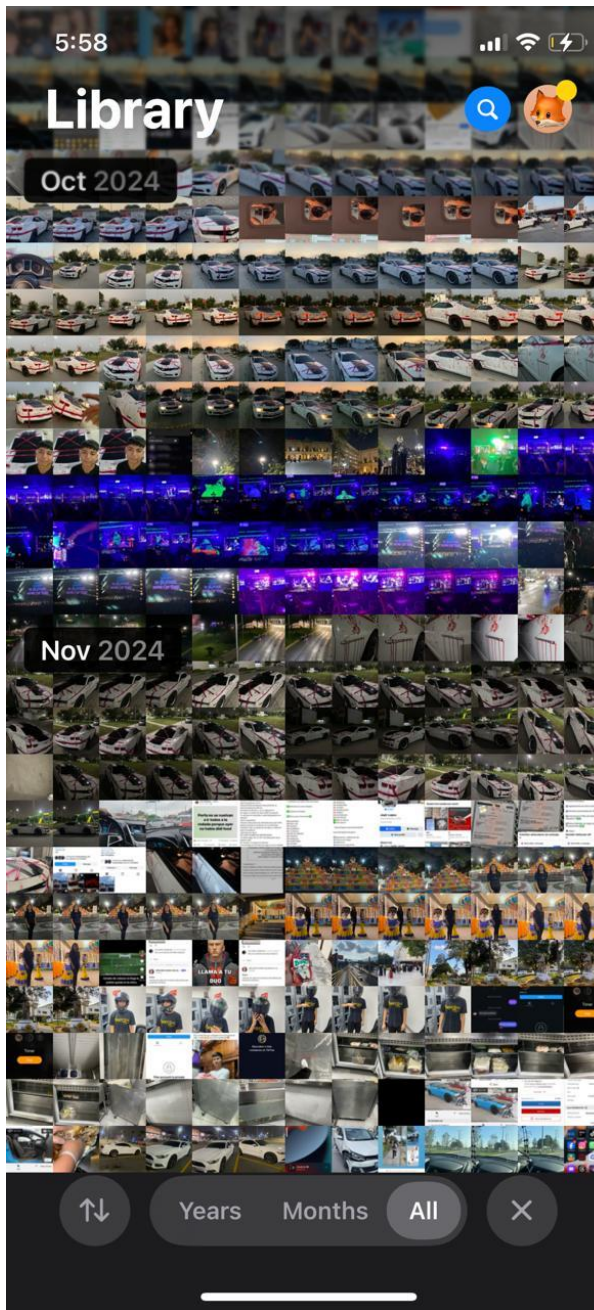
```
class Foto:
    def __init__(self, nombre):
        self.nombre = nombre
        self.siguiente = None
        self.anterior = None

class GaleriaFotos:
    def __init__(self):
        self.cabeza = None

    def agregar_foto(self, nombre):
        nueva_foto = Foto(nombre)
        if not self.cabeza:
            self.cabeza = nueva_foto
        else:
            actual = self.cabeza
            while actual.siguiente:
                actual = actual.siguiente
            actual.siguiente = nueva_foto
            nueva_foto.anterior = actual
        print(f"Foto añadida: {nombre}")

    def mostrar_fotos(self):
        actual = self.cabeza
        print("Fotos en galería:")
        while actual:
            print(f"🖼️ {actual.nombre}")
            actual = actual.siguiente
```

Descripción: La GaleriaFotos permite moverse entre las fotos en ambas direcciones, permitiendo que el usuario revise imágenes previas o avance a las siguientes en el álbum. Esto permite gestionar eficientemente el desplazamiento en ambas direcciones en la galería.



Listas Doblemente Ligadas Circulares

Las listas doblemente ligadas circulares son una extensión de las listas doblemente ligadas, en las cuales el último nodo apunta al primer nodo y el primer nodo apunta al último, creando un ciclo bidireccional. Esta estructura permite recorrer la lista tanto hacia adelante como hacia atrás de manera continua.

Características de las Listas Doblemente Ligadas Circulares:

- **Navegación bidireccional continua:** Cada nodo tiene una referencia al siguiente nodo y al nodo anterior, y la lista se conecta en un ciclo continuo.
- **Sin nodo nulo de terminación:** Al ser circular, no hay un nodo con puntero nulo, lo que permite recorrer la lista indefinidamente.

Aplicaciones de las Listas Doblemente Ligadas Circulares:

- **Sistemas operativos:** Gestión de colas de procesos circulares.
- **Reproducción multimedia:** Implementación de listas de reproducción que se repiten infinitamente.
- **Juegos:** Gestión de turnos de jugadores en juegos de mesa multijugador.
- **Gestión de procesos en sistemas operativos:** Las listas doblemente ligadas circulares pueden gestionar colas de procesos, donde cada proceso recibe una cantidad equitativa de tiempo de CPU y después vuelve al inicio de la cola, permitiendo una planificación continua y justa.

Las listas doblemente ligadas circulares permiten navegar entre nodos en ambas direcciones, con el último nodo apuntando al primero y viceversa, creando un ciclo cerrado.

Ejemplo 1: Cambio de perfiles en LinkedIn

En LinkedIn, al ver perfiles sugeridos, puedes avanzar o retroceder entre perfiles sugeridos en un ciclo continuo.

```
class Perfil:
    def __init__(self, nombre):
        self.nombre = nombre
        self.siguiente = None
        self.anterior = None

class SugerenciasPerfil:
    def __init__(self):
        self.cabeza = None

    def agregar_perfil(self, nombre):
        nuevo_perfil = Perfil(nombre)
        if not self.cabeza:
            self.cabeza = nuevo_perfil
            self.cabeza.siguiente = self.cabeza
            self.cabeza.anterior = self.cabeza
        else:
            ultimo = self.cabeza.anterior
            ultimo.siguiente = nuevo_perfil
            nuevo_perfil.anterior = ultimo
            nuevo_perfil.siguiente = self.cabeza
            self.cabeza.anterior = nuevo_perfil
        print(f"Perfil sugerido: {nombre}")

    def ver_sugerencias(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"Perfil: {actual.nombre}")
            actual = actual.siguiente
```

SugerenciasPerfil permite agregar perfiles sugeridos en un ciclo continuo, donde puedes pasar del último perfil al primero, o navegar de regreso al anterior. Esto permite que los perfiles se vean repetidamente sin tener que reiniciar la lista.

Inicio

Mi red

Empleos

Mensajes

Notificaciones

Yo

Para negocios

Probar Premium por 0 MXN

Más sugerencias para ti

Alfredo Hernández ..

Comprador en Plato Express

Muncher House

Conectar

Enrique De Jesús Bá...

Practicante universitaria en Hard Rock Hotel ...

Muncher House

Conectar

Lizbeth Pérez

Cocinero b en Eurostars Hotels

Based on your profile

Conectar

Cynthia Luna

Lic. fotografía

Muncher House

Conectar

Alfredo V.

Admin. de plataforma de aprendizaje en línea y c...

Muncher House

Conectar

Alvaro Blancarte Yofú

CMO | Méndez Jr.

Muncher House

Conectar

Luis Fernando Mejia..

Cocinero B en Hilton Mexico City Reforma

Based on your profile

Conectar

Eduardo Lopez

Cocinero B en Grupo RosaNegra

Based on your profile

Conectar

aro-blancarte-yofu-54143848/ Mensajes

Inicio

Mi red

Empleos

Mensajes

Notificaciones

Yo

Para negocios

Probar Premium por 0 MXN

Zuly M.

Muncher House

Conectar

Javier SanJuan

Beccario de Producción en Carnes Gust

Muncher House

Conectar

Sergio Sánchez

Cocinero B en Fairmont Mayakoba

Based on your profile

Conectar

Arturo Hiram Rodri...

Asesor de Ventas Laben Food Service

Muncher House

Conectar

Jairo Tadeo Ferman ..

Jefe de cocina en Muncher House

Muncher House

Conectar

Jose Patricio Romer...

Cocinero B

Based on your profile

Conectar

Ramiro Alejandro R...

Representante de Servicio y Asistencia en Alliance ..

Muncher House

Conectar

Andrea Rios Rodriguez

Innovación y Desarrollo

Muncher House

Conectar

Erika Garza Garza ...

Estudiante en Facultad de ciencias políticas y rela...

Muncher House

Conectar

Marcos Eduardo ...

Cocinero B

Muncher House

Conectar

Josseth García

Cocinero b en Orion pizzeria

Muncher House

Conectar

Rodrigo Beltrán C.

Fisicomista p en el estudio y

Mensajes

Inicio

Mi red

Empleos

Mensajes

Notificaciones

Yo

Para negocios

Probar Premium por 0 MXN

Connect with your buyers

virtually with Sales Navigator and tap into LinkedIn's network. Learn More

Acerca de

Accesibilidad

Centro de ayuda

Privacidad y condiciones

Opciones de publicidad

Publicidad

Servicios empresariales

Descargar la aplicación de LinkedIn

Más

LinkedIn LinkedIn Corporation © 2024

Personas con cargos similares al tuyo que podrías conocer

Mostrar todo

Sergio Sánchez

Cocinero B en Fairmont Mayakoba

Based on your profile

Conectar

Luis Fernando Mejia..

Cocinero B en Hilton Mexico City Reforma

Based on your profile

Conectar

Jose Patricio Romer...

Cocinero B

Based on your profile

Conectar

Josseth García

Cocinero b en Orion pizzeria

Based on your profile

Conectar

Eber Jocsan Morales..

Cocinero b en Plato Express

Based on your profile

Conectar

jazmin flores

Cocinero B at The Ritz-Carlton, Mexico City

Based on your profile

Conectar

Jairo Tadeo Ferman ..

Jefe de cocina en Muncher House

Muncher House

Conectar

Lizbeth Pérez

Cocinero b en Eurostars Hotels

Based on your profile

Conectar

Ejemplo 2: Rueda de noticias en Microsoft Start (Recopilación de Noticias de diferentes Sitios Webs antes “MSN”)

En apps de noticias, las noticias destacadas suelen mostrarse en una rueda o "slider" continuo.

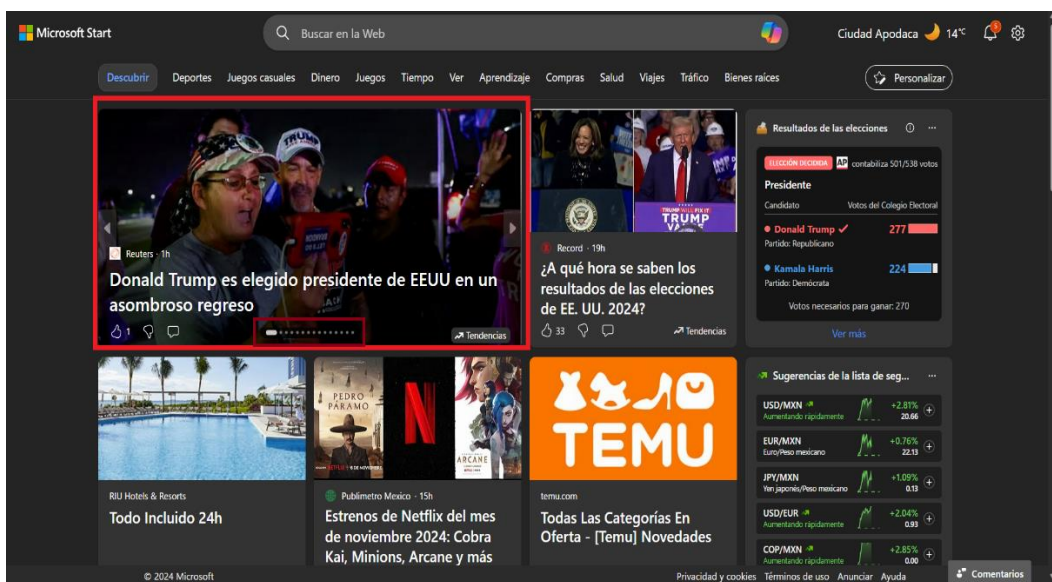
```
class Noticia:
    def __init__(self, titulo):
        self.titulo = titulo
        self.siguiente = None
        self.anterior = None

class RuedaNoticias:
    def __init__(self):
        self.cabeza = None

    def agregar_noticia(self, titulo):
        nueva_noticia = Noticia(titulo)
        if not self.cabeza:
            self.cabeza = nueva_noticia
            self.cabeza.siguiente = self.cabeza
            self.cabeza.anterior = self.cabeza
        else:
            ultimo = self.cabeza.anterior
            ultimo.siguiente = nueva_noticia
            nueva_noticia.anterior = ultimo
            nueva_noticia.siguiente = self.cabeza
            self.cabeza.anterior = nueva_noticia
        print(f"Noticia añadida: {titulo}")

    def mostrar_noticias(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"■ {actual.titulo}")
            actual = actual.siguiente
```

RuedaNoticias permite desplazarse por las noticias y, al final, volver al inicio. Esta estructura de lista doblemente ligada circular permite recorrer las noticias en ambas direcciones en un ciclo infinito, como ocurre en muchos carruseles de noticias.



Ejemplo 3: Lista de reproducción en bucle continuo con control hacia adelante y atrás en Spotify

En Spotify, puedes navegar hacia adelante y atrás en la lista de canciones en un bucle infinito, permitiendo avanzar y retroceder en cualquier punto de la lista.

```
class Cancion:
    def __init__(self, titulo):
        self.titulo = titulo
        self.siguiente = None
        self.anterior = None

class ListaReproduccion:
    def __init__(self):
        self.cabeza = None

    def agregar_cancion(self, titulo):
        nueva_cancion = Cancion(titulo)
        if not self.cabeza:
            self.cabeza = nueva_cancion
            self.cabeza.siguiente = self.cabeza
            self.cabeza.anterior = self.cabeza
        else:
            ultimo = self.cabeza.anterior
            ultimo.siguiente = nueva_cancion
            nueva_cancion.anterior = ultimo
            nueva_cancion.siguiente = self.cabeza
            self.cabeza.anterior = nueva_cancion
        print(f"Canción añadida: {titulo}")

    def reproducir_lista(self, ciclos=1):
        actual = self.cabeza
        for _ in range(ciclos):
            print(f"🔊 Reproduciendo: {actual.titulo}")
            actual = actual.siguiente
```

ListaReproduccion permite que las canciones se enlacen en un ciclo doblemente ligado, donde puedes avanzar o retroceder continuamente por la lista. Esto permite que las canciones se reproduzcan en un bucle sin fin, replicando la funcionalidad de listas de reproducción en modo "repetir" en Spotify

Tu biblioteca

PlaylistsPodcastsÁlbumes

Tus me gusta

Tus episodios

Las Leyendas Nunca Mueren

Rolones

Bhavi

CORA DE HIELO

DJ

INCÓMODO

Dime Tú

Playlist

Tus me gusta

vladimirmta • 4774 canciones

1

PERFUME

Bhavi, Nicki Nicole

PERFUME

hace 15 horas

2:37

2

MI CIUDAD

YSY A, Bhavi, Tiago P...

MI CIUDAD

hace 15 horas

3:41

3

High Fashion Drugs

Nessly, Bhavi

High Fashion Drugs

hace 15 horas

3:28

4

DIOS Y LA MUERTE

Bhavi, Santa Fe Klan

DIOS Y LA MUERTE

hace 15 horas

2:52

5

Teca (Remix) [feat. Ne...

Asan, Bhavi, YSY A, N...

Teca (Remix) [feat. Ne...

hace 15 horas

4:24

Capacidad de poder Avanzar o retroceder en la lista

1:17

Repetición en bucle infinito

2:58

Fila de reproducción

Escuchado recientemente

Estás reproduciendo

Dime Tú

Anuel AA, Ozuna

Siguiente en la fila de reproducción

Te Esperaré

Arcángel

Dos Mil 16

Bad Bunny

Sentosa

Peso Pluma, Tornillo, Polo Gonzalez, Garena...

Inhale Exhale

Skrillex, Aluna, Kito

Friends - Remix

Eladio Carrion, Lil TJay, Luar La L

Coco Chanel

Eladio Carrion, Bad Bunny

El Doctor

Aplicaciones de Listas (Ventajas y Desventajas)

Aplicaciones:

- Gestión de memoria: Permiten una gestión eficiente de la memoria cuando los tamaños de los elementos son variables.
- Historial de navegación: Utilizadas en navegadores web para avanzar y retroceder entre páginas.
- Gestión de tareas: En sistemas operativos, para gestionar colas de procesos.
- Redes sociales: Manejo de listas de amigos, seguidores y publicaciones.
- Reproducción multimedia: Gestión de listas de reproducción en música y video.

Ventajas:

- Flexibilidad: Pueden crecer y reducirse dinámicamente.
- Fácil inserción/eliminación: Las operaciones son eficientes, especialmente en listas enlazadas.
- Uso de memoria: No requieren memoria contigua.
- Modelo simple: Fáciles de entender e implementar.

Desventajas:

- Acceso secuencial: Puede ser lento en comparación con los arreglos.
- Mayor uso de memoria: Cada nodo necesita almacenamiento adicional para los punteros.
- Complejidad: Las listas doblemente enlazadas y circulares son más complejas de implementar.
- Desempeño: Menos eficientes en operaciones que requieren acceso frecuente a elementos intermedios.

Árboles

Los árboles son una estructura de datos muy útil para organizar datos que tienen una jerarquía natural, como los sistemas de archivos o las jerarquías de empresas.

Estos son una estructura de datos que imita la forma de un árbol y se representa con nodos conectados por aristas, las cuales están unidas a un nodo Raíz (parentNode) conectandose con los nodos (childNode), siendo la primera línea de nodos hijos conocida como las ramas y la que le prosigue como hojas

Características

- Estructura Jerárquica y no lineal
- Cada Nodo puede tener 0 o más hijos conectados a él
- Cualquier Árbol no vacío tiene un nodo raíz

Aplicaciones de Árboles

Los árboles muy utilizados debido a que son un método eficiente para búsquedas grandes y complejas por ejemplo en algunas áreas como lo son inteligencia artificial, algoritmos de cifrado y también son utilizados por los sistemas operativos para almacenar archivos.

Ejemplo de construcción de arbol binario en javascript

```
* Metodo para construir un arbol para una expresion aritmetica dada.
* @param expresion -- Cadena con la expresion aritmetica
* @return NodoArbol -- nodo raiz del arbol creado
*/
public NodoArbol construirArbol(String expresion) {
    StringTokenizer tokenizer;
    String token;
    NodoArbol raiz = null;

    tokenizer = new StringTokenizer(expresion, "()+*/<=> ", true);
    while (tokenizer.hasMoreTokens()) {
        token = tokenizer.nextToken();
        if (token.indexOf(" ") >= 0) {
            // Es un espacio en blanco, se ignora
        } else if (token.indexOf("(") < 0) {
            // Es operando y lo guarda como nodo del arbol
            pOperandos.push(new NodoArbol(token));
        } else if (token.equals("(")) { // Saca elementos hasta encontrar (
            while (!pOperadores.estaVacia() && !pOperadores.top().equals("(")) {
                guardarSubArbol();
            }
            pOperadores.pop(); // Saca el parentesis izquierdo
        } else {
            if (!token.equals("(") && !pOperadores.estaVacia()) {
                // operador diferente de cualquier parentesis
                String op = (String) pOperadores.top();
                while (!op.equals("(") && !pOperadores.estaVacia()
                    && operadores.indexOf(op) >= operadores.indexOf(token)) {
                    guardarSubArbol();
                    if (!pOperadores.estaVacia())
                        op = (String) pOperadores.top();
                }
            }
            pOperadores.push(token); // Guarda el operador
        }
    }
    // Sacar todo lo que queda
    raiz = (NodoArbol) pOperandos.top();
    while (!pOperadores.estaVacia()) {
        guardarSubArbol();
    }
}
```

Activar Windows
Ve a Configuración para activar Windows.

Árboles Binarios

En los árboles binarios los nodos no pueden tener más de 2 nodos unidos a ellos, sea así, un nodo puede tener de 0 a 2 nodos hijos conectados a el

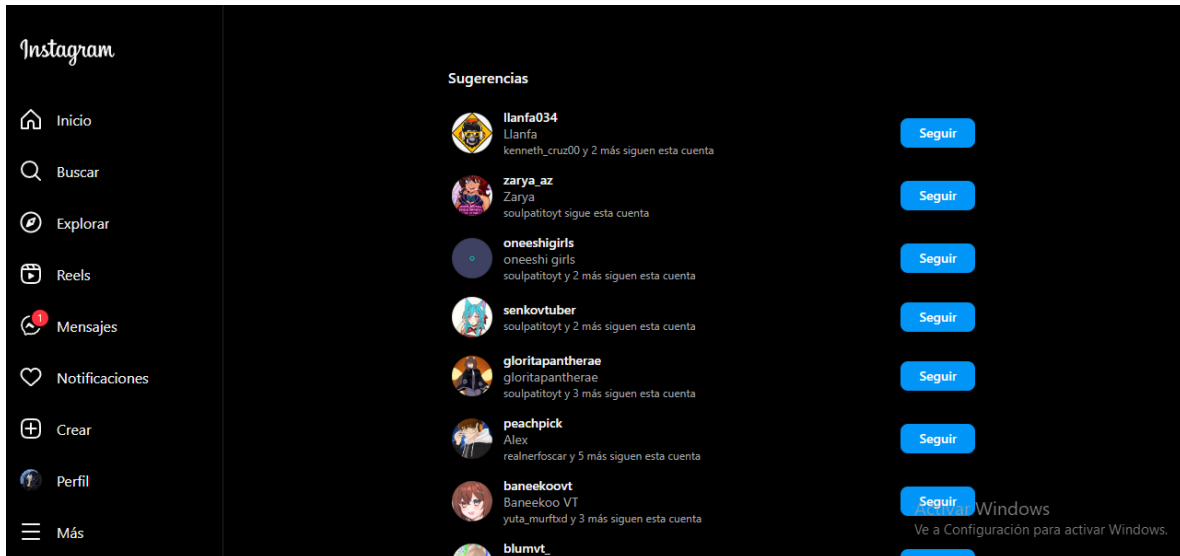
Ventajas

- Son simples de entender y de interpretar
- Si el árbol no es excesivamente grande, puede visualizarse

Desventajas

- Los árboles de decisión tienden al sobreentrenamiento, especialmente cuando el número de características predictivas es alto
- Son inestables: cualquier pequeño cambio en los datos de entrada puede suponer un árbol de decisión completamente diferente

Ejemplo. Sugerencias de Seguir en Instagram



Código: La estructura de árbol para sugerencias de seguir en Instagram consiste en un nodo raíz que representa al usuario actual, conectado a un nivel 1 de usuarios relacionados (amigos en común, intereses similares, ubicación geográfica), que a su vez se conectan a un nivel 2 de usuarios sugeridos (contenido similar, etiquetas similares, intereses similares), y finalmente a un nivel 3 de perfiles de usuarios específicos con información detallada (nombre, biografía, foto de perfil). Cada nodo tiene atributos como puntuación de relevancia, número de seguidores y publicaciones. El algoritmo de sugerencias explora los nodos, calcula la puntuación de relevancia y ordena los usuarios sugeridos para mostrar los más relevantes, permitiendo explorar relaciones entre usuarios y contenido y proporcionar sugerencias personalizadas.

Árboles Balanceados

Un árbol balanceado es cuando todos sus niveles, excepto el ultimo, están al máximo de nodos

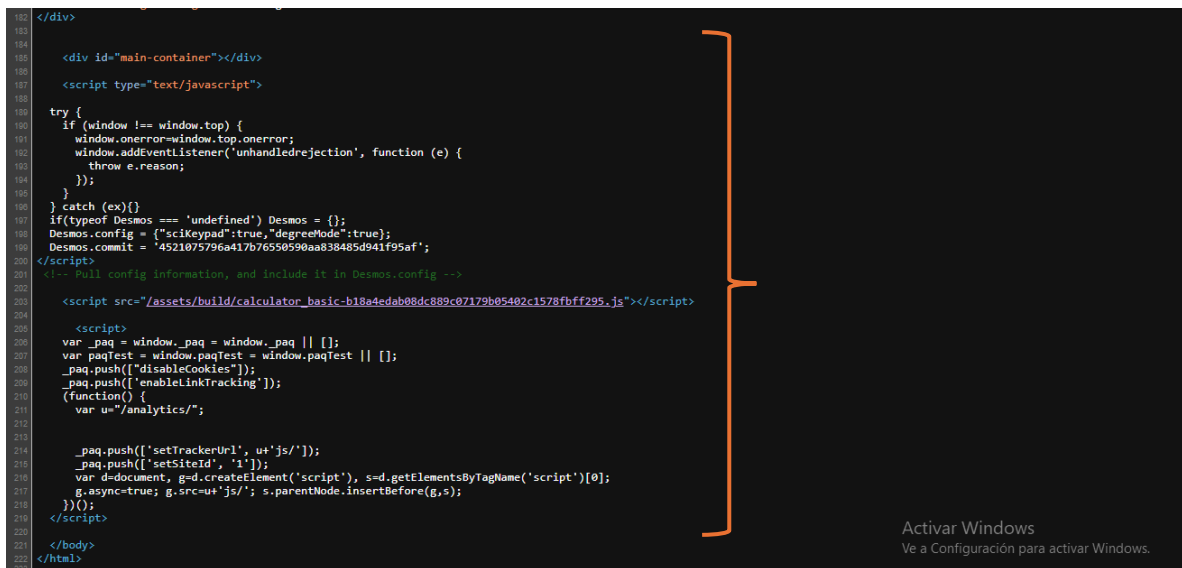
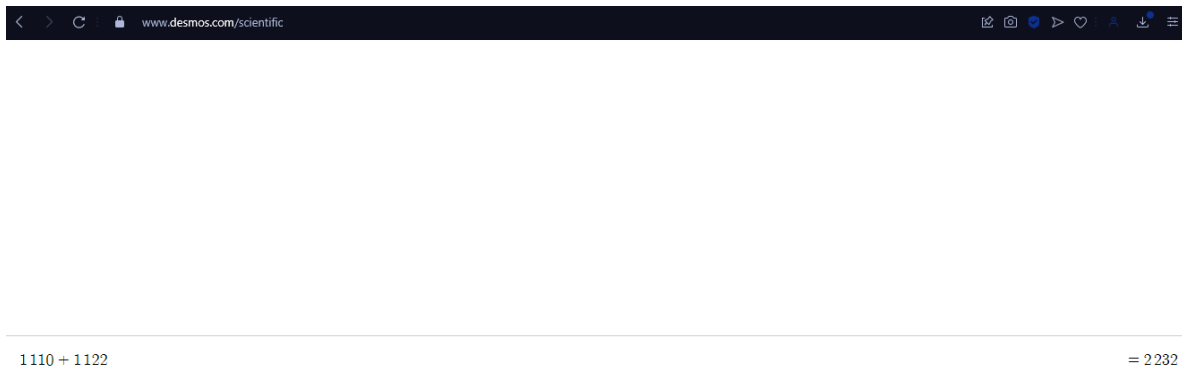
Ventajas

- Permite ser eficaz insertando, localizando y borrando.

Desventajas

- Al ser Personalizado tanto, es imposible el reutilizarlo en casos distintos

Ejemplo. Calculadora de pag web



[illegible]

Árboles Multi-caminos

Un árbol multi-camino es una estructura de datos en la que cada nodo de información tiene un máximo de g hijos, donde g es un valor mayor a dos.

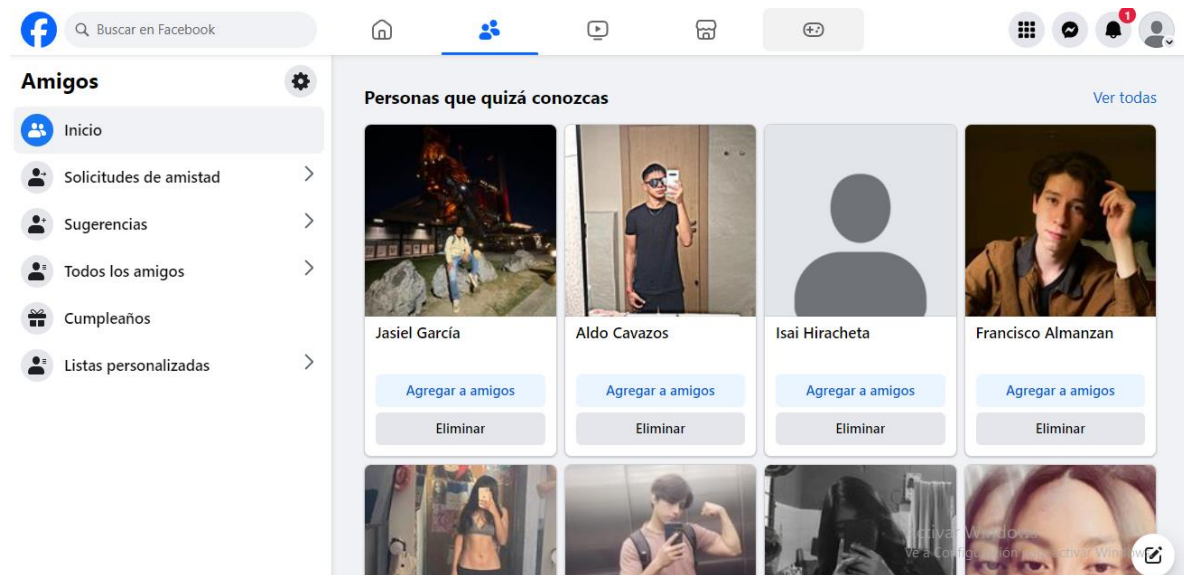
Ventajas

Existen más nodos en un mismo nivel que en los árboles binarios con lo que se consigue que, si el árbol es de búsqueda, los accesos a los nodos sean más rápidos.

Desventajas

Mayor ocupación de memoria

Ejemplo. Sugerencias de amistad de Facebook



Graficas

Un grafo es una estructura de datos que representa un conjunto de objetos o entidades físicas que están relacionadas entre sí. Los grafos son útiles para modelar problemas y tienen aplicaciones en redes sociales y en el modelado de máquinas de estado

- **Nodos o vértices:** Son los puntos que almacenan los objetos de datos. Cada nodo puede tener un número ilimitado de relaciones y tipos.
- **Aristas o arcos:** Son las líneas que conectan los nodos. Cada arista tiene un nodo inicial y un nodo final, además de un tipo y una dirección.
- **Adyacencia:** Es la relación que existe entre dos aristas que comparten un vértice.
- **Incidencia:** Es cuando un vértice está unido a otro.
- **Grado:** Es el número de aristas que inciden en un vértice.

Ventajas

- La velocidad de búsqueda depende del número de relaciones, no del conjunto de datos.
- Los resultados se obtienen en tiempo real.
- La presentación de las relaciones es intuitiva y resumida.
- Las estructuras son flexibles y ágiles.

Desventajas

- Es difícil escalarlo, ya que está diseñado para arquitecturas con un solo servidor.
- No tiene un lenguaje de consulta consistente.

Graficas Dirigidas

Un grafo dirigido, también conocido como dígrafo, es un tipo de grafo en el que las aristas tienen un sentido definido:

- Los arcos se dirigen desde un nodo origen hasta un nodo destino.
- El nodo origen precede al nodo destino, y este sucede al origen.
- Los arcos de un grafo dirigido se representan gráficamente con flechas.

Los grafos dirigidos se diferencian de los grafos no dirigidos, en los que las aristas son relaciones simétricas y no apuntan en ningún sentido.

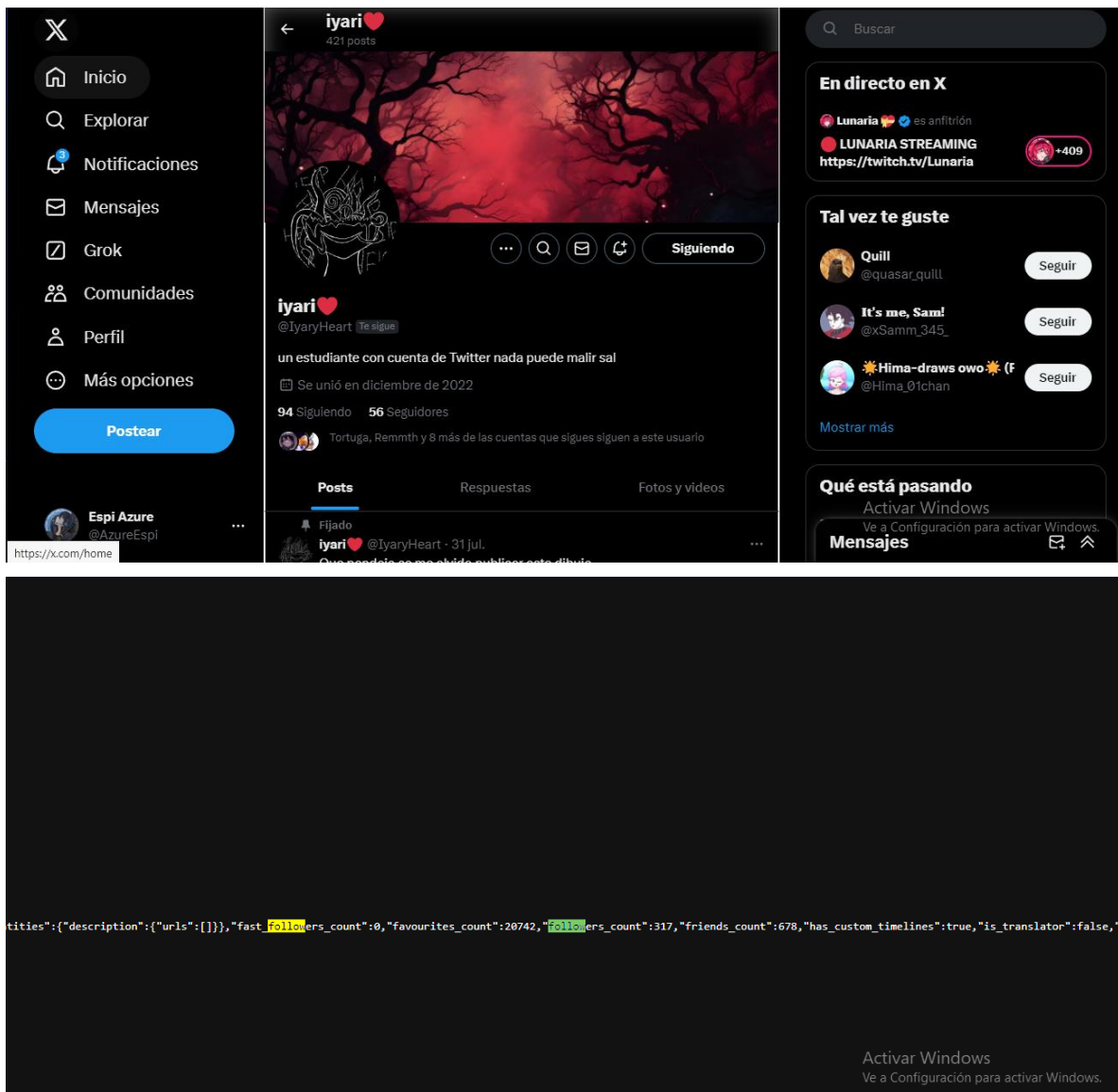
Ejemplo Seguir perfil de Twitter


```

< > 1 de 86
"entities": {}, "errors": {}, "fetchStatus": {}, "users": {
  "entities": {
    "1470575511453814785": {
      "following": false, "can_dm": true, "can_media_tag": true, "default_profile": true, "default_profile": true
    }
  }
}
```

Graficas No Dirigidas

Una gráfica no dirigida es un tipo de estructura de datos que representa relaciones simétricas entre entidades, denominadas nodos, mediante aristas que no tienen un sentido definido. Esto significa que cada arista se puede recorrer en ambas direcciones, indicando una relación bidireccional.



Métodos de Ordenación

Los métodos de ordenación consisten en reorganizar un conjunto de datos u objetos en una secuencia específica, de acuerdo con un criterio preestablecido, como el orden numérico o alfabético. Estos métodos son fundamentales en el procesamiento de la información, ya que facilitan su búsqueda, acceso y recuperación de manera eficiente. Ejemplos de la vida cotidiana incluyen directorios telefónicos, índices de libros y registros médicos, los cuales están ordenados para un fácil acceso y consulta. La ordenación es una tarea esencial para optimizar la gestión de grandes volúmenes de datos.

Ordenación Interna

La ordenación interna se refiere a los métodos de ordenación que trabajan con los datos almacenados en la memoria principal de la computadora, sin necesidad de utilizar memoria externa o arreglos auxiliares. Se clasifican en dos tipos principales: métodos directos y métodos logarítmicos. Los métodos directos, aunque son más sencillos de implementar, son menos eficientes para grandes conjuntos de datos, mientras que los logarítmicos, aunque más complejos, son más eficientes en términos de comparaciones y movimientos. La elección del método adecuado depende del tamaño del conjunto de datos a ordenar.

Ejemplos:

WhatsApp - Orden de chats: En WhatsApp, los chats están ordenados internamente en la aplicación según la fecha del último mensaje recibido o enviado. Esta ordenación ocurre directamente en la memoria del dispositivo, ya que no se necesita un acceso a servidores externos para organizar los mensajes.



Código: Cuando el usuario ingresa un nuevo chat y hace clic en el botón "Add Chat", se ejecuta la función `addChat`, que valida la entrada y agrega el nuevo chat al arreglo. Luego, la función `displayChats` actualiza la lista visible en la interfaz. Al hacer clic en el botón "Sort Chats", se invoca la función `sortChats`, que utiliza el método `sort()` de JavaScript para ordenar el arreglo en orden descendente basado en la propiedad de tiempo de cada objeto, asegurando que los chats más recientes se muestren primero. Finalmente, se vuelve a llamar a `displayChats` para reflejar la nueva ordenación en la interfaz, lo que permite al usuario ver fácilmente los chats más recientes. Este enfoque ilustra cómo funcionan los algoritmos de ordenación interna al manipular datos en memoria sin necesidad de acceder a un servidor externo.

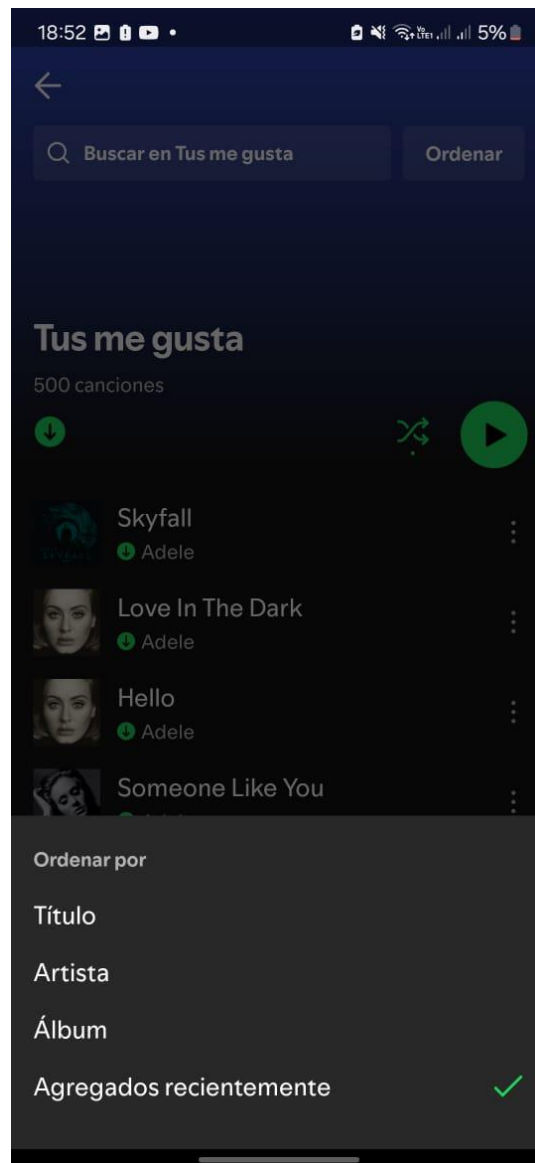
```
<h1>WhatsApp Chat Sorting Simulation</h1>
<input type="text" id="chatName" placeholder="Chat Name">
<input type="datetime-local" id="chatTime">
<button id="addChatButton">Add Chat</button>
<button id="sortChatsButton">Sort Chats</button>

<ul id="chatList"></ul>

<script>
  const chatList = document.getElementById('chatList');
  const addChatButton = document.getElementById('addChatButton');
  const sortChatsButton = document.getElementById('sortChatsButton');
  const chats = [];
  function addChat() {
    const chatName = document.getElementById('chatName').value;
    const chatTime = document.getElementById('chatTime').value;

    if (chatName && chatTime) {
      chats.push({ name: chatName, time: new Date(chatTime) });
      document.getElementById('chatName').value = '';
      document.getElementById('chatTime').value = '';
      displayChats();
    } else {
      alert('Please enter both chat name and time.');
```

Spotify - Ordenar canciones descargadas: Spotify permite ordenar las canciones descargadas por el usuario según criterios como título, artista o fecha de descarga. Este proceso de ordenación se realiza internamente dentro de la app, sin necesidad de conexión a servidores externos.

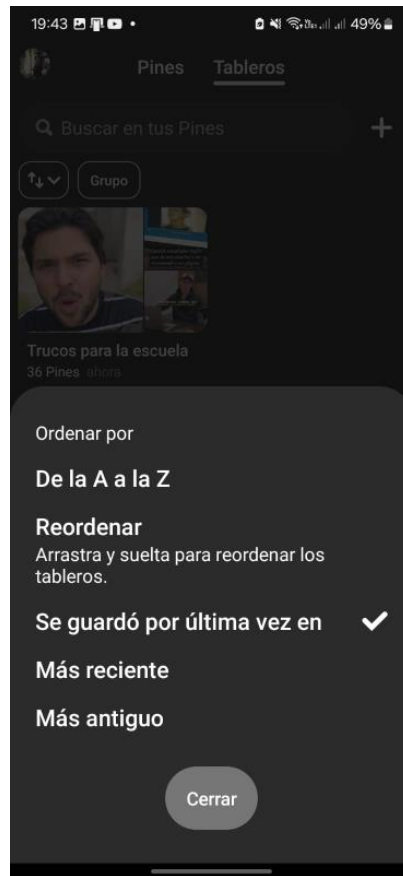


Código: Al hacer clic en el botón "Add Song", se valida la entrada y se añade un nuevo objeto de canción al arreglo. Luego, se pueden ordenar las canciones mediante botones que activan funciones específicas para clasificar el arreglo en función del título, el artista o la fecha de descarga, utilizando el método `sort()` de JavaScript. Cada vez que se realiza una ordenación, se actualiza la visualización de la lista de canciones en la interfaz, lo que permite al usuario ver los resultados de la ordenación de manera inmediata y sin necesidad de conexión a servidores externos.

```

<input type="text" id="songTitle" placeholder="Song Title">
<input type="text" id="songArtist" placeholder="Artist">
<input type="date" id="downloadDate">
<button id="addSongButton">Add Song</button>
<h2>Sort by:</h2>
<button id="sortByTitleButton">Title</button>
<button id="sortByArtistButton">Artist</button>
<button id="sortByDateButton">Download Date</button>
<ul id="songList"></ul>
<script>
  const songList = document.getElementById('songList');
  const addSongButton = document.getElementById('addSongButton');
  const sortByTitleButton = document.getElementById('sortByTitleButton');
  const sortByArtistButton = document.getElementById('sortByArtistButton');
  const sortByDateButton = document.getElementById('sortByDateButton');
  const songs = [];
  function addSong() {
    const songTitle = document.getElementById('songTitle').value;
    const songArtist = document.getElementById('songArtist').value;
    const downloadDate = document.getElementById('downloadDate').value;
    if (songTitle && songArtist && downloadDate) {
      songs.push({ title: songTitle, artist: songArtist, date: new Date(downloadDate) });
      document.getElementById('songTitle').value = '';
      document.getElementById('songArtist').value = '';
      document.getElementById('downloadDate').value = '';
      displaySongs();
    } else {
      alert('Please enter all fields.');
```

□ **Pinterest - Organizar tableros por fecha de creación:** Pinterest permite a los usuarios ordenar sus tableros de acuerdo con la fecha de creación o la última actualización. Este tipo de ordenación ayuda a los usuarios a gestionar mejor su contenido, permitiéndoles acceder a sus ideas más recientes con mayor facilidad.



Código: La ordenación se realiza mediante el botón "Creation Date", que activa la función `sortByDate`, utilizando el método `sort()` de JavaScript para organizar el arreglo de tableros en orden descendente según la fecha de creación. Cada vez que se ordenan los tableros, la lista se actualiza automáticamente en la interfaz, facilitando el acceso a los tableros más recientes y mejorando la gestión del contenido por parte del usuario.

```

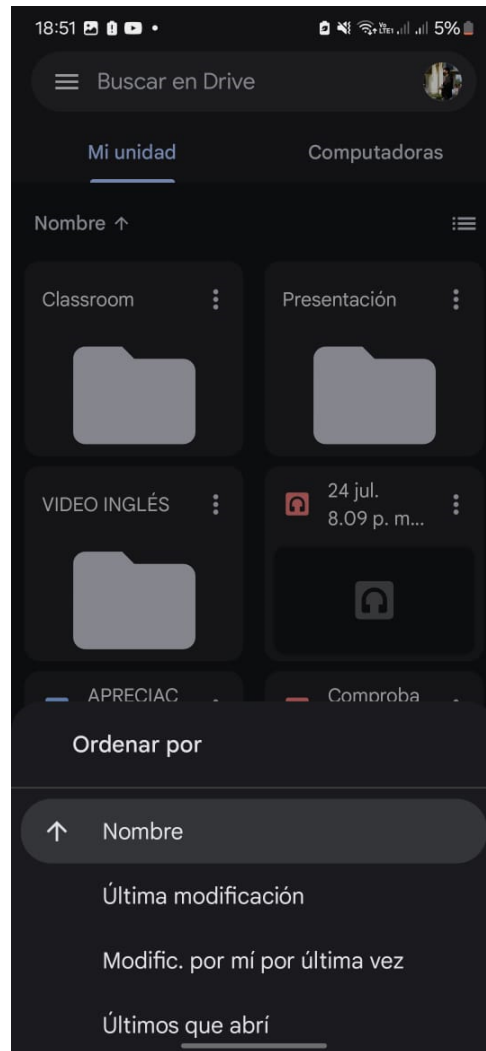
const boardList = document.getElementById('boardList');
const addBoardButton = document.getElementById('addBoardButton');
const sortByDateButton = document.getElementById('sortByDateButton');
const boards = [];
function addBoard() {
  const boardTitle = document.getElementById('boardTitle').value;
  const creationDate = document.getElementById('creationDate').value;
  if (boardTitle && creationDate) {
    boards.push({ title: boardTitle, date: new Date(creationDate) });
    document.getElementById('boardTitle').value = '';
    document.getElementById('creationDate').value = '';
    displayBoards();
  } else {
    alert('Please enter all fields.');
```

Ordenación Externa

La ordenación externa se utiliza cuando el volumen de datos es tan grande que no puede almacenarse completamente en la memoria principal de la computadora. Este proceso se realiza en dispositivos de almacenamiento secundario, como discos o cintas, y suele requerir técnicas como la fusión o mezcla de secuencias ordenadas. La ordenación externa es crucial en sistemas donde se manejan grandes volúmenes de datos, como bases de datos corporativas o archivos de instituciones gubernamentales, donde la eficiencia es clave para reducir el tiempo de acceso.

Ejemplos:

□ **Google Drive - Ordenar archivos en la nube:** En Google Drive, los usuarios pueden ordenar sus archivos almacenados en la nube por nombre, fecha de modificación o tamaño. Este tipo de ordenación es externa, ya que los datos se encuentran en servidores de Google y no en el dispositivo local.



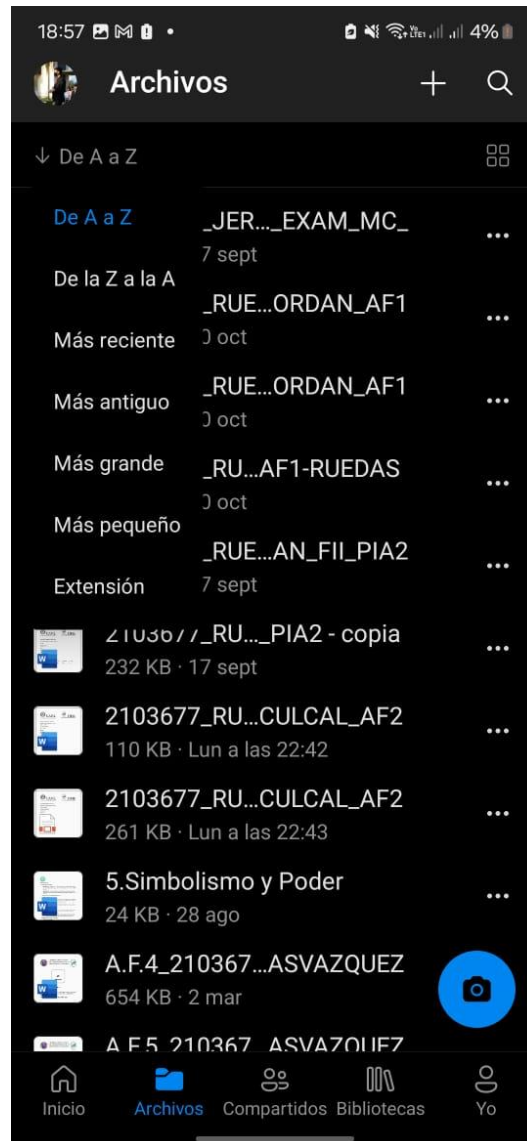
Código: La ordenación se realiza mediante botones específicos para cada criterio: "Name", "Modification Date" y "Size", que activan funciones que utilizan el método `sort()` de JavaScript para organizar el arreglo de archivos según el criterio seleccionado. Cada vez que se ordenan los archivos, la lista se actualiza automáticamente en la interfaz, reflejando los cambios y facilitando la gestión del contenido almacenado en la nube.

```

<h1>Google Drive File Sorting Simulation</h1>
<input type="text" id="fileName" placeholder="File Name">
<input type="date" id="modificationDate">
<input type="number" id="fileSize" placeholder="File Size (KB)">
<button id="addFileButton">Add File</button>
<h2>Sort by:</h2>
<button id="sortByNameButton">Name</button>
<button id="sortByDateButton">Modification Date</button>
<button id="sortBySizeButton">Size</button>
<ul id="fileList"></ul>
<script>
  const fileList = document.getElementById('fileList');
  const addFileButton = document.getElementById('addFileButton');
  const sortByNameButton = document.getElementById('sortByNameButton');
  const sortByDateButton = document.getElementById('sortByDateButton');
  const sortBySizeButton = document.getElementById('sortBySizeButton');
  const files = [];
  function addFile() {
    const fileName = document.getElementById('fileName').value;
    const modificationDate = document.getElementById('modificationDate').value;
    const fileSize = document.getElementById('fileSize').value;

    if (fileName && modificationDate && fileSize) {
      files.push({ name: fileName, date: new Date(modificationDate), size: parseInt(fileSize) });
      document.getElementById('fileName').value = '';
      document.getElementById('modificationDate').value = '';
      document.getElementById('fileSize').value = '';
      displayFiles();
    } else {
      alert('Please enter all fields.');
```

□ **OneDrive - Organizar documentos compartidos:** En Dropbox, los usuarios pueden organizar sus archivos por diferentes criterios, como nombre, tipo de archivo o tamaño. Este proceso es un ejemplo de ordenación externa, ya que se gestiona desde servidores de OneDrive y no en la memoria del dispositivo del usuario.



Código: La ordenación se realiza mediante botones específicos que permiten organizar los documentos por nombre, tipo de archivo o tamaño, utilizando el método `sort()` de JavaScript para modificar el arreglo según el criterio seleccionado. Cada vez que se ordenan los documentos, la lista se actualiza automáticamente en la interfaz, reflejando así la gestión de los archivos almacenados en la nube.

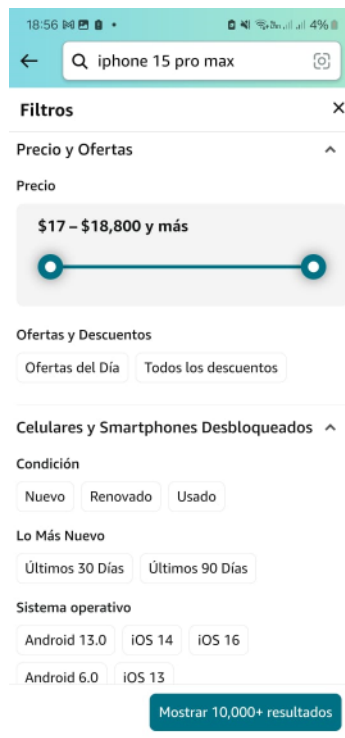
```

function displayDocuments() {
  documentList.innerHTML = '';
  documents.forEach(doc => {
    const li = document.createElement('li');
    li.textContent = `${doc.name} - Type: ${doc.type} - Size: ${doc.size} KB`;
    documentList.appendChild(li);
  });
}
function sortByName() {
  documents.sort((a, b) => a.name.localeCompare(b.name));
  displayDocuments();
}
function sortByType() {
  documents.sort((a, b) => a.type.localeCompare(b.type));
  displayDocuments();
}
function sortBySize() {
  documents.sort((a, b) => b.size - a.size);
  displayDocuments();
}
addDocumentButton.addEventListener('click', addDocument);
sortByNameButton.addEventListener('click', sortByName);
sortByTypeButton.addEventListener('click', sortByType);
sortBySizeButton.addEventListener('click', sortBySize);

```

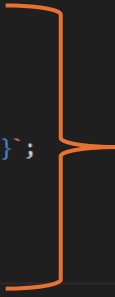
Ejemplos:

□ **Amazon - Ordenar productos por precio:** En Amazon, los usuarios pueden ordenar los productos por precio de menor a mayor. Este es un claro ejemplo de ordenación, ya que el sistema reorganiza los elementos del catálogo según el criterio de costo, lo cual es útil para aquellos que buscan productos accesibles o de mayor valor.



Código: La ordenación se realiza mediante el botón "Sort by Price", que utiliza el método `sort()` de JavaScript para reorganizar el arreglo de productos en función de su precio, de menor a mayor. Cada vez que se ordenan los productos, la lista en la interfaz se actualiza automáticamente, reflejando así la gestión de los elementos en un catálogo de productos.

```
const productList = document.getElementById('productList');
const addProductButton = document.getElementById('addProductButton');
const sortByPriceButton = document.getElementById('sortByPriceButton');
const products = [];
function addProduct() {
  const productName = document.getElementById('productName').value;
  const productPrice = document.getElementById('productPrice').value;
  if (productName && productPrice) {
    products.push({ name: productName, price: parseFloat(productPrice) });
    document.getElementById('productName').value = '';
    document.getElementById('productPrice').value = '';
    displayProducts();
  } else {
    alert('Please enter all fields.');
```



```
  }
}
function displayProducts() {
  productList.innerHTML = '';
  products.forEach(product => {
    const li = document.createElement('li');
    li.textContent = `${product.name} - Price: ${product.price.toFixed(2)}`;
    productList.appendChild(li);
  });
}
function sortByPrice() {
  products.sort((a, b) => a.price - b.price);
  displayProducts();
}
addProductButton.addEventListener('click', addProduct);
sortByPriceButton.addEventListener('click', sortByPrice);
```

Métodos de Búsqueda

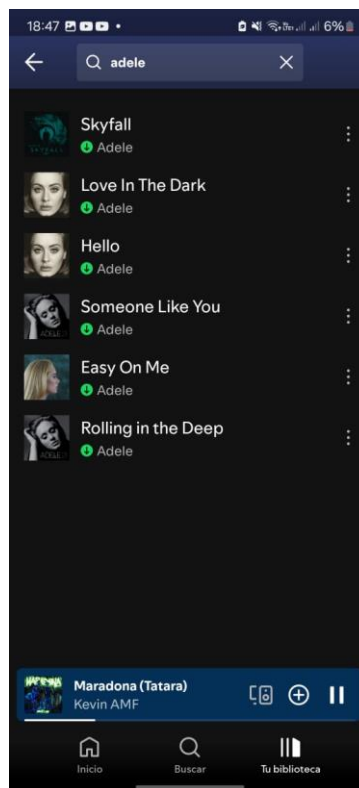
Los métodos de búsqueda son procedimientos utilizados para localizar un elemento dentro de una estructura de datos. Son una parte esencial del procesamiento de información, ya que permiten recuperar datos almacenados previamente. Estos métodos se emplean en una amplia variedad de situaciones cotidianas, como la búsqueda de números telefónicos, productos en línea o registros en bases de datos. La eficiencia de un método de búsqueda está determinada por el tiempo necesario para localizar el dato deseado dentro de un conjunto de elementos.

Búsqueda Interna

La búsqueda interna trabaja con los datos que se encuentran almacenados en la memoria principal de la computadora. Los elementos pueden estar en estructuras de datos estáticas, como arreglos, o dinámicas, como listas ligadas o árboles. Los métodos más comunes de búsqueda interna incluyen la búsqueda secuencial, la búsqueda binaria, la búsqueda por transformación de claves y los árboles de búsqueda. Cada método tiene sus propias ventajas y desventajas, dependiendo del tipo y tamaño de los datos.

Ejemplos:

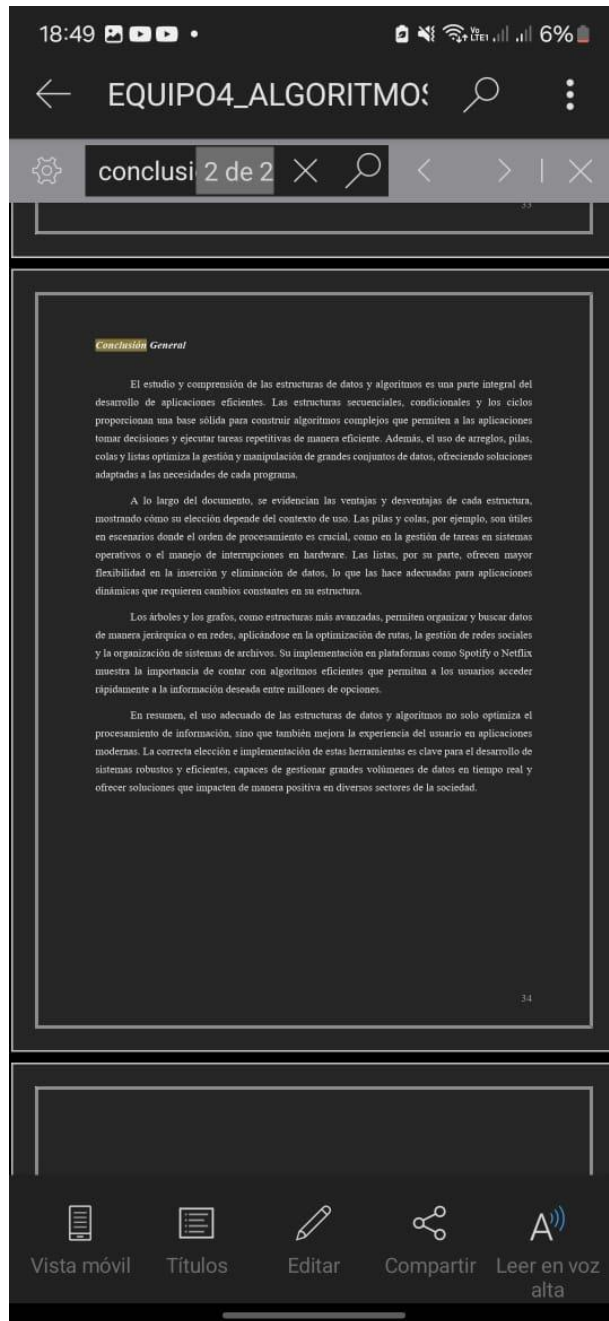
Spotify - Buscar canciones descargadas: Spotify permite a los usuarios buscar canciones que han descargado directamente en su dispositivo. Esta búsqueda interna no requiere conexión a internet, ya que los datos están localizados en la memoria del teléfono.



Código: La búsqueda se realiza en tiempo real a medida que el usuario escribe en el campo de búsqueda; la función `searchSongs` filtra el arreglo de canciones utilizando el método `filter()` para encontrar coincidencias que contengan el término de búsqueda, actualizando la lista mostrada en la interfaz con los resultados filtrados. Esto representa un algoritmo de búsqueda interna que opera sobre datos almacenados en memoria, sin necesidad de conexión a internet.

```
const songList = document.getElementById('songList');
const addSongButton = document.getElementById('addSongButton');
const searchInput = document.getElementById('searchInput');
const songs = [];
function addSong() {
  const songName = document.getElementById('songName').value;
  if (songName) {
    songs.push(songName);
    document.getElementById('songName').value = '';
    displaySongs(songs);
  } else {
    alert('Please enter a song name.');
```

Microsoft Word - Buscar palabras en un documento: En Microsoft Word, los usuarios pueden buscar una palabra específica dentro de un documento, utilizando la función de búsqueda interna. Esto se realiza en la memoria del dispositivo, sin necesidad de conexión a internet.



Código: Cuando el usuario presiona el botón "Search", la función `searchWord` utiliza una expresión regular para buscar coincidencias de la palabra ingresada en el texto del documento, ignorando mayúsculas y minúsculas. Si se encuentran coincidencias, se muestra cuántas veces aparece la palabra; de lo contrario, se informa que no hay coincidencias. Este proceso representa un algoritmo de búsqueda que opera directamente sobre los datos almacenados en memoria, sin necesidad de conexión a internet.

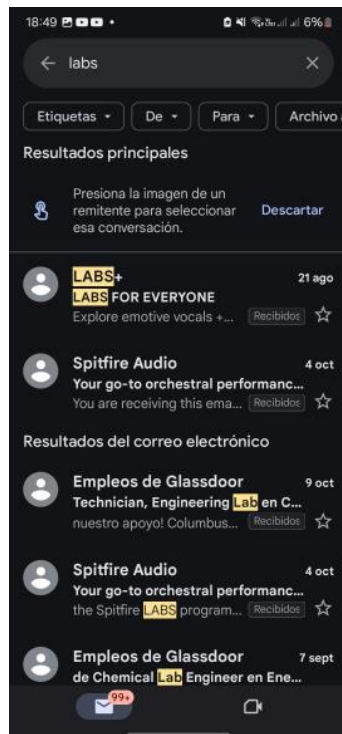

```

<textarea id="documentContent" placeholder="Type or paste your document content here..."></textarea>
<input type="text" id="searchInput" placeholder="Search for a word...">
<button id="searchButton">Search</button>
<div id="searchResults"></div>
<script>
  const searchButton = document.getElementById('searchButton');
  const searchInput = document.getElementById('searchInput');
  const documentContent = document.getElementById('documentContent');
  const searchResults = document.getElementById('searchResults');
  function searchWord() {
    const content = documentContent.value;
    const searchTerm = searchInput.value.trim();
    if (searchTerm === "") {
      alert("Please enter a word to search.");
      return;
    }
    const regex = new RegExp(searchTerm, 'gi');
    const matches = content.match(regex);
    if (matches) {
      searchResults.innerHTML = `Found "${searchTerm}" ${matches.length} time(s).`;
    } else {
      searchResults.innerHTML = `No matches found for "${searchTerm}";`;
    }
  }
}

```

Ejemplos:

Gmail - Búsqueda de correos electrónicos: Gmail permite buscar correos electrónicos mediante palabras clave, el nombre del remitente o una fecha específica. Esto facilita la localización de mensajes importantes en bandejas de entrada con muchos correos almacenados.



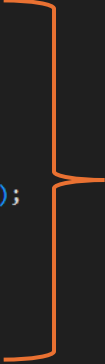
Código: Al introducir una palabra clave, nombre del remitente o fecha en el campo de búsqueda y presionar el botón "Search", la función searchEmails divide el contenido en líneas individuales y verifica si la búsqueda coincide con alguna parte de los correos ingresados. Si se encuentran coincidencias, se muestran los correos relevantes; de lo contrario, se notifica que no hay resultados. Este proceso representa un algoritmo de

búsqueda que opera en memoria, facilitando la localización de mensajes importantes sin necesidad de conexión a internet.

```
const searchButton = document.getElementById('searchButton');
const searchInput = document.getElementById('searchInput');
const emailContent = document.getElementById('emailContent');
const searchResults = document.getElementById('searchResults');
function searchEmails() {
    const content = emailContent.value.trim().split('\n');
    const searchTerm = searchInput.value.trim().toLowerCase();
    let results = [];

    if (searchTerm === "") {
        alert("Please enter a keyword, sender, or date to search.");
        return;
    }

    content.forEach(email => {
        if (email.toLowerCase().includes(searchTerm)) {
            results.push(email);
        }
    });
    if (results.length > 0) {
        searchResults.innerHTML = "Found the following emails:<br>" + results.join('<br>');
    } else {
        searchResults.innerHTML = `No emails found for "${searchTerm}"`;
    }
    searchResults.style.display = 'block';
}
```

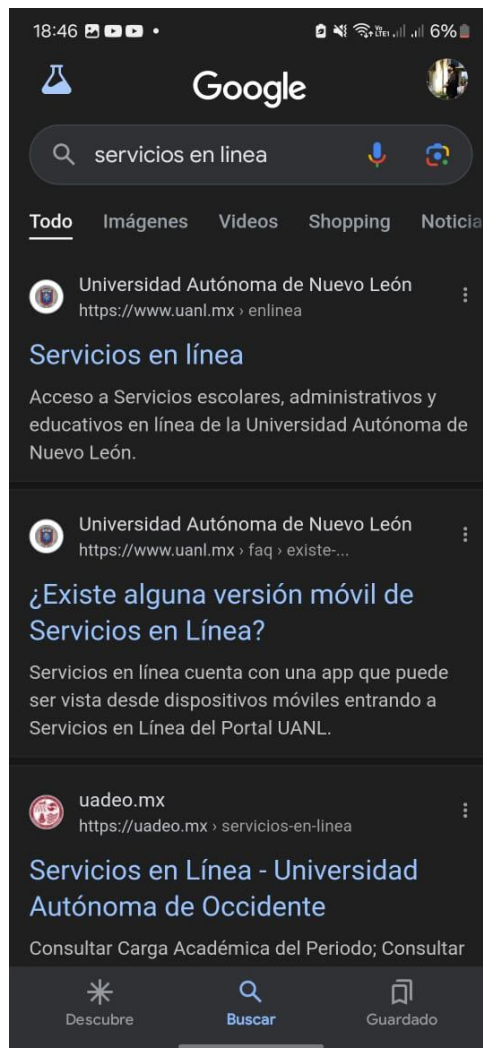


Búsqueda Externa

La búsqueda externa se refiere a los métodos de búsqueda que se utilizan cuando los datos no pueden ser almacenados completamente en la memoria principal y se encuentran en dispositivos de almacenamiento secundario, como archivos en discos o cintas. Este tipo de búsqueda es común en aplicaciones donde se maneja un gran volumen de información o cuando los datos deben conservarse incluso después de que el programa haya finalizado. Dado que las operaciones de lectura y escritura en dispositivos externos son más costosas en términos de tiempo, la optimización de estos procesos es fundamental para mejorar la eficiencia del sistema.

Ejemplos:

Google - Búsqueda de sitios web: Cuando los usuarios realizan búsquedas en Google, están accediendo a un enorme volumen de datos almacenados en servidores distribuidos por todo el mundo. La búsqueda externa en Google es uno de los ejemplos más comunes y efectivos de esta técnica.



Código: Al escribir una consulta en el campo de búsqueda y hacer clic en el botón "Search", la función `searchWeb` recorre un array de objetos que representan sitios web, verificando si el término de búsqueda coincide con los títulos o descripciones de estos sitios. Si se encuentran coincidencias, se generan enlaces a los sitios relevantes y se muestran en el área de resultados; de lo contrario, se informa que no hay resultados. Este proceso imita cómo funcionan los motores de búsqueda como Google, que acceden a grandes volúmenes de datos distribuidos para proporcionar resultados relevantes a los usuarios.

```
const websites = [
  { title: "OpenAI", url: "https://www.openai.com", description: "AI research and deployment company." },
  { title: "Google", url: "https://www.google.com", description: "Search the world's information." },
  { title: "Wikipedia", url: "https://www.wikipedia.org", description: "Free encyclopedia that anyone can edit." },
  { title: "GitHub", url: "https://www.github.com", description: "Platform for version control and collaboration." },
  { title: "Stack Overflow", url: "https://stackoverflow.com", description: "Question and answer site for programmers." },
  { title: "YouTube", url: "https://www.youtube.com", description: "Video sharing platform." },
  { title: "Twitter", url: "https://www.twitter.com", description: "Social media platform for short messages." },
  { title: "Facebook", url: "https://www.facebook.com", description: "Social networking site." }
];

function searchWeb() {
  const searchTerm = searchInput.value.trim().toLowerCase();
  let results = [];

  if (searchTerm === "") {
    alert("Please enter a search term.");
    return;
  }

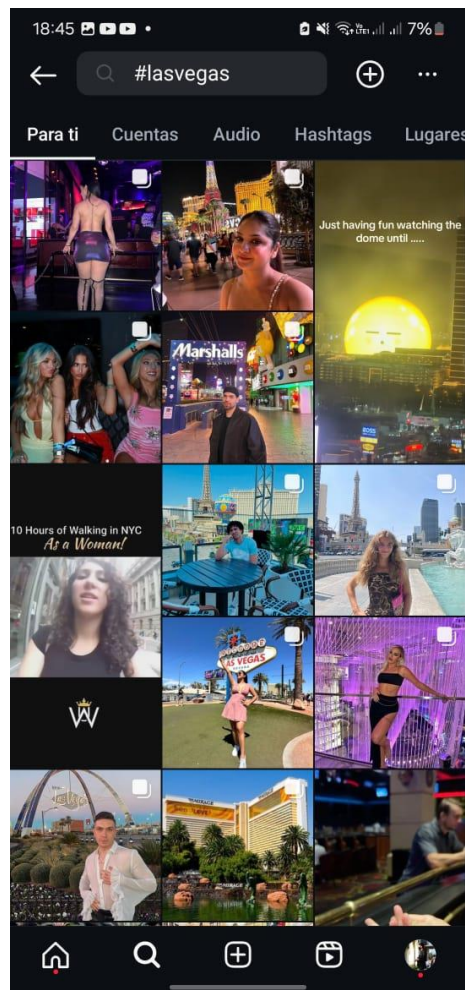
  websites.forEach(website => {
    if (website.title.toLowerCase().includes(searchTerm) ||
        website.description.toLowerCase().includes(searchTerm)) {
      results.push(`<a href="${website.url}" target="_blank">${website.title}</a> - ${website.description}`);
    }
  });

  if (results.length > 0) {
    searchResults.innerHTML = "Search Results:<br>" + results.join('<br>');
  } else {
    searchResults.innerHTML = `No results found for "${searchTerm}";`;
  }

  searchResults.style.display = 'block';
}

searchButton.addEventListener('click', searchWeb);
```

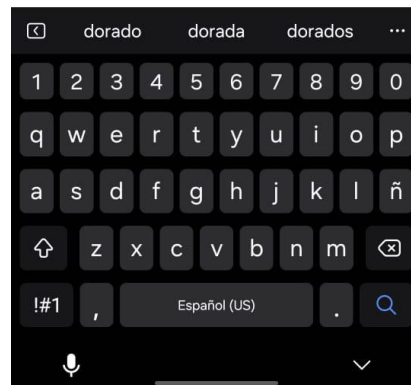
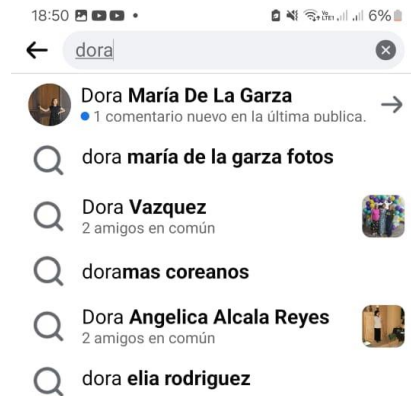
Instagram - Buscar hashtags o perfiles: En Instagram, los usuarios pueden buscar hashtags o perfiles específicos. Esta búsqueda es externa, ya que se realiza en los servidores de Instagram, que contienen millones de publicaciones y cuentas almacenadas en la plataforma.



Código: Cuando el usuario ingresa un término en el campo de búsqueda y hace clic en el botón "Search", la función `searchInstagram` envía una solicitud a la API utilizando el término proporcionado, recuperando las publicaciones más recientes asociadas a ese hashtag. Los resultados se muestran en la página, presentando imágenes de las publicaciones y enlaces a ellas, lo que refleja cómo los motores de búsqueda acceden a datos externos para ofrecer información relevante a los usuarios.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Instagram Search</title>
<script>
  async function searchInstagram() {
    const query = document.getElementById('searchQuery').value;
    const response = await fetch('https://api.instagram.com/v1/tags/${query}/media/recent?access_token=YOUR_ACCESS_TOKEN');
    const data = await response.json();
    const resultsContainer = document.getElementById('results');
    resultsContainer.innerHTML = '';
    if (data.data.length > 0) {
      data.data.forEach(post => {
        const postElement = document.createElement('div');
        postElement.innerHTML = `<a href="${post.link}" target="_blank">
```

Facebook - Buscar amigos por nombre: Facebook utiliza un método de búsqueda que permite a los usuarios encontrar a otras personas por su nombre. Esta búsqueda es secuencial, ya que la plataforma compara el nombre ingresado con todos los perfiles en su base de datos.



Código: Cuando el usuario ingresa un nombre y hace clic en el botón "Search", la función `searchFriends` compara el nombre ingresado con los nombres en la lista de amigos, realizando una búsqueda secuencial. Los resultados que coinciden se muestran dinámicamente en la página, lo que ilustra cómo un sistema puede acceder y filtrar datos externos para proporcionar información relevante al usuario.

```
<h1>Search for Friends on Facebook</h1>
<input type="text" id="friendName" placeholder="Enter friend's name">
<button onclick="searchFriends()">Search</button>
<div class="results" id="results"></div>
<script>
  function searchFriends() {
    const name = document.getElementById('friendName').value;
    const resultsDiv = document.getElementById('results');
    resultsDiv.innerHTML = '';
    const friendsList = [
      { name: 'John Doe', id: 1 },
      { name: 'Jane Smith', id: 2 },
      { name: 'Alice Johnson', id: 3 }
    ];
    const filteredFriends = friendsList.filter(friend => friend.name.toLowerCase().includes(name.toLowerCase()));
    if (filteredFriends.length > 0) {
      filteredFriends.forEach(friend => {
        const friendDiv = document.createElement('div');
        friendDiv.textContent = friend.name;
        resultsDiv.appendChild(friendDiv);
      });
    }
  }
</script>
```

El estudio y comprensión de las estructuras de datos y algoritmos es una parte integral del desarrollo de aplicaciones eficientes. Las estructuras secuenciales, condicionales y los ciclos proporcionan una base sólida para construir algoritmos complejos que permiten a las aplicaciones tomar decisiones y ejecutar tareas repetitivas de manera eficiente. Además, el uso de arreglos, pilas, colas y listas optimiza la gestión y manipulación de grandes conjuntos de datos, ofreciendo soluciones adaptadas a las necesidades de cada programa.

A lo largo del documento, se evidencian las ventajas y desventajas de cada estructura, mostrando cómo su elección depende del contexto de uso. Las pilas y colas, por ejemplo, son útiles en escenarios donde el orden de procesamiento es crucial, como en la gestión de tareas en sistemas operativos o el manejo de interrupciones en hardware. Las listas, por su parte, ofrecen mayor flexibilidad en la inserción y eliminación de datos, lo que las hace adecuadas para aplicaciones dinámicas que requieren cambios constantes en su estructura.

Los árboles y los grafos, como estructuras más avanzadas, permiten organizar y buscar datos de manera jerárquica o en redes, aplicándose en la optimización de rutas, la gestión de redes sociales y la organización de sistemas de archivos. Su implementación en plataformas como Spotify o Netflix muestra la importancia de contar con algoritmos eficientes que permitan a los usuarios acceder rápidamente a la información deseada entre millones de opciones.

En resumen, el uso adecuado de las estructuras de datos y algoritmos no solo optimiza el procesamiento de información, sino que también mejora la experiencia del usuario en aplicaciones modernas. La correcta elección e implementación de estas herramientas es clave para el desarrollo de sistemas robustos y eficientes, capaces de gestionar grandes volúmenes de datos en tiempo real y ofrecer soluciones que impacten de manera positiva en diversos sectores de la sociedad.

Programación Orientada a Objetos

¿Qué es la programación?

La programación es el proceso de diseñar y codificar un conjunto de instrucciones que una computadora puede ejecutar para resolver problemas, automatizar tareas o crear sistemas funcionales. Este proceso implica el uso de lenguajes de programación, los cuales actúan como un puente entre el pensamiento humano y la lógica computacional. Cada lenguaje, como C, Python o Java, tiene su propia sintaxis y reglas, adaptándose a diferentes tipos de problemas y plataformas. En esencia, la programación busca traducir ideas complejas en algoritmos que una máquina pueda procesar.

Además de ser una disciplina técnica, la programación es un ejercicio de lógica y creatividad. Diseñar un programa no solo requiere conocimientos técnicos, sino también la capacidad de analizar problemas, dividirlos en partes más manejables y encontrar soluciones eficientes. Por ello, aprender a programar desarrolla habilidades críticas como el pensamiento estructurado y la capacidad de resolver problemas, competencias fundamentales en áreas como la ingeniería y la ciencia de datos.

En la actualidad, la programación se ha convertido en una habilidad esencial en prácticamente todas las industrias, desde la salud hasta la manufactura. Su impacto es innegable, ya que impulsa avances tecnológicos, mejora procesos y facilita la innovación. Es una herramienta poderosa que permite a los profesionales interactuar con sistemas tecnológicos complejos y desarrollar soluciones adaptadas a las necesidades contemporáneas.

¿Qué es la programación orientada a objetos?

La programación orientada a objetos (POO) es un paradigma de programación que organiza los programas en torno a objetos, los cuales representan entidades del mundo real con propiedades (atributos) y comportamientos (métodos). Este enfoque permite modelar sistemas complejos de manera más natural y eficiente, haciendo que el código sea modular, reutilizable y más fácil de mantener. Entre los lenguajes más populares que emplean la POO se encuentran Java, C++ y Python.

El concepto central de la POO es el objeto, que combina datos y funciones en una sola unidad. Los objetos se crean a partir de plantillas llamadas clases, las cuales definen las características y comportamientos comunes de un conjunto de objetos similares. Por ejemplo, en un sistema de gestión de estudiantes, una clase “Estudiante” puede incluir atributos como nombre y matrícula, y métodos como `inscribirCurso()` o `calcularPromedio()`.

La POO ha transformado el desarrollo de software al introducir principios como encapsulación, herencia y polimorfismo. Estos principios facilitan la creación de sistemas escalables y adaptables, reduciendo la complejidad del desarrollo. Por ejemplo, en un juego de simulación, diferentes personajes pueden compartir comportamientos comunes a través de herencia, mientras que polimorfismo permite personalizar sus acciones específicas sin alterar la estructura general del código.

Características de la programación orientada a objetos

La programación orientada a objetos se caracteriza por cuatro pilares fundamentales: encapsulación, herencia, polimorfismo y abstracción.

1. **Encapsulación:** Permite que los datos y los métodos relacionados se agrupen en una sola unidad llamada clase. Esto protege los datos de accesos no autorizados y establece un control sobre cómo los métodos externos interactúan con ellos. Por ejemplo, en una clase "CuentaBancaria", el atributo "saldo" puede estar protegido para que solo sea accesible mediante los métodos "depositar" o "retirar".
2. **Herencia:** Facilita la reutilización del código al permitir que una clase hija derive de una clase padre, heredando sus atributos y métodos. Esto evita la duplicación de código y simplifica las modificaciones futuras. Por ejemplo, una clase "Empleado" puede servir como base para las clases "Gerente" y "Técnico", que pueden agregar sus propias características.
3. **Polimorfismo:** Permite que un método o función tome diferentes formas dependiendo del contexto. Esto se logra mediante la sobrecarga y la sobrescritura de métodos. Por ejemplo, un método "calcularPago" puede comportarse de manera diferente para empleados de tiempo completo y contratistas, dependiendo de la clase específica a la que pertenezcan.
4. **Abstracción:** Consiste en definir las características esenciales de un objeto, ocultando los detalles innecesarios. Por ejemplo, una clase abstracta "FiguraGeometrica" puede tener métodos genéricos como "calcularArea", que son implementados por sus subclases "Círculo" o "Cuadrado".

Ejemplo o aplicación de las características de la programación orientada a objetos

Un ejemplo común de la POO se encuentra en el desarrollo de sistemas de gestión escolar. Imaginemos una aplicación que gestiona estudiantes, profesores y cursos.

- **Encapsulación:** La clase "Estudiante" puede tener atributos como "nombre", "matrícula" y "calificaciones". Estos datos están protegidos mediante métodos como "obtenerNombre" y "establecerCalificaciones", asegurando que las calificaciones solo se actualicen mediante métodos controlados.
- **Herencia:** Las clases "Estudiante" y "Profesor" pueden derivarse de una clase base "Persona", heredando atributos como "nombre" y "dirección", y métodos como "registrar".
- **Polimorfismo:** Un método "calcularPromedio" puede comportarse de manera diferente dependiendo del tipo de estudiante. Por ejemplo, para estudiantes regulares puede basarse en exámenes, mientras que para estudiantes de posgrado puede incluir proyectos y publicaciones.
- **Abstracción:** Una clase abstracta "Usuario" puede definir métodos generales como "iniciarSesión", mientras que las clases "Administrador" y "Alumno" implementan estos métodos de manera específica.

¿Qué son los diagramas UML?

Los diagramas UML (Lenguaje Unificado de Modelado, por sus siglas en inglés) son una representación gráfica estándar utilizada en la ingeniería de software para visualizar, especificar y documentar sistemas. UML permite a los desarrolladores diseñar la estructura y el comportamiento de un sistema de manera clara y precisa, sirviendo como una guía para el desarrollo y la comunicación entre equipos. Existen diferentes tipos de diagramas UML, como diagramas de clases, de secuencia, de casos de uso y de actividad, cada uno enfocado en distintos aspectos del sistema.

Por ejemplo, el diagrama de clases es útil para modelar la estructura del sistema al mostrar las clases, sus atributos, métodos y relaciones. Por otro lado, los diagramas de secuencia detallan la interacción entre objetos a lo largo del tiempo, facilitando la comprensión del flujo de procesos. Los diagramas UML son ampliamente

utilizados en metodologías de desarrollo como Scrum o XP, donde la comunicación clara y precisa es fundamental.

La flexibilidad de UML lo hace aplicable en una variedad de contextos, desde aplicaciones pequeñas hasta sistemas empresariales complejos. Su uso mejora la eficiencia del equipo al reducir ambigüedades en el diseño y permitir una documentación robusta y estandarizada, esencial para proyectos de larga duración o con alta rotación de personal.

Conclusión General

La programación se ha vuelto un elemento importante de la sociedad en general y la digital, fomentando la innovación y cambiando la manera en que nos relacionamos con el mundo. Entender los conceptos básicos de programación, desde las variables y estructuras de control hasta la programación orientada a objetos y la gestión de datos, nos permite crear soluciones tecnológicas eficientes, fáciles, sólidas y flexibles. La actividad nos mencionó, como hacer un programa funcional en C y ver algoritmos mediante diagramas de flujo en RAPTOR, nos ayudó al reforzamiento de estos conceptos y nos ayudó a ver su uso práctico en la solución de problemas. Más allá de la sintaxis y la estructura del código, la programación fomenta habilidades de pensamiento crítico, lógica y resolución de problemas, que son útiles en cualquier área. Al aprender los fundamentos de la programación, no solo estamos preparados para enfrentar problemas complejos, sino que también desarrollamos un enfoque analítico y creativo que nos ayuda a innovar y a buscar soluciones efectivas en una área que está en constante cambio.

Conclusiones Individuales

Ruedas Vazquez Jordan Emanuel

Mi experiencia con la programación en C ha sido como aprender un nuevo idioma, con sus propias reglas y estructuras que requieren exactitud y práctica continua. Al examinar el programa, entendí la importancia de declarar variables y cómo las funciones `scanf` y `printf` permiten la interacción con el usuario. Este ejercicio no solo reafirmó mi comprensión de la estructura básica de un programa, sino que también me permitió ver cómo las estructuras de control y la lógica de programación se utilizan en diferentes lenguajes. Además, me llevó a reflexionar sobre la relevancia de la depuración y la exactitud en la escritura de código, habilidades que son esenciales para cualquier programador.

Enríquez Nungaray Héctor Miguel

Al volver hacer diagramas de flujo en RAPTOR fue una buena experiencia. Pude visualizar cada paso del proceso y entender cómo la lógica y las estructuras de datos se combinan para obtener un resultado específico. Esta actividad me enseñó que la programación no es solo escribir código, sino también planificar y diseñar soluciones de manera ordenada. Ahora aprecio aún más la importancia de la lógica en la programación y cómo las herramientas de desarrollo pueden facilitar la gestión de información.

Heredia López Betsy Aracely

La programación es una herramienta poderosa que nos permite solucionar problemas de manera ordenada y eficiente. Al trabajar con el diagrama de flujo en RAPTOR, pude ver cómo los algoritmos procesan entradas, manipulan datos y generan salidas. Este enfoque me ayudó a entender la importancia de la planificación en la programación y cómo cada paso debe estar claramente definido para lograr el resultado deseado. Además, este ejercicio me permitió explorar conceptos básicos como las estructuras de datos y las herramientas de programación, que son esenciales para crear aplicaciones sólidas en diferentes lenguajes.

Hernández Silva Carlos Yahir

Al estudiar el programa en C, me sorprendió cómo un código que parece simple puede realizar tareas útiles. Este ejercicio me hizo pensar en la importancia de la sintaxis y la precisión en la escritura de código, ya que un pequeño error puede afectar el funcionamiento de todo el programa. Además, reforzó mi comprensión de conceptos clave como las estructuras de control y la programación orientada a objetos, que se pueden aplicar en lenguajes como Python, C++ y Java. Este análisis también me permitió valorar la importancia de la depuración y la optimización del código.

Referencias Bibliográficas

- Cairo, O. (2024). *Algoritmos y estructuras de datos*. Editorial Universitaria.
- Gauchat, J. D. (2017). *El gran libro de HTML5, CSS3 y JavaScript*. Marcombo.
- Mozilla Developer Network. (2024). *JavaScript Guide*.
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>
- TypeScript Handbook. (2024). *Basic Types*. <https://www.typescriptlang.org/docs/handbook/basic-types.html>
- W3C. (2024). *HTML Specifications*. <https://www.w3.org/TR/html/>
- Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico (8ª ed.)*. McGraw-Hill.
- Sommerville, I. (2015). *Software Engineering (10th ed.)*. Pearson.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language (2nd ed.)*. Prentice Hall.
- Vázquez Gómez, J. B. (Año). *Análisis y diseño de algoritmos*. Red Tercer Milenio.
- Cairó Battistutti, O. (Año). *Fundamentos de programación: Piensa en C*. Pearson Educación.
- KeepCoding. (n.d.). *¿Qué son los arreglos en programación?* KeepCoding. Recuperado el 29 de agosto de 2024, de <https://keepcoding.io/blog/arreglos-en-programacion/>
- Hernández, L. (2018). *Metodología de análisis y diseño de sistemas de información*. Editorial Alfaomega.
- García, J. R. (2010). *Algoritmos y estructuras de datos: Un enfoque práctico (2ª ed.)*. McGraw-Hill Interamericana.
- Leandro Alegsa, (2023). *Definición de estructura de decisión*. ALEGSA.com.ar
- Walter Madrigal, (2020). *Estructuras de decisión*. San marcos.
- Helmut, (2020). *Algoritmo: características, para que sirve, partes, ejemplos*. Lifeder.
- Hernández, L. (2018). *Metodología de análisis y diseño de sistemas de información*. Editorial Alfaomega.
- Cairó, O. (2006). *** Fundamentos de programación. Piensa en C**. Pearson Educación. ISBN 970-260810-4.
- García, J. R. (2010). *Algoritmos y estructuras de datos: Un enfoque práctico (2ª ed.)*. McGraw-Hill Interamericana.
- Introduction to Algorithms" de Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, y Clifford Stein*
- Cairó Battistutti, O. (2006). *Fundamentos de programación. Piensa en C*. Pearson Educación. ISBN 970-26-0810-4

García Cano, E. E., Solano Gálvez, J. A., Sandoval Montaña, L., & Velázquez Mena, A. (2024) *Guía práctica de estudio 04: Diagramas de flujo*

De Recursividad es muy amplio leen relatos que describen relatos, E. C., De y, H. O. M.)) E. M. H. en la L. se A. M. a. U. P. U. E., Divertida, un T., & ver Recursividad, S. D. de R. D. (n.d.). 9.1. ¿Qué es la Recursividad? Uji.Es. Retrieved August 30, 2024, from <https://repositori.uji.es/xmlui/bitstream/handle/10234/168711/Tema9IS04.pdf?sequence=1&isAllowed=y>

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide (2nd Edition)*. Addison-Wesley.

Pressman, R. S. (2014). *Ingeniería del software: Un enfoque práctico (7ª ed.)*. McGraw-Hill Education.

SciELO. (n.d.). *Repositório científico sobre programação e sistemas de software*. Recuperado de <https://www.scielo.org>.

IEEE Computer Society. (2023). *Best Practices in Object-Oriented Programming*. Recuperado de <https://www.ieee.org>.